6/6/2018

מערכות הפעלה- תשע"ח

השימוש בסיכום מותנה בקריאת 3 מזמורי תהלים להצלחת חיילי ישראל.

מבוסס על ההרצאות ,
הספר וסיכומים שונים

גד כהן

GADIMAN@GMAIL.COM

תוכן עניינים

פרק 1: הקדמה ומושגים 1-5

פרק 2: מבנה מערכת ההפעלה 6-12

פרק 3-4: תהליכים ותהליכונים 13-26

פרק 5: תזמון ה-Cpu 27-38

פרק 6: סנכרון תהליכים 39-53

פרק 7: קיפאון 54-64

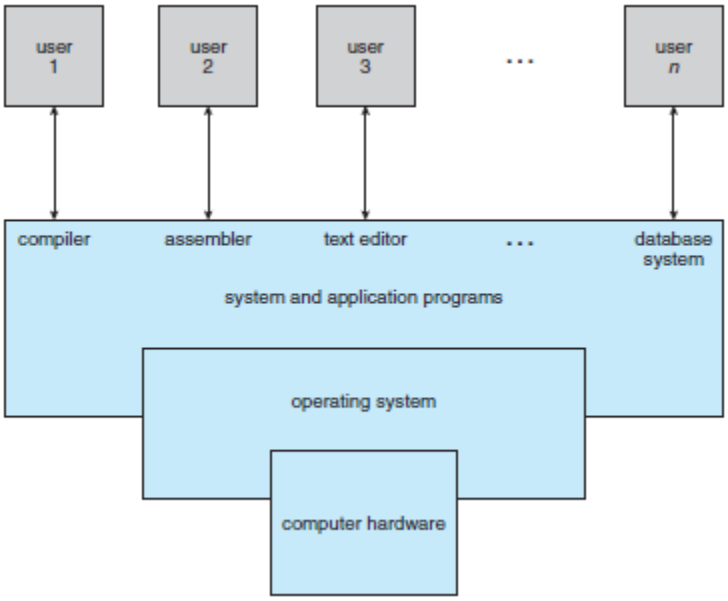
פרק 8: זיכרון ראשי 65-78

פרק 9: זיכרון וירטואלי 79-89

פרק 10: אחסון 90-94

פרק 11-12: ניהול קבצים 95-109

פרק 1: הקדמה ומושגים



קורס מערכות הפעלה מתחיל עם האיור הבא: ניתן לראות שהמלבנים מייצגים שכבות : המלבן הגדול הוא אזור התוכנות והאפליקציות, המלבן הבינוני הוא המלבן המאפיין את מערכת ההפעלה והמלבן הקטן זוהי החומרה של המחשב (מעבד, כרטיס קול , זיכרון וכו').

מערכת ההפעלה מגשרת בין המשתמשים השונים לבין החומרה.

נשים לב שכל אפליקציה או תוכנה מוגדרים כמשתמשים שונים.

ישנם מס הגדרות למערכת הפעלה:

1. תוכנית המקשרת בין המשתמש לבין החומרה של המחשב (משתמש [user]: אנושי, תוכנה או משאב אחר)
2. מנווטת את החומרה התוכנה והנתונים, תוך ניצול יעיל של מערכת המחשב, מחליטה בצורה הוגנת מה לעשות כאשר יש התנגשויות (כמה משתמשים רוצים את אותו רכיב חומרה, בד"כ cpu).

אמרנו שמערכת ההפעלה מנווטת את החומרה אז נכיר את 3 החלקים החשובים בחומרה:

1. **Cpu-** המעבד, אחד מהרכיבים היקרים במחשב, ולכן המטרה שלנו שהוא תמיד יעבוד ולא יצמא במצב סרק (idle) .
2. **Memory-** הזיכרון, הראשי, המשני, דיסק קשיח.
3. **I/O Structure-** מבני הקלט/ פלט של המחשב.

איך מערכת ההפעלה נטענת בעת הדלקת המחשב?!

ישנה תוכנית מיוחדת בשם "**bootstrap**" שהיא צרובה על זיכרון ה**Room** (זיכרון לא נדיף) של המחשב והיא אחראית "להרים" את מערכת ההפעלה כאשר המחשב נדלק.
איך זה קורה?! ע"י המנגנון **"System Boot"**

אתחול מערכת ההפעלה צרוב בזיכרון קבוע ולא נדיף (ROM) בתוך בלוק זיכרון שנקרא "**Boot Block**" , התוכנית "bootstrap" טוענת אותו ומריצה את מערכת ההפעלה (כמו סטרטר).

<u>פסיקה interrupt :</u>

הקורס ברובו סובב סביב ה-**Cpu,** היות והוא רכיב החומרה המרכזי במחשב והמון תוכנות ואפליקציות זקוקות לו בתדירות גבוהה, גם עבור קלט ופלט.

לכל רכיב חומרה יש רכיב שנקרא "בקר" (כל הבקרים מחוברים בניהם ע"י bus במטרה לאפשר גישה לזיכרון משותף) תפקידו לבצע סנכרון בין ההתקן לבין ה cpu (ולעיתים יש גם "buffer"), הcpu יכול לטפל בהתקן אחד בו זמנית כך שנוצר מצב שיש ביקוש מכמה מקורות למעבד בזמן נתון.
כאשר התקן נזקק להתערבות של ה-Cpu , הבקר של אותו התקן שולח בקשה לעיבוד המידע ב-Cpu, בקשה זו נקראת **"פסיקה (interrupt)** ", לאחר שהבקשה מגיע ל-**Cpu** הוא שולח אותה למערכת ההפעלה שהיא פוסקת ע"י אלגוריתם, מה צריך לעשות.
נבדיל בין פסיקה של חומרה לבין פסיקה של תוכנה- פסיקה מחומרה יכולה להתבצע בכל שלב ע"י שליחת אות (signal) ל-Cpu ואילו מהתוכנה מתבצעת ע"י הוראה מיוחדת הנקראת **"System Call"** (המונח מפורט בהמשך).

כאשר ה-Cpu מקבל פסיקה הוא מפסיק את מה שהוא עושה באותו הרגע (תוך כדי שמירה על המצב שבו הפסיק בהמשך נקרא לזה ()frok) ומעביר את הפסיקה למערכת ההפעלה (כדי שתפסוק) כמובן שהתהליך חייב להיעשות במהירות כי ה-Cpu מפסיק לעבוד על אותו ה-Process.(לשם כך המיקום אליו מועברות הפסיקות נמצא בתחילת הזיכרון).

מערכת ההפעלה שומרת את הפסיקה בטבלה בעלת שתי עמודות שנקראת "**interrupt vector**" , עמודה אחת מציינת איזה רכיב חומרה הזמין את הפסיקה ועמודה שניה מציינת איזה אלגוריתם הוא צריך.

מערכת ההפעלה מטפלת בפסיקות ושולחת את התוצאה למעבד (Cpu), מערכת ההפעלה משתמשת ב**ISR** (" Interrupt Service Routine") על מנת לטפל בפסיקה (להחליט מה לעשות).הISR הוא תהליך (תוכנה) המופעל כאשר יש בקשת פסיקה מרכיב חומרה או מתהליך. הוא מטפל בבקשה ושולח אותה למעבד (קוטע את התהליך הפעיל במעבד). כאשר ISR הושלם התהליך שב להתחרות על מקומו במעבד(עובר לתור המוכנים- מסוקר בהמשך).

לכל פסיקה קיימת פונקציית שירות פסיקה שאחראית לטפל בה (פסיקה מאוחסנת בתור מספר) בטבלת הISR שמורות כתובות של אלגוריתמים שמטפלים בפסיקות(צריך להפעיל את אלגוריתם x על מנת לטפל בפסיקה y).

עד כה דיברנו על פסיקות הנובעות מצורך כלשהו של חומרה או תוכנה, אבל ישנם פסיקות הנוצרות גם מחריגה של תוכנית, כמו חילוק באפס, גישה לכתובת לא מורשית וכו'. פסיקות כאלו נקראות **"Trap".**

בשל ריבוי ההתקנים והאפליקציות מערכת ההפעלה מתעסקת כל הזמן בפסיקות (interrupt driven).

כמובן שיש חשיבות לסדר של טיפול הפסיקות, במי מערכת ההפעלה תטפל קודם, כאמור הפסיקות שטרם נפסקו מאוחסנים בווקטור, ישנם 2 שיטות לסריקת הווקטור בהן יכולות לנהוג מערכות ההפעלה:

1) **Poling** – סריקה בצורה מעגלית, עוברים אחד אחד ושואלים מי הזמין את הפסיקה וכשמגיעים לסוף הווקטור חוזרים להתחלה.
   ישנו חיסרון לשימוש בשיטה זאת, התגובה אינה מהירה, במקרה שבו נכנסה פסיקה האמורה להיות מטופלת במהירות ואנחנו רק במקום השני בווקטור, נצטרך לחכות מחזור שלם עד לטיפול בפסיקה לכן שיטה זאת לא תתאים למערכות כמו שיש בבתי חולים או מערכת ברקסים או כל מערכת זמן אמת אחרת.

2) **Vectored-** גישה ישירה, לכל סריקה המאוחסנת בווקטור יש key ולכן אפשר לגשת לפסיקה ב(1)O.

מבנה הקלט פלט- I/O STRUCTURE

כשאר תהליך משתמש מבקש רכיב i/o כלשהו ה-Cpu טוען את הבקר של אותו רכיב ברגיסטרים המתאימים וכשאר הרכיב מסיים את פעולת ה-i/o הוא מודיע ל- Cpu בעזרת פסיקה שהוא סיים.

מה קורה עם אותו תהליך שביקש את פעולת הi/o בזמן ביצועה?! ישנם 2 שיטות : **סינכרוני/א-סינכרוני.**

סינכרוני- תהליך המשתמש עובר למצב wait במידה וקיים מנגנון כזה המשהה את התהליך.
או לולאה אינסופית, הלולאה תיעצר ותאפשר את המשך התהליך רק כאשר תגיע פסיקה המעידה על סיום הקלט או הפלט.

א-סינכרוני- ה-Cpu יעביר את הבקרה לתהליך גם לפני שתהליך ה-i/o סיים והתהליך ייבצע את הפעולות שהוא מסוגל גם מבלי הקלט או הפלט (שפעולתם טרם נגמרה).

בשיטה הא-סינכרונית יש צורך בSystem Call על מנת שנוכל לאפשר לתוכנית להמתין ל i/o שיסתיים (צריך לדעת מי הקורא לשירות הi/o כיוון שהבקרה אצלו) ובנוסף צריך לעקוב ולנהל את כל בקשות ה i/o השונות לשם כך מערכת ההפעלה שומרת טבלה הנקראת **"Device status table"** אשר בה מאוחסנים כל שמות ההתקנים שמבצעים פעולות i/o וגם את הסטטוס של הפעולה(כאשר מתקבלת פסיקה על תהליך i/o מערכת ההפעלה מוצאת בטבלה את ההתקן ששלח ומעדכנת את הסטטוס שלו). בנוסף יש תור בקשות עבור כל ההתקנים שדורשים i/o.

היתרון בשיטה הא-סינכרונית הוא שה -Cpu ממשיך לעבוד על דברים אחרים בזמן פעולת ה-i/o.

כאמור, אנו שואפים שה-Cpu יעבוד תמיד ולא יתעכב על פעולות שיכולות להיפתר גם בדרכים אחרות.
לשם כך יש את ה**"DMA (Direct Memory Access)**" שהוא דואג להעביר את המידע מרכיב ה-i/o אל הזיכרון ישירות וגם להיפך ללא התערבות של הcpu, המטרה היא לאפשר ל-Cpu לעבוד על תהליכים אחרים בינתיים.
התזמון בין ה-DMA לבין ה-Cpu נעשה בעזרת מערכת ההפעלה.

מבנה הזיכרון- Storage Structure:

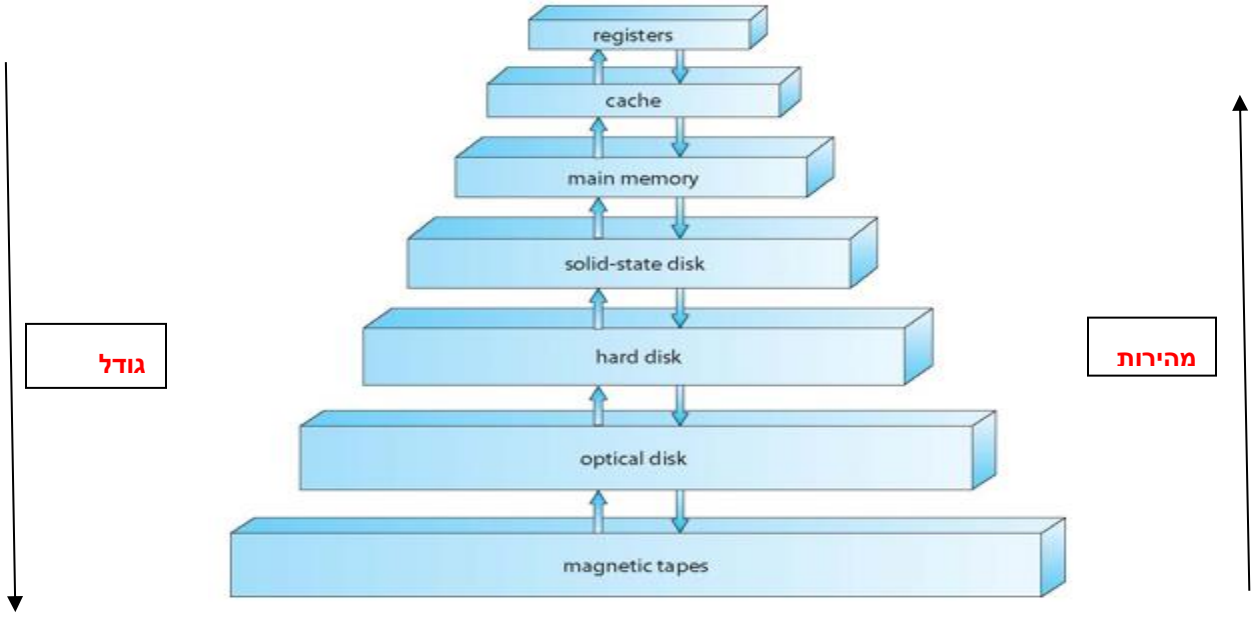
זיכרון ראשי -Main Memory: זהו הזיכרון שאתו ה-Cpu יכול לעבוד בצורה ישירה(יכול גם עם זיכרון הרגיסטרים) , שם התוכניות שהמעבד מריץ מאוחסנות, הזיכרון מתחלק ל-2 סוגים: א) **ROM(Read Only Memory)-** זמין לקריאה בלבד, לא נמחק כאשר המחשב נכבה.
ב) **RAM (Read Access Memory)-** זיכרון חשמלי, נמחק עם כיבוי המחשב.

זיכרון משני-Secondary Storage: הרחבה לזיכרון הראשי, אחסון גבוהה , מאפשר גישה רק בעזרת פסיקות.

זיכרון דיסק קשיח –Hard Disk : איטי יותר, פיזי ומגנטי, לדיסק יש רכיב הנקרא " **Disk Controller**" שאחראי על האינטראקציה בין ההתקן למערכת ההפעלה. (לא רוצים לעכב את הcpu על העברת מידע בצורה איטית)

זיכרון דיסק SSD: מהיר יותר מדיסק קשיח יותר יקר (ולכן בא בתצורות של זיכרון קטן יותר).

כשאר מדברים על זיכרון יש קשר בין המהירות לבין הגודל: ככל שהזיכרון יותר קטן כך **מיוצר** מהיר יותר.
ז"א שעלות (ממונית) של מהירות בזיכרון גבוהה ולכן רק הזיכרונות הקטנים נהנים ממנה.



Caching-כשאר מתבצע העתקה של תוכן מזיכרון איטי יותר לזיכרון מהיר יותר, לדוגמא העתקה של תוכן מהזיכרון המשני לזיכרון הראשי.

פרק 2: מבנה מערכת ההפעלה

מבנה מערכת ההפעלה- Operating System Structure:

ישנן גישות שונות באשר למבנה הפנימי של מערכת הפעלה. לכל גישה יש את המאפיינים הייחודיים לה.

Multiprogramming: שיטה (עיקרון) יעילה לניצול ה- Cpu אשר לא מאפשרת מצב שבו המעבד והתקני הקלט והפלט "מוחזקים" על ידי משתמש אחד, אלא יש סוג של עיבוד מקבילי שבו כמה תוכניות פועלות "בו זמנית".

מכיוון שיש רק מעבד אחד, לא ניתן לבצע ביצוע סימולטני של תוכניות שונות. במקום זאת, מערכת ההפעלה מבצעת חלק מתוכנית אחת וכשהיא נכנסת למצב wait (אולי בגלל פעולת i/o) המעבד עובר ל job (תהליך) אחר ולאחר שהתהליך מתחדש היא המעבד חוזר אליה, וכן הלאה. למשתמש נראה שכל התוכניות מבוצעות בו-זמנית.

Timesharing: (מקרה פרטי של Multiprogramming) המחשב יודע לחלק את הזמן בין התהליכים בצורה מהירה מאוד , המעבד מקצה לכל תהליך זמן שבו הוא יקבל את הcpu. וככה הוא יכול "לדדות בין התהליכים" .

הזמן שבו ה- Cpu עוזב תהליך אחד לטובת השני צריך להיות ממש קטן (פחות משניה) על מנת שהמשתמש לא ישים לב לעזיבת ה-Cpu. (הבהרה: לא מחכים לפסיקה עלמ"נ לעבו בין jobs אלא מקצים מראש זמן התעסקות לכל תהליך).

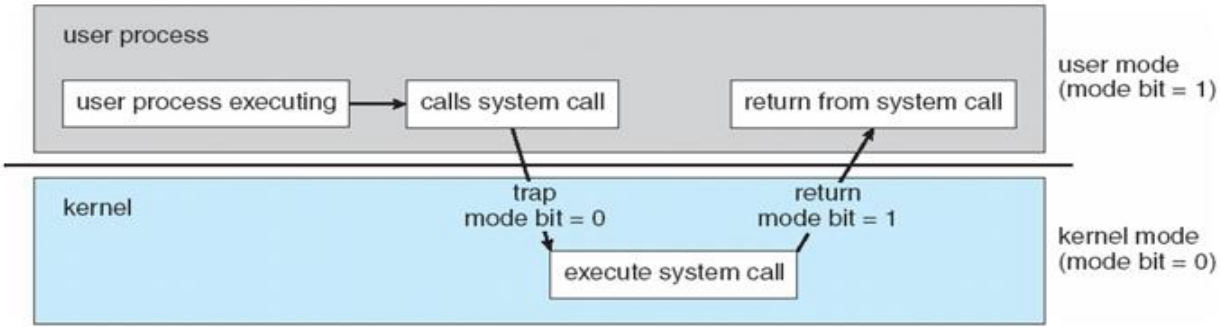
בנוסף הוא יודע לחלק כל תהליך למשימות. הדבר מאפשר לעבוד עם מספר משתמשים במקביל .

הגנה על חומרה – Hardware Protection:

מערכת ההפעלה מספקת הגנה על משאבים קריטיים במערכת, (בעבר משתמשים היו כותבים ישירות לחומרה אבל מובן שזה לא בטיחותי ואכן פריצות רבות וחורי אבטחה היו אז), נסקור את ההגנות השונות על המשאבים הקריטיים:

Dual Mode Operation: נועד כדי להגן על מערכת ההפעלה בשל השימוש במשאבי מערכת משותפים של מערכת ההפעלה (גם חומרה) יש להבטיח שטעות בתוכנית אחת לא תגרור טעיות בשאר התוכנית, לשם כך מערכת ההפעלה מספקת שני מצבי עבודה: 1)**User Mode-** רק user יכול לרוץ (מקבל את המעבד).
2)**Kernel Mode-** רק מערכת ההפעלה יכולה לרוץ.

ישנו ביט המתווסף לחומרת המחשב (mode bit) המציין מהו המצב הנוכחי: כאשר ערכו 1 אז המערכת במצב user וכאשר ערכו 0 אז המערכת במצב kernel.



<u>הגנה על קלט ופלט- I/O Protection</u>: כל פעולות הi/o מוגנות , למעשה גישה ל-i/o מתבצעת רק דרך מערכת ההפעלה וזאת על מנת שלא יהיה ניתן לשנות ערכי פסיקות.

כדי להגדיר הגנה על הזיכרון נבחין בין 2 מושגים: 1) **process** (תהליך) -תוכנית שרצה.
2)**progress (Program)**- קוד כתוב.

למעשה progress הופך לprocess כאשר מרצים אותו.

<u>הגנה על הזיכרון- Memory Protection:</u> הגנה על הזיכרון היא משמעותית היות וברגע שניתן לגעת בזיכרון ניתן לעשות הכל. לכן מערכת ההפעלה מקצה לכל process (תוכנית שרצה) שטח בזיכרון שרק אליו הוא יכול לגשת וכל גישה לשטח אחר תגרור שגיאה.

<u>הגנה על ה-Cpu:</u> יש צורך להגן על ה-Cpu מפני process שלא ישתלטו עליו (ע"י לולאה אינסופית למשל) ולכן רק למערכת ההפעלה יש גישה ל-Cpu . איך זה עובד?!
יש שעון הנקרא **"Time"** אשר מודד זמן לכל process (ע"י ספירה לאחור) אם הזמן פקע נשלחת פסיקה והשליטה עוברת למערכת ההפעלה (monitor mode) ומערכת ההפעלה נותנת עדיפות לprocess אחר.

צריך להבדיל בין protected לבין security :
protected: הגנה על משאבים מפני המשתמש .
security: הגנה על משאבים מפני גורמים חיצוניים ווירוסים.

<u>ניהול תהליכים-Process Management:</u> כאמור קוד כתוב (progress) הופך להיות תהליך (process) כאשר מריצים אותו, כדי לרוץ הוא זקוק למספר משאבים כגון: זמן Cpu, זיכרון, i/o, קבצים.
מהמערכת הפעלה מקצה את המשאבים הנ"ל לכל תהליך וכשהוא מסיים היא משחררת אותם.
לכל תהליך גם מוקצה **"PC (Program Counter)"** שמצביע על ההוראה הבאה לביצוע בתוכנית.
ישנם תוכניות בעלי מספר תהליכונים (**threads**) שלהם מוקצה pc לכל תהליכון.

מערכת ההפעלה דואגת גם ליצירה ומחיקה של תהליכים, השעיה או הפעלה מחדש של תהליכים ומנגנון תזמון בין תהליכים שונים.

<u>ניהול הזיכרון-Memory Management :</u> כדי ש- process-ים יוכלו לרוץ , מערכת ההפעלה צריכה לדאוג שכל המידע הנדרש יהיה על הזיכרון הראשי באותו הרגע.
לכן מערכת ההפעלה אחראית על: מעקב אחר חלקים בזיכרון (מי נמצא בשימוש וע"י מי), להחליט איזה תהליך יוטען לזיכרון במקום התהליך שכעת הסתיים והקצאה ופינוי הזיכרון כשצריך.

<u>ניהול מערכות ה- I/O:</u> למערכות קלט ופלט יש buffer בו המידע נשמר(באופן זמני) עד אשר התהליך מבקש אותו.
וזאת על מנת להסתיר את המבנה של התוכנית שמריצה את ההתקנים (התכנית שקראה ל-i/o) .
כאשר נכנס על התוכנית מידע (קלט) הוא נכנס לזיכרון זמני שנקרא "**spool**" שם הוא מתחרה על קבלת זמן Cpu כדי לבצע את הi/o.

שירותים של מערכת ההפעלה -Operating System Services:

מערכת ההפעלה מייצרת סביבה להרצת תוכניות ומספקת שירותים שונים לתוכניות אלו, השירותים המסופקים משתנים ממערכת הפעלה אחת לשנייה אבל בדרך כלל יש את אלו:

מצד המשתמש:

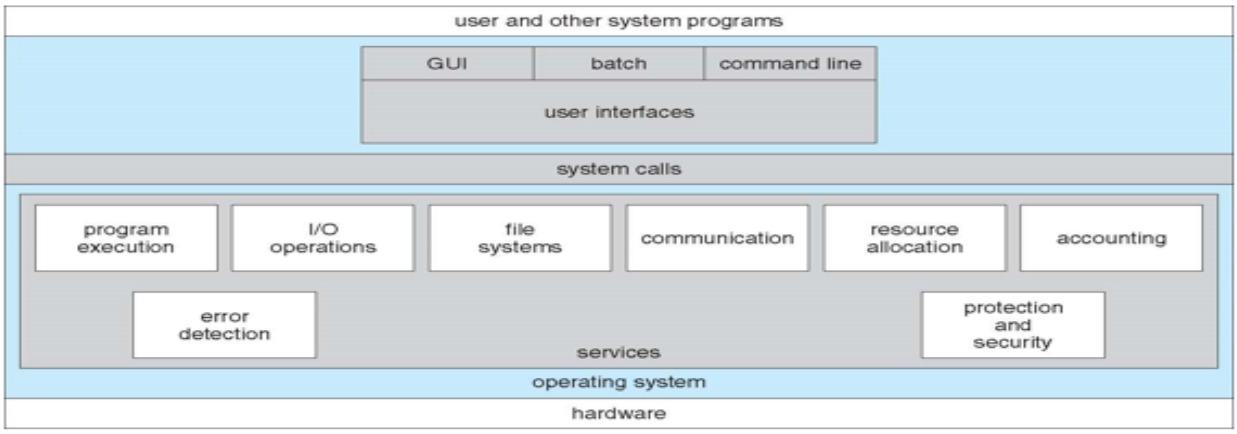
- **הרצת תוכניות-** מערכת ההפעלה מסוגלת לטעון תוכנית אל הזיכרון הראשי (caching) ולהריץ אותה.
- **ממשק למשתמש (interface user)** -מה המשתמש רואה: שורת פקודה/ ממשק גראפי(GUI)/ מערכת אצווה (מערכת שרצה ברקע).

שורת פקודה(-CLI-command line): הכנסת פקודות ישירות למערכת ההפעלה , למערכת ההפעלה יש **"command interpreter"** שהוא מפענח את הפקודות ומבצע אותן.
באופן כללי הממשק של הkernel עם המשתמש נקרא **"shell"** – אחראי לפענח את התוכנית ולהעביר את הפקודות למערכת ההפעלה.

ממשק גראפי (GUI-graphic user interface): ממשק ידידותי למשתמש עם מערכת ההפעלה , כולל עכבר / צג / מקלדת, הרעיון להקל על המשתמש לבצע פעולות ע"י גרפיקה פשטוה.

מצד מערכת ההפעלה:

- **פעולות I/O-** מכיוון שהמשתמש לא יכול לגשת לאמצעים אלו , על מערכת ההפעלה לדאוג לדרכי גישה.
- **מערכות קבצים-** לכתוב/ לקרוא / למחוק מקובץ ע"י מערכת ההפעלה.
- **תקשורת-** החלפת מידע בין תהליכים שרצים על אותו המחשב או על מערכות שונות המחוברות ביניהם.
- **טיפול בשגיאות (error dictation)-** מערכת ההפעלה צריכה להיות ערכוה לכל תרחיש שגיאה ולדעת לטפל.
- **הקצאת משאבים(resource allocation)-** חלוקת המשאבים (יש הרבה משאבים: cpu, זיכרון ראשי , i/o) בין התהליכים/ המשתמשים הרצים במקביל.
- **חשבונאות(accounting)-** מעקב סטטי אחרי משתמשים במערכת (כדי לדעת לייעל את המערכת)
- **אבטחה והגנה (protection and security)–** לנהל הרשאות ולשמור מפני משתמשים זדוניים.



System Call: פניה ישירות למערכת ההפעלה כדי שתבצע עבודה מסוימת, (זה בעצם הממשק בין התוכניות הרצות לבין התוכניות של מערכת ההפעלה- רוצה להשתמש בתוכנית של מערכת ההפעלה (פונקציה, כמו קריאה או כתיבה לזיכרון) בתוכנית שלך?! אין בעיה, דרך system call).
הבקשה יכולה להיות כתובה בשפת אסמבלי או בשפה עילית (לרוב בעילית).
פקודת system call מפעילה תוכניות של מערכת ההפעלה (בשביל התוכנית של המשתמש) דרך פונקציה שנקראת **API "(Application programming Interface)"**
דוגמא לפונקציית API היא פונקציית ()read שנמצאת במערכות Linux וגם ב- Unix :

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

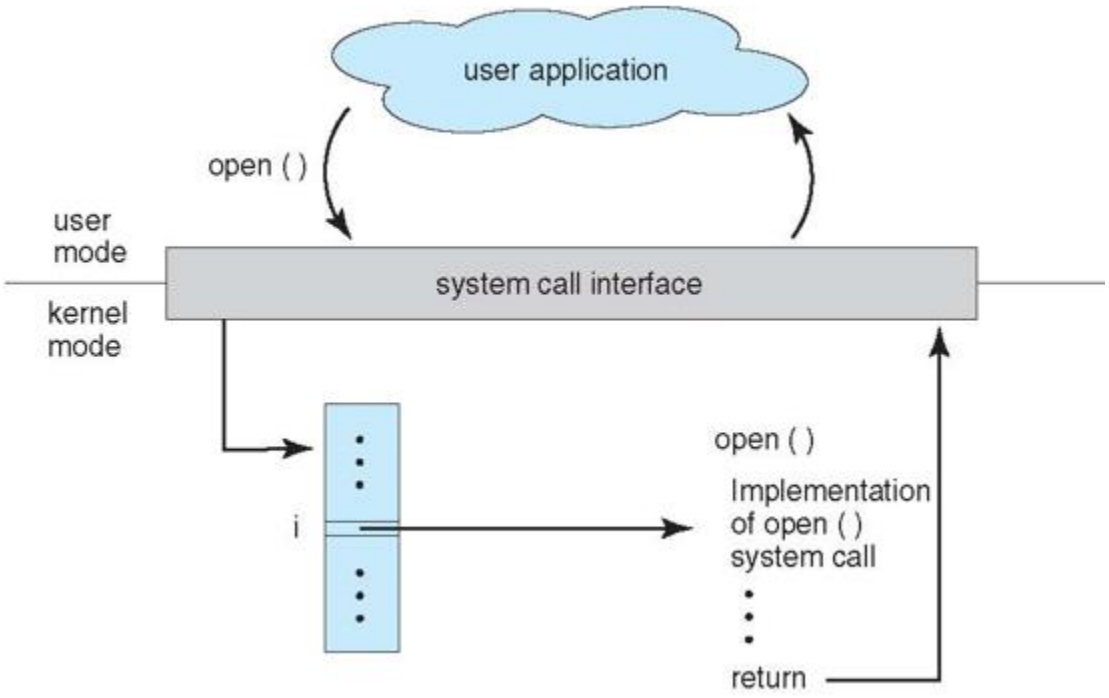
כל system call מקבל מספר סידורי שלאותו מספר יש פונקציית API מתאימה המנגנון הזה מתוחזק ע"י טבלה עם אינדקסים שנקראת **"System Call Interface"** .
הממשק יודע להתאים לכל System Call מספר סידורי מתאים לפונקציית ה- API אליה הוא זקוק.

כדי להעביר פרמטרים (בין התוכנית של המשתמש לתוכנית של מערכת ההפעלה) ישנם 3 שיטות:

1. **באמצעות רגיסטרים**, חיסרון- מספר הרגיסטרים מוגבל (לפעמים פרמטרים >רגיסטרים).
2. **באמצעות טבלה השמורה בזיכרון**, מעבירים את כתובת ההתחלה של אותה טבלה ע"י רגיסטר.
3. **שימוש במחסנית,** התוכנית תשמור את המידע במחסנית ומערכת ההפעלה תשלוף אותם משם.
   חיסרון- גודל המחסנית אינו דינאמי ולכן יכול להיות stack overflow).

מבחינת המשתמש שמריץ את התוכנית, הפונקציות של מערכת ההפעלה הם שקופות לגמרי (מתייחס אליהם כקופסא שחורה), המשתמש כתב תוכנית-> התוכנית רצה(אחרי שעברה 2 שלבים: הבאת הנתונים וקומפילציה) -> התוכנית זקוקה לגישה לזיכרון או לתוכנית i של מערכת ההפעלה-> התוכנית של המשתמש תצבע קריאת system call תוך כדי העברת הפרמטרים הרלוונטיים דרך אחת משלושת השיטות שלקמן->דרך הAPI התוכנית הרלוונטית במערכת ההפעלה רצה על הנתונים שנשלחו (עוברים למצב monitor mode)-> ערכי החישוב חוזרים (במידה ורלוונטי)-> חוזרים לuser mode-> המשך הרצת תוכנית המשתמש.

כאשר פונקציית API הצליחה לבצע את הSystem call מוחזר סטטוס יציאה 0, אם יש שגיאה מוחזר הערך 1-.



מבנה המערכת- System Structure:

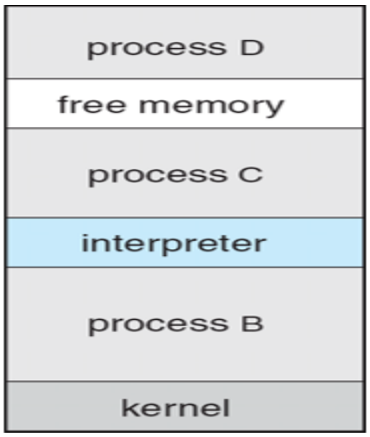
נסקור 2 מבנים:

MD-DOS- מבנה פשוט של מערכת ההפעלה (לא מאורגן)ישנם 4 שכבות כאשר כל אחת יכולה לגשת לשנייה.

בנוסף יש device driver -תוכנית ממשק בין התקנים לחומרה, מאפשרת גישה של אותו התקן לכל החומרה(לא בטוח).

מאפשר להריץ תוכנית אחת כל פעם (single tasking) .

היתרון שאפשר לגשת לROM (הזיכרון הראשי) מכל מקום.



FreeBSD- Unix , מבנה מערכת ההפעלה מורכב מגרעין (kernel) ותכוניות מערכת.

הגרעין אחראי על הקצאת משאבים (זיכרון ,Cpu וניהול מערכות קבצים) דרך שיטת ה- System call .

דבר זה מאפשר למספר תהליכים (משתמשים) לרוץ "במקביל" בזיכרון הראשי (multi-tasking).

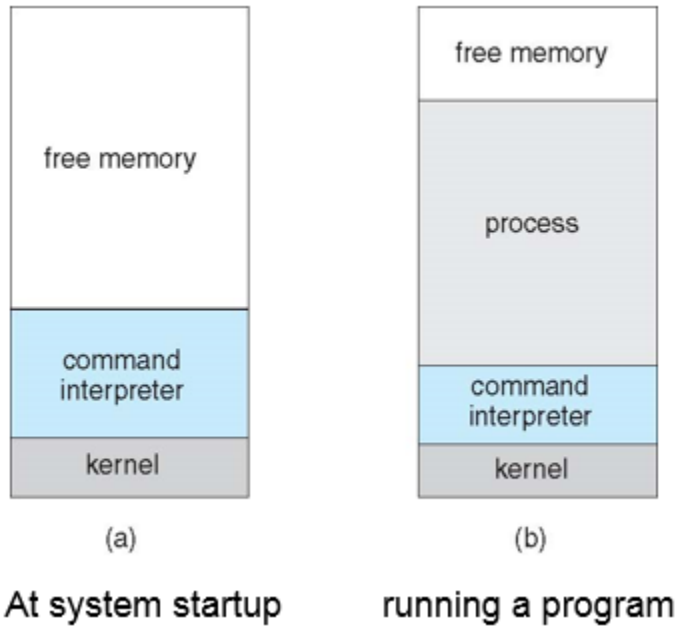
ישנה חלוקה ל-3 שכבות : משתמש->גרעין-> חומרה.

כאמור התוכניות מפוענחות ומבוצעות ע"י Shell שאחראית לממשק בין מערכת ההפעלה לבין המשתמש.

על מנת לאפשר ריבוי משתמשים או במקרה שתהליך של משתמש רוצה ליצור תהליך חדש תוך כדי הרצה הshell מבצעת פקודת System Call שנקראת **"frok()**" כדי ליצור תהליך חדש, בנוסף הוא יוצר העתק של התהליך שכרגע אנו עוצרים (כדי לאפשר לחדש לרוץ) שנקרא **"תהליך הבן-child process**" כדי שהמעבד ידע לחזור ולטפל בתהליך שהופסק.

על מנת לטעון את התהליך החדש הshell משתמש ב- System Call שנקרא **"exec()**" שזה טעינת הקוד של התוכית החדשה להרצה.

בצורה הזאת מאפשרים Multi-Tasking.



באיור נתון הזיכרון הראשי ב2 מצבים, במצב (a) המערכת רק עלתה ובמצב (b) רואים שרץ רק תהליך אחד.

במבנה ה-FreeBSD כאשר תהליך מסתיים הוא מחזיר סטטוס יציאה:
הערך 0 חוזר כאשר תהליך הסתיים בהצלחה.
כל ערך שגדול ממש מ0 מסמן Error, התוכנית לא הסתיימה בהצלחה.

<u>גישת השכבות-הגישה המודולרית-Layered Approach:</u>
גישת השכבות יושמה לראשונה ב-Microsoft וזו הרחבה של Unix.
בגישה זו מערכת ההפעלה מחולקת למספר שכבות כאשר כל שכבה נבנית על השכבה שמתחתיה.
השכבה התחתונה ביותר זוהי החומרה ואילו השכבה הגבוהה ביותר זוהו המשתמש.
כל שכבה יכולה להשתמש בשירותים של השכבה שמתחיה בלבד.

מעט טענות נכון /לא נכון :

1. האם אפשר לממש מערכת הפעלה שתומכת בעיקרון ה-Multi Programing ללא שימוש ב-Time Sharing?

-כן, אפשר להשתמש בMultiprocessing (כמה מעבדים לאותה מערכת הפעלה).
ובכלל אפשר לומר ש-Multiprogramming זה עיקרון שממומש ע"י כל מיני מקריים פרטיים (כמו עיבוד מקבילי שבו המעבר בין תהליכים הוא בגלל פסיקה, בשונה מTime-Sharing שבו המעבר בין תהליכים הוא פונקציה של זמן).

2. האם Tarp היא פסיקה הקשורה לחומרה?

-לא, זוהי פסיקה שנובעת מחריגה בעת הרצת תהליך כלשהו.

3. האם נכון לממש את ה- Interrupt Vector בשיטת ה-Polling כאשר אנו מתכננים מערכת שתסחור בבורסה?

-לא, במקרה הגרוע התגובה תהיה איטית מידי.

פרק 3-4: תהליכים

תהליכים-process:

ישנם 2 סוגי תהליכים:

1. **Job** – תהליך שפועל ברקע (בערכת אצווה-Batch).
2. **Task / User Program-** תוכניות שיש להם קשר עם המשתמש (התהליך כולל תגובה של ה-User).

כדי למנוע בלבול בין המושגים השונים נפרט על 3 המושגים הבאים ועל הקשר בניהם: Program, Process, Thread .

כאמור ישנו הבדל בין **תוכנית (Program -**קוד כתוב ששמורה בזיכרון המשני או בדיסק הקשיח) לבין **תהליך** (**Process** התוכנית בזמן הרצה שמאוחסן בזיכרון הראשי אחרי קומפילציה והבאת הנתונים הנדרשים לתוכנית).

מהי תוכנית (Program)?
התוכנית היא קובץ הרצה המכיל את אוסף ההוראות שנכתב כדי לבצע עבודה מסוימת במחשב
לדוגמה, notepad.exe הוא קובץ הפעלה המכיל את סדרת **ההוראות** (קוד בשפה עילית או אסמבלי) אשר מסייעות לנו לערוך ולהדפיס את קבצי הטקסט.
התוכניות אינן מאוחסנות בזיכרון הראשי במחשב אלא מאוחסנות בדיסק קשיח או בזיכרון המשני .
לאחר שלב של קומפילציה התוכנית נקראת לזיכרון הראשי (caching) ומבוצעת על ידי מערכת ההפעלה (בשלב הזה זה כבר תהליך).
תוכנית מכונה ישות פאסיבית בגלל שהיא מתגוררת בזיכרון משני.
תוכנית אחת מכילה כמה תהליכים בתוכה.

מהו תהליך (Process)?
התהליך הוא מופע של ביצוע תוכנית. לדוגמה, כאשר אתה לוחץ פעמיים על סמל פנקס במחשב, **מתחיל תהליך שיפעיל את תוכנית פנקס הרשימות.**
תהליך מכונה לפעמים ישות אקטיבית בגלל שהוא מאוחסן בזיכרון הראשי. (מספר תהליכים עשויים להיות קשורים לאותה תוכנית, לדוגמא, באפשרותך להפעיל מופעים רבים של תוכנית פנקס רשימות **כך שכל מופע הוא תהליך בפני עצמו.**)
בתוך התהליך יכולים להמצאות כמה תהליכונים Threads):

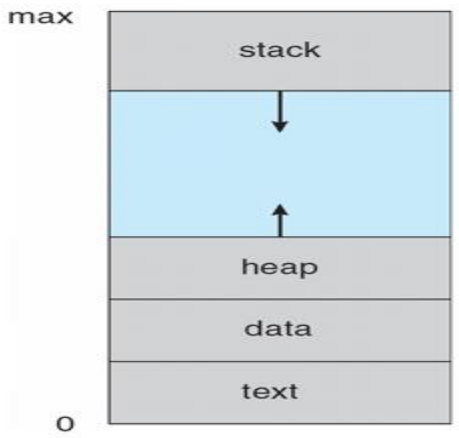
מהו תהליכון (Thread)?
תהליכון הוא יחידת ההפעלה הקטנה ביותר של תהליך. לדוגמה, כאשר אתה מפעיל תוכנית פנקס רשימות, מערכת ההפעלה יוצרת תהליך ומתחיל את ביצוע התהליכון הראשי של תהליך זה.
תהליך יכול להכיל מספר תהליכונים , כל תהליכון יהיה בעל משימה משלו ,לדוגמא, בתוכנית פנקס, תהליכון אחד יהיה אחראי על הקשר עם המשתמש ותהליכון נוסף יהיה אחראי על הדפסת המסמך.
כל התהליכונים של אותו תהליך חולקים את הזיכרון של התהליך אליו הם שייכים ולכן התקשורת בין תהליכונים מהירה מאוד.

(ההסבר והתמונות לקוחות מהאתר --https://www.quora.com/What-is-difference-between-thread-process-and-program)

מבנה התהליך- Process Concept:

מערכת ההפעלה מקצה לכל תהליך זיכרון משלו שמחולק ל-4 חלקים:

1. **Heap (ערימה)-** זיכרון דינאמי מוקצה בזמן ריצה-משתנים דינמיים של התהליך (כמו מופע של מחלקה).
2. **Stack (מחסנית)-** בו נשמרים כתובות חזרה לפונקציה /פרמטרים של פונקציה / משתנים מקומיים.
3. **Data (נתונים)-** עבור נתונים ומשתנים גלובליים.
4. **Text (התוכנית)-** שם שמור הקוד הכתוב של התוכנית.



לכל תוכנית מוקצה Program Counter (PC) שתפקידו לעבור על כל הפקודות של התוכנית באופן סדרתי (מצביע על שורת הקוד הבאה).

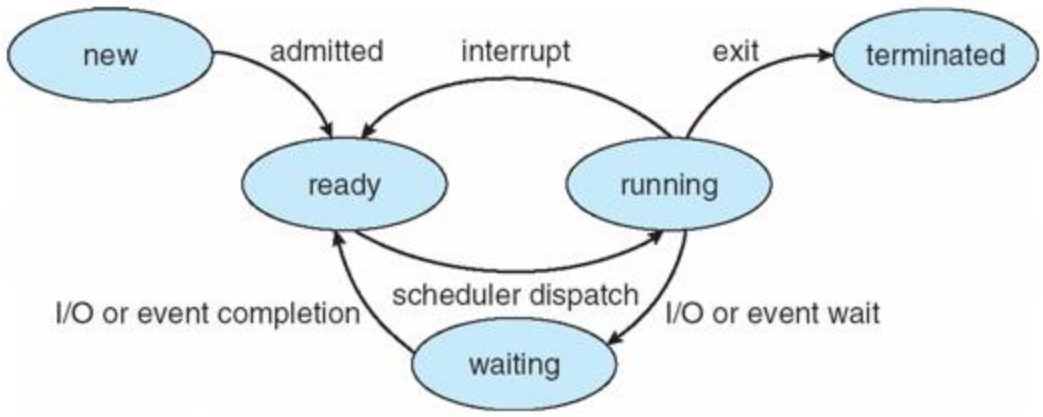
ברגע שעשינו Caching לתוכנית (מהזיכרון המשני לזיכרון הראשי) היא הופכת לתהליך.

מצבי התהליך- Process State:

כל תהליך יכול להמצאות באחד מתוך חמישה מצבים שונים:

1. **New-** הרגע שבו הprocess נוצר.
2. **Ready-** התהליכון מוכן להרצה אבל ממתין לקבלת זמן Cpu .
3. **Running-** התהליך מריץ פקודות (בזמן נתון רק תהליך אחד יכול להימצא במצב זה).
4. **Waiting-** התהליך זקוק לקלט או פלט ולכן עובר ממצב Running למצב Waiting.
5. **Terminated-** התהליך הסתיים.

Diagram of Process State



זוהי דיאגרמה של המצבים שתהליך יכול להימצא בהם.
המצב הראשוני של תהליך הוא New שם מאותחל בנתונים הרלוונטיים ומיד עובר למצב Ready .
ברגע שמגיע תורו הוא עובר למצב הרצה Running , כעת אם תתקבל פסיקה תוך כדי ההרצה אפשר לעבור ל-2 מצבים: אם פסיקה של קלט פלט נעבור למצב Waiting אם פסיקה מסוג Trap (חריגה כמו חילוק ב-0) נעבור למצב Ready וימשיך לרוץ כאשר ה-Cpu יתפנה.

כעת נכיר את מבנה הנתונים בו מאוחסן כל תהליך:

Process Control Block-PCB:

כל תהליך מיוצג במערכת ההפעלה ע"י PCB שזהו מבנה נתונים המאחסן את התהליך ואת המידע המשויך לאותו התהליך, (כאשר התהליך נוצר- כשמגיע לזיכרון הראשי- נוצר מבנה הנתונים הנ"ל) ומכיל בתוכו:

1. **מצב התהליך-** Ready ,New, Running, Waiting ,Terminated
2. **Program Counter-** מצביע לפקודה הבאה אותה צריך לבצע.
3. **Cpu Registers-** כאשר תהליך במצב Running והגיע interrupt התהליך עובר למצב Ready לאחר ביצוע הפסיקה ולכן נרצה לשמור את המצב של התהליך לפני הגעת הפסיקה כדי שיוכל לרוץ מאותו מצב כאשר יגיע שוב לCpu.
4. **Cpu Scheduling information-** עדיפות התהליך, מצביע לתור התזמונים(ב- Ready queue).
5. **מידע על ניהול הזיכרון-** מידע על הזיכרון שהוקצה לתהליך.
6. **Accounting information-** כמה זמן התהליך השתמש ב-Cpu (Timesharing).
7. **מצב I/O-** רשימת התקני קלט פלט וקבצים פתוחים בהם משתמש התהליך.

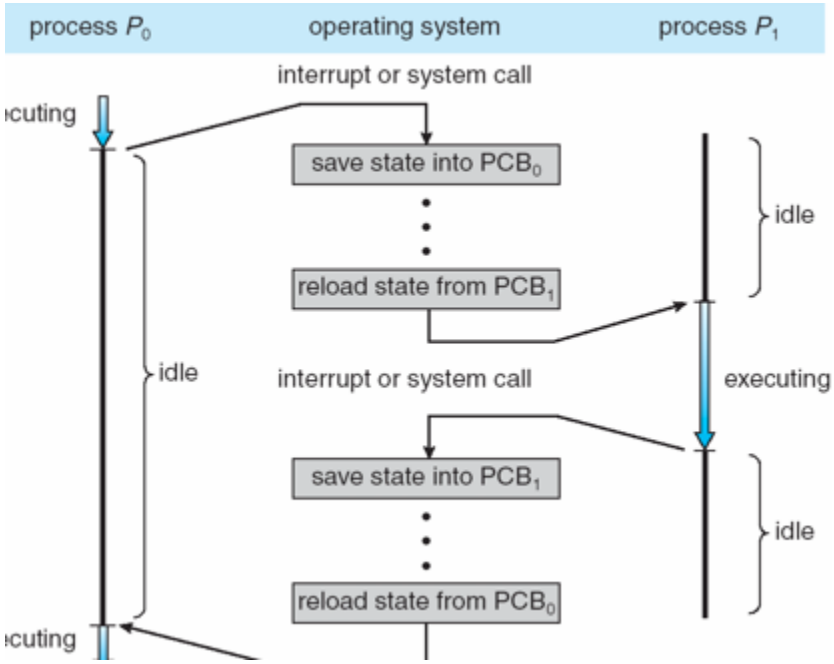
מעבר בין תהליכים- Cpu Switch:

כאמור אנו שואפים לכך שה-Cpu יהיה כמה שפחות זמן במצב סרק (Idle) ולכן המעבר בין תהליכים צריך להיות חכם ומהיר.

גם ראינו את העיקרון של Multiprogramming (שמכיל בתוכו את Timesharing) שגם בו צריך להתבצע חילוף בין תהליכים בצורה יעילה.

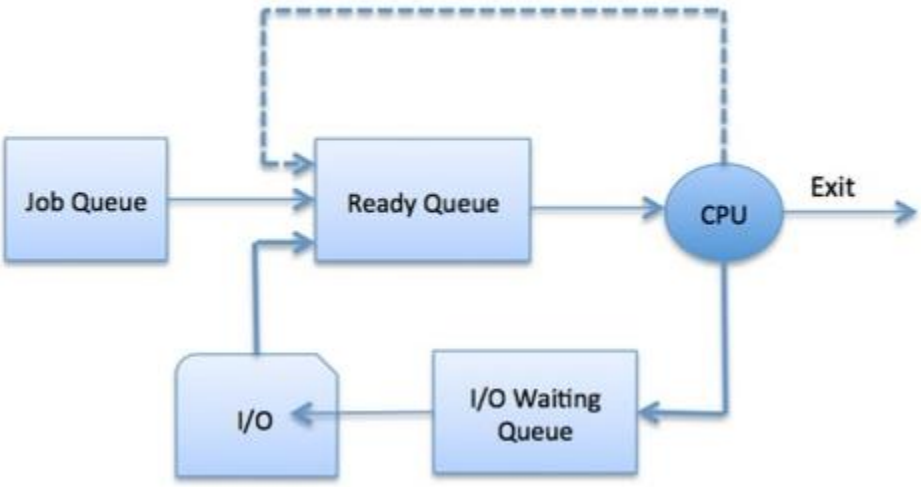
כאמור בכל פעם שיש בפסיקה ה-Cpu מחליף בין תהליכים, כדי שהמעבר ביניהם יהיה מהיר התור של ה-PCB נשמר בזיכרון הראשי.

אך כפי שמתואר באיור- עדיין יש זמן תקורה שנוצר מהחלפת התהליכים הרצים במעבד.



Process Scheduling -תיזמון תהליכים:

ברגע שתהליך נכנס למערכת ההפעלה הוא נשמר ב-**Job Queue**, תור זה מכיל את כל התהליכים במערכת.
תהליכים שכבר מוכנים לריצה ורק ממתינים לזמן cpu מאוחסנים ב-**Ready Queue**(בזיכרון הראשי).
ה-Ready Queue לרוב ממומש כרשימה מקושרת.

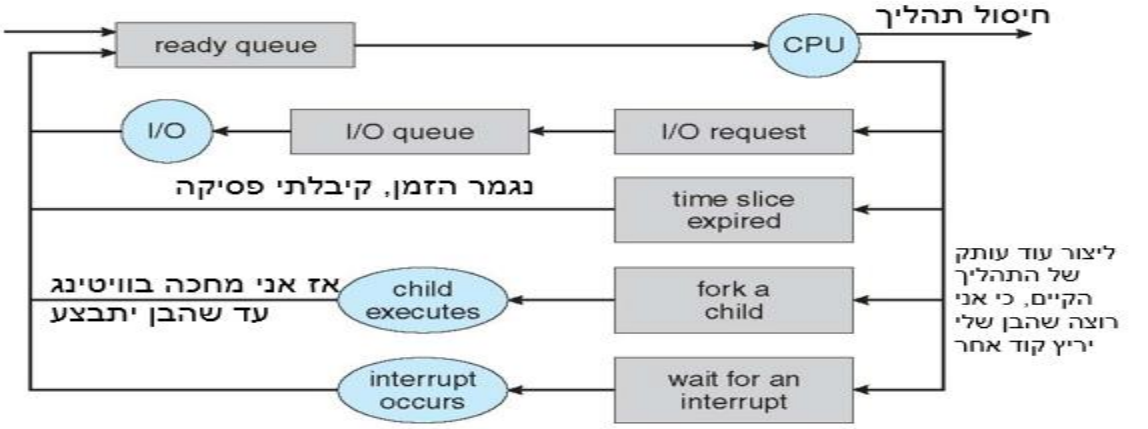


#הערה: לעיתים ה-I/O Waiting Queue נקרא Device Queue.

נסכם את התורים שראינו עד כה:

Job Queue- תור של תהליכים שנכנסו למערכת.
Ready Queue- תור של התהליכים המוכנים לריצה בזיכרון הראשי.
I/O Waiting Queue- תהליכים שמחכים לפעולת i/o להשלמת התהליך.

כל תהליך יכול להימצא בכמה תורים בחייו, להלן דיאגרמה המדגימה זאת:



כדי להזיז את התהליכים בין התורים השונים , מערכת ההפעלה משתמשת ב-2 מתזמנים:

Long term scheduler- נקרא גם job scheduler , זהו המתזמן של תור ה-Jobs , מחליט אילו תהליכים יעברו מתור ה-Jobs אל תור ה-Ready. נקרא Long כי התזמונים שלו קורים בדחיפות נמוכה (פעם בכמה שניות או דקות). המשמעות של המתזמן הזה היא כמה תהליכים אפשר להריץ "במקביל" (רמת ה- Multi-Programing תלויה בכמות התהליכים שיש ב-Ready Queue).

Short term scheduler- נקרא גם Cpu scheduler, זהו המתזמן של תור ה-Ready , מחליט אילו תהליכים יעברו ל-Running ויקבלו את זמן ה-Cpu. נקרא Short כי התזמונים הם מאוד מהירים ובדחיפות גבוהה.

Medium term scheduler- לא נמצא בכל מערכת הפעלה, זהו סדרן שכאשר יש צורך להוריד את רמת ה-Multi-Programing , הוא לוקח תהליכים מהזיכרון הראשי ומאחסן אותם בדיסק הקשיח ולאחר זמן מחזיר אותם לריצה ע"י החלפת תהליכים- **Swapping**: נאלצים להוציא תהליך מהזיכרון הראשי החוצה כיוון שיש עומס בזיכרון הראשי (מלא), רצוי להוציא לSwap-Disk (דיסק שמיועד לכך) כי הוא מהיר יותר מהדיסק הקשיח.
פעולת ההוצאה נקראת Swap-Out וברגע שמתפנה מקום בזיכרון הראשי הוא חוזר Swap-In ל-Ready Queue.

אפשר לסווג את התהליכים כך:
I/O- bound process- תהליכים הזקוקים כל הזמן לשירותי קלט פלט (תהליך מסוג זה נמצא הרבה בתור ה-Waiting -כמו תוכנית שקוראת מקובץ)- גישות קצרות ל-Cpu.

Cpu bound process- תהליך שצריך כל הזמן חישובי Cpu- גישות ארוכות ל-Cpu.

מערכת הפעלה טובה היא מערכת היודעת לשלב את שני סוגי תהליכים אלו.

Context Switch- חילוף תהליכים ב-CPU (נקרא גם חילוף הקשר):
כדי שה- Cpu יוכל להחליף בין תהליכים הוא זקוק למנגנון שיידע לשמור את המצב הנוכחי (בעת הוצאתו) של התהליך , וטעינת התהליך החדש, 2 פעולות אלו נקראות :**"Context Switch**" .
מובן שמערכת הפעלה טובה תבצע את ה-Context Switch בצורה מהירה עם אלגוריתם פשוט וקצר ככל האפשר.

כעת נפרט יותר על פעולות על תהליכים .

Operation On Process- פעולות על תהליכים:

תהליכים יכולים להיות עצמאיים או תלויים בתהליכים אחרים כך שיהיו מושפעים מתהליכים אחרים ויחלקו מידע בניהם.
מה המוטיבציה לתלות בין תהליכים? ובכן:

- ✓ שיתוף מידע.
- ✓ חילוק משימה לתתי תהליכים ונקבל האצה.
- ✓ מודולריות-לחלק משימה לתתי תהליכים כך שלכל תת תהליך תהיה משימה משלו.
- ✓ נוחות.

למעשה החלוקה היא כזו:

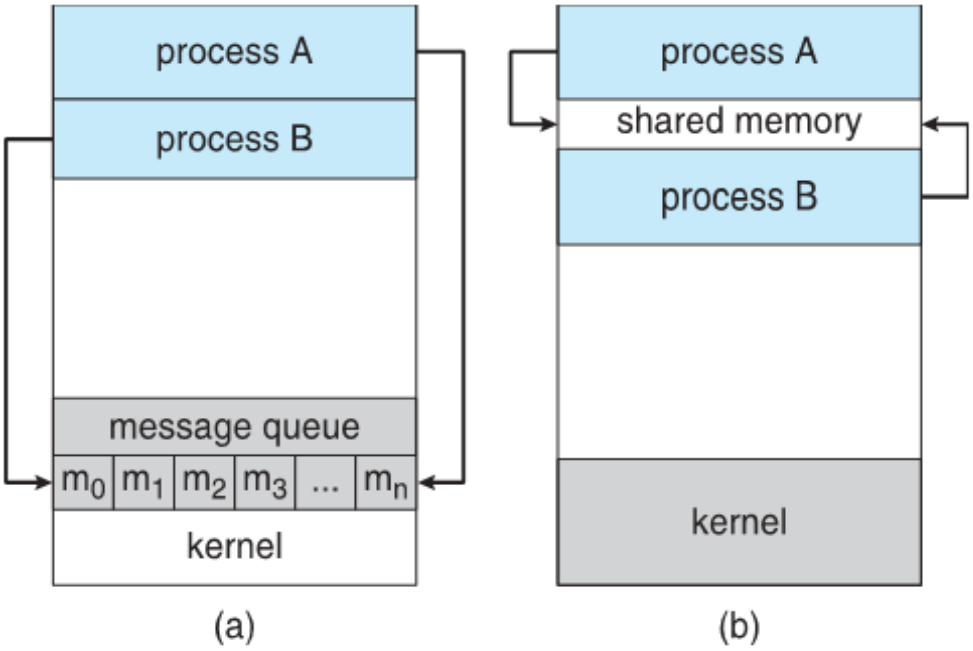
Independent Process-תהליכים עצמאיים: תהליכים לא תלויים ולכן לא מושפעים מתהליכים אחרים (אין זיכרון משותף).

Cooperating Process- תהליכים שיתופיים- מושפעים אחד מהשני , חולקים בניהם משאבים משותפים.

כדי שתהליכים יוכלו להיות תלויים זה בזה הם זקוקים לתקשורת בניהם –**"interposes communication (IPC)"**, ישנם 2 מנגנונים לתקשורת בין תהליכים:

1. **Shared Memory-** זיכרון משותף- יחידת זיכרון משותפת ששני התהליכים יכולים לגשת אליה ולכתוב בה ולקרוא ממנה.
2. **Message Passing-** העברת מסרים- הם שולחים ומקבלים הודעות אחד לשני דרך ה-kernel .

(a) Message passing. (b) shared memory.



לתהליכים שיתופיים ישנה בעיה ידוע הנקראת **"בעיית יצרן צרכן- Producer-Consumer Problem**"

ויקיפדיה:

"במדעי המחשב, בעיית הצרכן- היא דוגמה קלאסית לתיאור בעיית סנכרון שני תהליכים או יותר הנגרמת במערכות המאפשרות ריבוי משימות.

הבעיה נגרמת כאשר שני תהליכים או יותר חולקים חוצץ המוגבל בגודלו. חלק מהתהליכים אחראים להכניס מידע לחוצץ (יצרנים) וחלק אחראים לשלוף ממנו מידע (צרכנים).

פתרון הבעיה דורש מנגנון שימנע משני תהליכים לעדכן בו-זמנית את אותה רשומה ,בעזרת שימוש בסנכרון בין התהליכים.

פתרון לא מספק יגרום בחלק מהמקרים למצב של קיפאון ,כאשר שני התהליכים "ישנים" וממתינים ש"יעירו" אותם

פתרון אפשרי לבעיה מתבצע באמצעות סמפורים או מוניטורים .כאשר יש יותר מיצרן אחד וצרכן אחד, יש להשתמש גם במנעול. "

ישנם 2 פתרונות להתמודדות עם הבעיה:

- **Unbounded-buffer-** מאגר לא מוגבל (הצרכן ממשיך לייצר מידע).
- **Bounded- buffer-** מאגר מוגבל, כאשר מתמלא היצרן צריך לחכות עד שהצרכן ירוקן אותו.

קטע קוד המשקף את פתרון ה-Bounded buffer:

Shared data

```
#define BUFFER_SIZE 10      (17:13 b)
typedef struct {
  . . .
} item;  האיתם שאני מייצר

item buffer[BUFFER_SIZE];   במקרה זה יש לי מקום ל10 עצמים
int in = 0;   מצביע על המיקום הנוכחי בבאפר איליו יכניס היצרן
int out = 0;  מצביע על האיבר הבא בתור שהצרכן צריך להשתמש בו
```

Solution is correct, but can only use BUFFER_SIZE-1 elements

היצרן:

```
item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)   אם אין לי מקום כי הבאפר מלא
        ; /* do nothing */   אז אני לא עושה כלום
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;   קידום האין פויינטר.
}
```

while (((in + 1) % BUFFER_SIZE) == out)

מה שקרה הוא שהמצביע של היצרן הצליח להדביק את המצביע של הצרכן, ולכן הוא יפסיק לייצר, עד שהצרכן יצרוך את מיקום אווט.

בעיה שתמיד צריך להשאיר מקום אחד ריק בבאפר

הצרכן:

```
item next_consumed;     (19:05 b)
while (true) {
while (in == out)   כל עוד שאין כלום בבאפר
    ; /* do nothing */   לא עושה כלום
next_consumed = buffer[out];   // (in > out) כאשר
out = (out + 1) % BUFFER_SIZE;

/* consume the item in next consumed */
}
```

כעת נרחיב בעניין התהליכונים.

תהליכון- Thread:

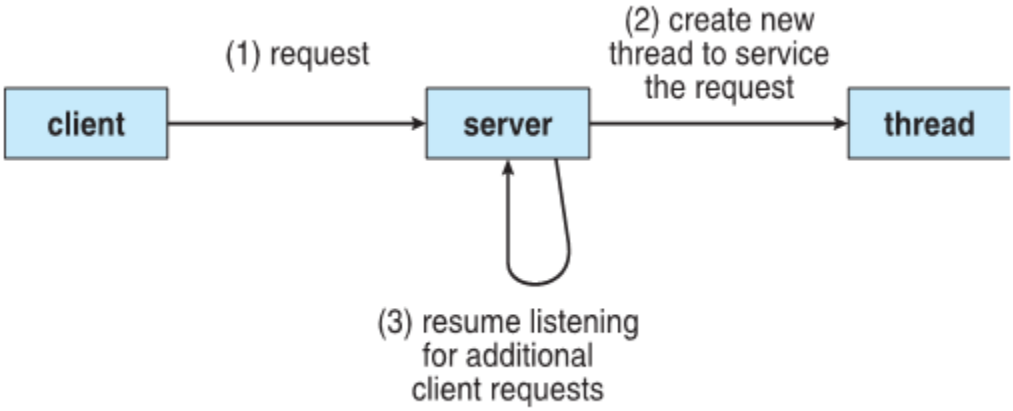
כאמור יש הבדל בין תהליך (Process) לבין תהליכון (Thread), תהליך אחד מכיל כמה תהליכונים שפועלים במקביל .
מוקדם יותר בסיכום הגדרנו תהליכון כ-" יחידת ההפעלה הקטנה ביותר של תהליך " לכן לכל תהליכון יש מבנה נתונים (בדומה למבנה הנתונים של תהליך- PCB) שנקרא "TCB" שבו נשמרים הנתונים של התהליכון (מובן שיש פחות נתונים ממה שיש ב- PCB היות וחלק מהנתונים משותפים בין התהליכונים).
לתהליכונים יש אחריות לא לדרוס אחד את השני ולא לשנות מידע משותף.

נראה את המידע המשותף בין כל התהליכונים ואת המידע שאינו משותף (אלא כל תהליכון מכיל כזה מידע):

לא משותף	משותף
• Program Counter • רגיסטרים • מחסנית	• קוד • נתונים • משאבים של מערכת ההפעלה

האפליקציות (תוכניות) המודרניות משתמשות בהרבה תהליכונים כדי לעדכן תצוגה / להביא מידע / לענות על שאילתות מהרשת וכו'

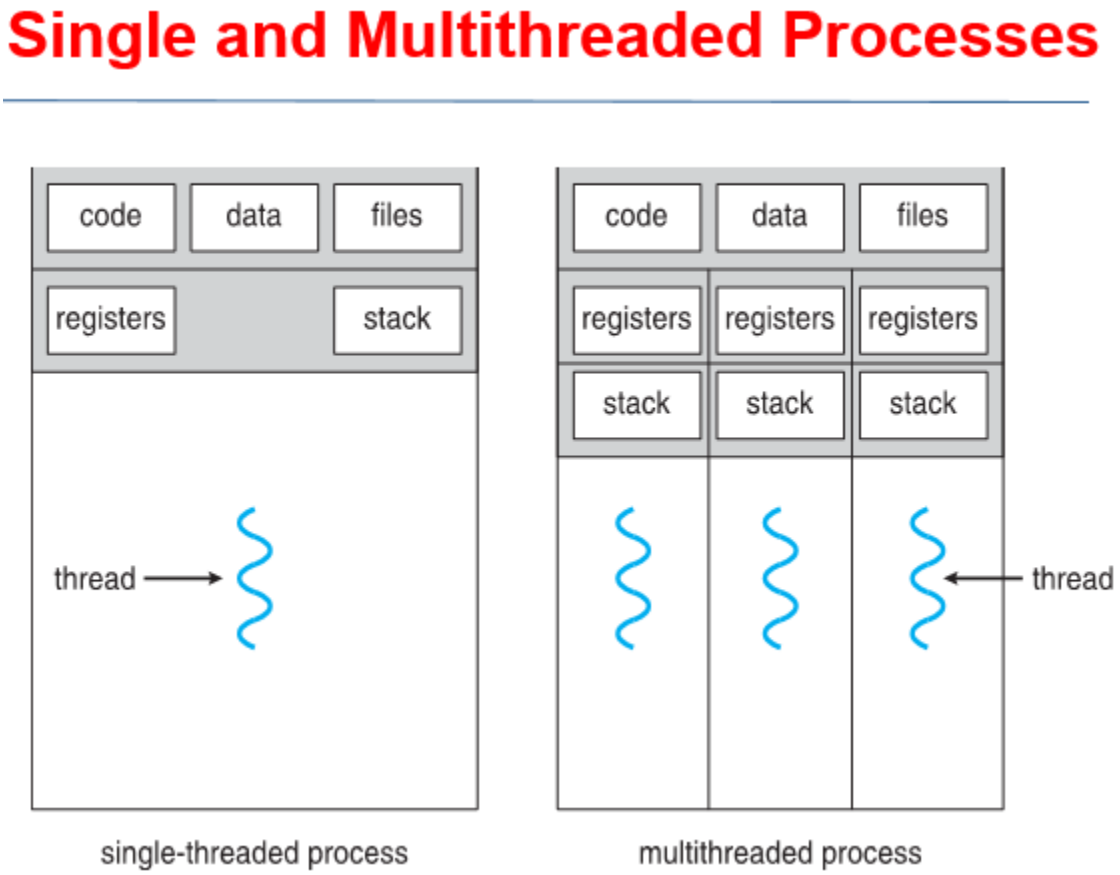
.



יתרונות של Threads:

- ✓ **Responsiveness-תגובה טובה יותר-** במידה וחלק אחד של התוכנית נחסם אפשר להמשיך להריץ את השאר.
- ✓ **Resource Sharing- שיתוף משאבים-** שיתוף המשאבים בין תהליכונים נעשה טוב יותר מאשר בין תהליכים.
- ✓ **Economy-חסכוני-** "Context Switching" קל יותר כי הנתונים קטנים מאשר בתהליכים ובנוסף מהיר יותר ליצור תהליכון.
- ✓ **Scalability-מערכת שניתנת לשינוי-** יכולים להתמודד עם עומס (לדוגמא: חלוקה לליבות- כל תהליכון ירוץ בליבה אחרת וכו')

האיור הבא מתאר את המשאבים המשותפים והפרטיים של התהליכונים ואת המצב בו תוכנית מכילה כמה תהליכונים :



<u>תהליכונים ברמת המשתמש וברמת מערכת ההפעלה:</u>

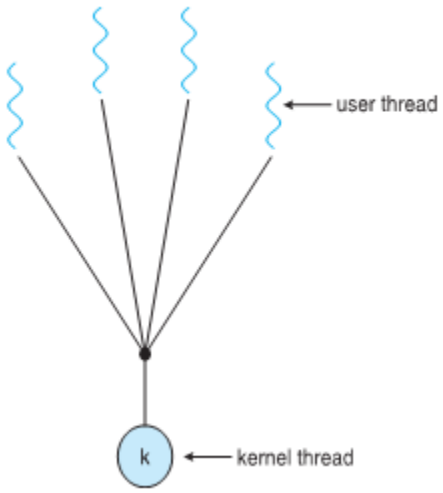
יש תהליכונים שרצים ברמת ה- User ויש שרצים ברמת ה-Kernel .

<u>User thread</u>- מנוהלים ע"י המשתמש (כותב הקוד), ישנה ספריית תוכנה שמספקת פונקציות של Threads כך שמערכת ההפעלה לא מתעסקת בזה אלא אנו אחראים לכך בקוד שלנו.
חיסרון לחוסר מעורבות של מערכת ההפעלה בתהליכוני משתמש היא חוסר מתן העדפה לתהליכונים כבדים.

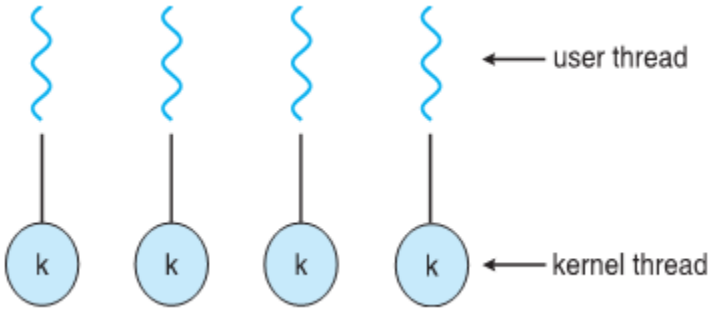
<u>Kernel threads</u>- תהליכונים שמנוהלים ע"י מערכת ההפעלה, היא מייצרת לכל תהליכון TCB משלו ויכולה גם לתת עדיפות לתהליכונים כבדים יותר (ולהקצות זמן).

ישנם כמה מודלים של מערכת ההפעלה שמקשרים בין תהליכונים ברמת ה-User לתהליכונים ברמת ה-Kernel:

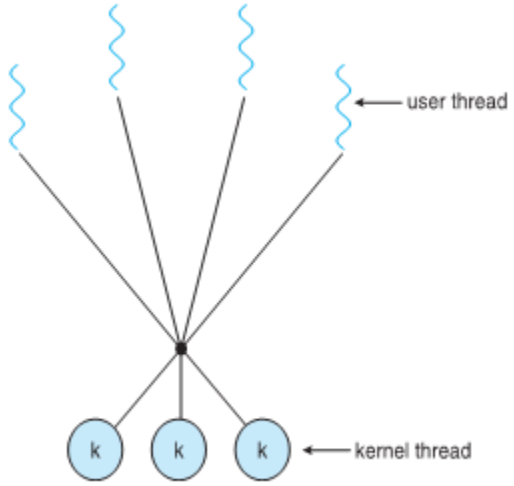
Many To One-הרבה לאחד: הרבה תהליכים ברמת המשתמש שהופכים לתהליך אחד ברמת מערכת ההפעלה.
חיסרון- אם תהליכון אחד של רמת המשתמש נחסם אז הכל נחסם.
אין ריצה במקביל של תהליכונים.



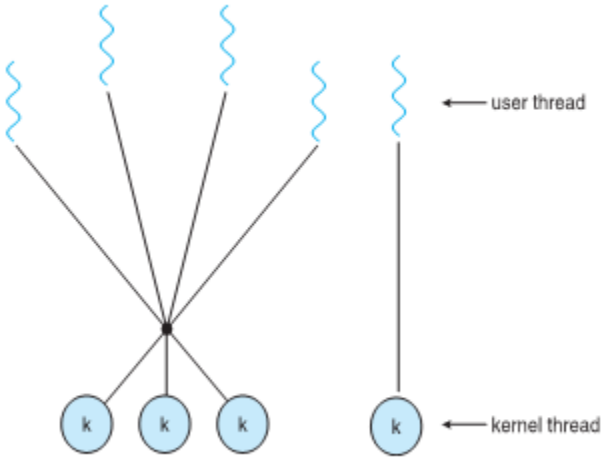
One To One- אחד לאחד: שיפור של המודל הקודם, כל תהליכון ברמת המשתמש הופך לתהליכון ברמת מערכת ההפעלה ובכך אם אחד נחסם הוא לא תוקע את השאר.
חיסרון- חוסר יכולת לחלק את התהליך להרבה תהליכונים (למשל- כל משימה בתהליכון) כי יש חסם של מערכת ההפעלה על כמות התהליכונים לתהליך אחת.



Many To Many – הרבה להרבה: ממפה כמה תהליכונים ברמת המשתמש לכמה תהליכונים ברמת מערכת ההפעלה ובכך הרווחנו שאם תהליכון אחד נחסם לא תקענו את השאר וגם חסכנו בכמות התהליכונים של מערכת ההפעלה (כך שלא נגיע לחסם).



Two-level Model- שילוב 2 המודלים: שילוב של 2 השיטות: one2one, many2many .
זהו המודל הממומש לרוב כיום.



פרק 5: תיזמון תהליכים

תזמון ה-CPU -CPU Scheduling:

כדי לאפשר את עיקרון ה-Multi-Programing (מספר תהליכים השוכנים בזיכרון הראשי בו זמנית והמעבד מבצע אותם לסירוגין) על CPU אחד אנו חייבים לתזמן בין התהליכים (כאמור רק תהליך בודד יכול לרוץ בזמן נתון).
תזכורת: ב- Multi-Programing תהליך רץ עד שהוא צריך להמתין (בגלל פסיקה או קלט פלט) אז מערכת ההפעלה מעבירה את משאב ה-Cpu לתהליך אחר (זאת המקביליות) עד שהתהליך שהפסיק כעת יוכל לחזור.
ריבוי התהליכים עשוי להכיל גם **"חטיפה -Preemption"** של המעבד מתהליך אחד לטובת תהליך אחר בגלל קדימות או בגלל שעבר הזמן שהוקצה לתהליך הנוכחי (כאשר עובדים ב-Timesharing).
המוטיבציה לריבוי משימות :

- זמן תגובה מהיר יותר (מטפלים בכמה תהליכים יחד- לסירוגין)
- שיתוף פעולה בין תהליכים.
- חווית משתמש (יכול להריץ מספר תוכניות במקביל- דפדפן, דוא"ל וכו')
- מאפשר להפעיל תוכניות ברקע כמו אנטי- וירוס וכו' .

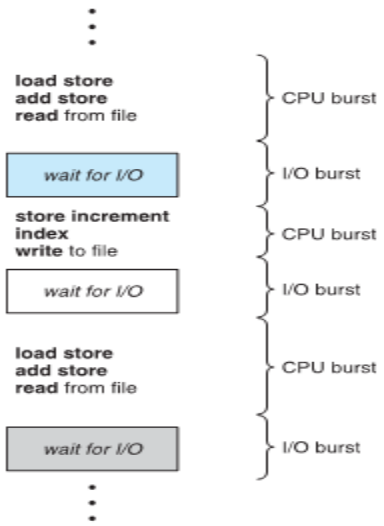
המטרה העיקרית ב- Multi-Programing היא למקסם את ניצול המעבד (לא לאפשר מצב סרק של ה-CPU).

תזמון הינה אחת מהפעולות החשובות ביותר במערכת ההפעלה (כמעט כל משאב מתוזמן).

כדי להפוך את ריבוי המשימות לאולטימטיבי צריך לאפיין את התהליכים, ובכן מאפיינים את התהליך כך:
CPU-I/O Burst Cycle- מעגל של עיבוד ושל קלט פלט: תהליך המורץ במחשב עובר לאורך ריצתו בין פרקי זמן בהם הוא:

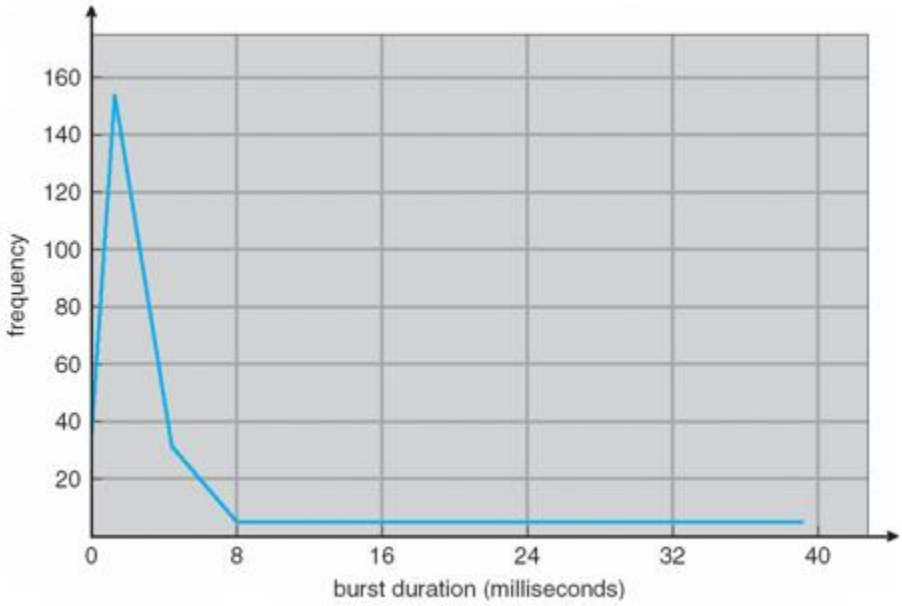
1. מבצע חישוב במעבד.
2. מצבע תהליכי קלט ופלט.

אנו אומרים שלתהליך יש פרק זמן עיבוד ואחריו פרק זמן של I/O וחוזר חלילה.



נשאלת השאלה, האם תהליכים משקיעים יותר זמן בעיבוד או ב-I/O?
כבר חילקנו בעבר את התהליכים ל-2 סוגים:
CPU Bounded- תהליך שדורש הרבה זמן עיבוד ומעט זמן i/o.
I/O Bounded- תהליך שדורש הרבה זמן I/O ומעט זמן עיבוד.

למרות השוני בין סוגי התהליכים מחקר סטטי של הרצות רבות מאוד מראה גרף כזה לCPU Duration:



רואים שרק בהתחלה יש יותר זמן של I/O ואח"כ זה מתאזן.

כעת ברור שעל מנת לאפשר ריבוי משימות אנו זקוקים למתזמן שיעשה סדר בין התהליכים.
בעבר דיברנו כבר על 2 סוגי מתזמנים:

Long term scheduler- נקרא גם job scheduler , זהו המתזמן של תור ה-Jobs , מחליט אילו תהליכים יעברו מתור ה-Jobs אל תור ה-Ready. נקרא Long כי התזמונים שלו קורים בדחיפות נמוכה (פעם בכמה שניות או דקות). המשמעות של המתזמן הזה היא כמה תהליכים אפשר להריץ "במקביל" (רמת ה- Multi-Programing תלויה בכמות התהליכים שיש ב-Ready Queue).

Short term scheduler- נקרא גם Cpu scheduler, זהו המתזמן של תור ה-Ready , מחליט אילו תהליכים יעברו ל-Running ויקבלו את זמן ה-Cpu. נקרא Short כי התזמונים הם מאוד מהירים ובדחיפות גבוהה.

כעת נדבר על המתזמן Short term scheduler.

התור של ה-Ready יכול להיות מסודר בכל מיני שיטות אבל בכל מקרה התהליך שנמצא ראשון בתור הוא הבא שיכנס למצב Running.
ה-CPU scheduler מחליט איך לסדר את התור, ולכן נשאל על פי מה הוא מחליט?

המתזמן מופעל במצבים הבאים:

1. מעביר תהליך ממצב Running למצב Waiting.
2. מעביר תהליך ממצב Runningלמצב Ready (אחרי פסיקה).
3. מעביר תהליך ממצב Waiting למצב Ready (אחרי I/O).
4. סיום תהליך.

במצבים 2 ו-3 יכולה להתבצע חטיפה. ואילו במצבים 1 ו-4 לא יכולה להתבצע חטיפה.
אם מערכת ההפעלה נוקטת במדיניות של חטיפה (preemptive) אז הסדרן יוכל לחטוף את המעבד עבור תהליך שרק עתה הגיע לתור ה-Ready אע"פ שהעדיפות שלו נמוכה מאשר העדיפות של התהליך שכעת רץ במעבד(זה מצב 2, ואם אנו עובדים ב-Timesharing - חוטפנית בפני עצמה אז מצב 3 יופעל).

כדי לבצע את החלטת המתזמן נכיר כעת את המגשר:

<u>Dispatcher- מגשר</u>:
כאמור, המגש הוא מודל שמבצע את החלטת המתזמן הקצר (CPU scheduler) ולכן מבצע:

1. context switch- החלפה בין התהליכים (הנוכחי שרץ לבין התהליך שכעת נכנס)
2. העברה ל-user mode (רק במידת הצורך).
3. להמשיך את הריצה מהמקום המתאים בתהליך שמקבל את המעבד.

Dispatch Latency- פרק הזמן שמתבזבז בין הזמן שמופסקת ריצת תהליך עד שמתחילה ריצתו של התהליך החדש.

<u>קריטריונים לתזמון -Scheduling Criteria:</u>
כיצד נחליט אילו אלגוריתמים של תזמון עלינו לבחור/להעדיף?

אנו רוצים להשוות בניהם תוך כדי שימוש בתהליכים P1,P2,……,Pn תוך כדי שאנו זוכרים שלכל תהליך יש פרקי זמן של עיבוד ופרקי זמן של I/O.

מה נעשה?
ובכן נעקוב ונבדוק איך מתוזמנים התהליכים האלו ע"י כל אחד מהאלגוריתמים ולבסוף נוכל לבחור.

הקריטריונים שנבדוק:

- CPU Utilization- זמן עיבוד: נעדיף אלגוריתם שמעסיק את המעבד כמה שיותר (נצילות גבוהה).
- Throughput-תפוקה: נעדיף את התהליך שתוך X יחידות זמן ביצע יותר תהליכים.
- Average Turnaround Time-זמן מחזור ממוצע: הזמן שלוקח למעבד להשלים את ביצוע התהליך, נעדיף אלגוריתם שיגרום למעבד להשלים את הביצוע יותר מהר.
- Waiting Time- זמן המתנה: הזמן שתהליך נמצא ב-Waiting Queue.
- Response Time-זמן תגובה: כמה זמן לוקח לתהליך לקבל תגובה ראשונית לאחר בקשה שנשלחה.

מובן שאת 2 הקריטריונים הראשונים נעדיף בזמן מקסימלי ואילו את השאר בזמן מינימאלי.

כעת נכיר כמה אלגוריתמי תזמון:

First-Come First-Served Scheduling (FCFS) -תזמון באמצעות תור:

אלגוריתם התזמון הפשוט ביותר: המעבד מוקצה לתהליכים על פי סדר הגעתם לתור ה-Ready.
אין חטיפות של המעבד.
הביצועים עלולים להיות גרועים כפי שמתואר באיור:

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The Gantt Chart for the schedule is:

- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

נצטרך לחכות 24 יחידות זמן עד לביצוע P2.

בכלליות נקרא למצב כזה, אפקט ההשיירה:

Convoy Effect- אפקט השיירה:

זהו המקרה הכללי שלנרמז מהדוגמא האחרונה : תהליך אחד גדול שמגיע לפני כמה תהליכים קצרים ויוצר מצב שבו זמן ההמתנה של התהליכים הקצרים ארוך מאוד (יכול להיות מצב שיתעכבו אחריו כמה פעמים במידה והתהליך הארוך צריך I/O וגם הקטנים שאחריו).

כעת נכיר את האלגוריתם השני:

<u>Shortest-Job-First Scheduling (SJF)- תזמון ג'ובים קצרים תחילה:</u>
האלגוריתם עובד כך, בוחר מתור ה- Ready את התהליכים שזמן העיבוד שלהם הוא הקצר ביותר.
במקרה שיש שוויון בין תהליכים, נשתמש בשיטת ה-FCFS (תור).

כעת זמן ההמתנה הממוצע יהיה בהכרח מהיר יותר, נתבונן בדוגמא הקודמת:

Suppose that the processes arrive in the order:

P_2 , P_3 , P_1

- The Gantt chart for the schedule is:

- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case

(נשים לב ש-P2 לפני P3 היות ושווים השתמשנו ב-FCFS).

בשיטה זו יש בעיה הנקראת **"<u>הרעבה</u>"** : תהליכים קצרים מגיעים בדחיפות גבוהה ל-Ready Queue ונוצר מצב שבו התהליכים הארוכים נדחים אחורה בתור וזמן ההמתנה שלהם גודל(נתקעים מאחור).
אבל למרות בעיה זו זמן ההמתנה בשיטה זו הינו היעיל ביותר.

אמרנו שבשיטה זו התהליכים עם זמן העיבוד הקצר ביותר מגיעים לראש התור, אבל איך נוכל לדעת מראש את זמן העיבוד של תהליך? נוכל רק לשער , כאשר לרשותנו נתוני הריצה בעבר ועל פיהם מנסים לשערך את פרץ (פרק הזמן) של הריצה הבאה:

1. קירוב מסדר ראשון: משך הפרץ הבא יהיה שווה לאורכו של משך הפרץ הקודם שעובד.
2. קירוב מסדר שני (מערכי): שימוש בנוסחה:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

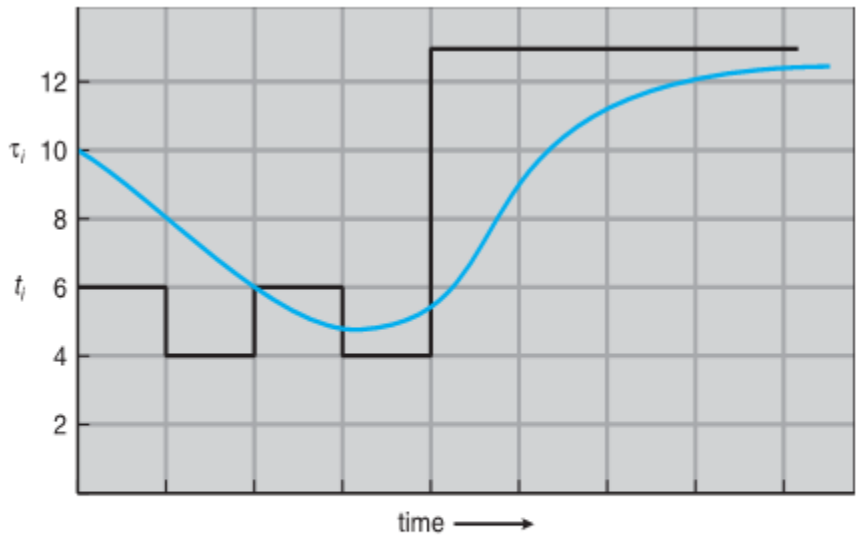
כאשר: τ_i הוא החיזוי של אורך הפרץ

t_i הוא הזמן בפועל של הפרץ

$0 \le \alpha \le 1$ הוא פרמטר

ניתן לראות שהנוסחה חוזה את משך הפרץ הבא ע"י 2 פרמטרים : כמה ניחשנו שיהיה אורך הפרץ של תהליך n וכמה באמת היה אורך הפרץ של תהליך n .
האלפא מאפשרת גמישות, כאשר שווה ל-0 לא מתייחסים לזמן בפועל שהתהליך הקודם רץ. ואילו כאשר שווה ל-1 מתעלמים מהניחוש של משך הפרץ של התהליך הקודם(אבל בד"כ אלפא יהיה קבוע בין 0 ל-1 שמשמעותו המשקל שאנו נותנים לחיזוי הקודם ולמשך הפרץ הקודם .
הערה- ניתן לחלץ את אלפא במידה ויש את הנתונים של 2 ניבויים סמוכים וכך לדעת את רמת הדיוק בין החיזוי ומציאות).

להלן דיאגרמה שממחישה את הניבויים לעומת המציאות (ע"פ שימוש בנוסחה הריבועית):



CPU burst (t_i)		6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

האלגוריתם הזה הינו במדיניות non-preemptive (ללא חטיפות) קיים גם אלגוריתם דומה שכל החידוש בו הוא מדיניות החטיפות, אלגוריתם זה נקרא :

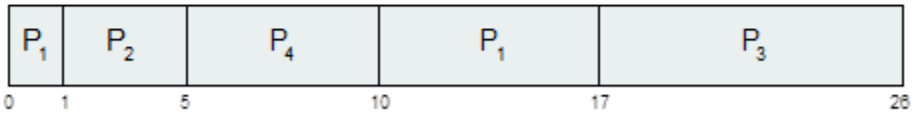
Shortest Remaining Time First (SRTF):

כאמור הוא בדיוק כמו ה SJF רק עם מדיניות של חטיפות, אפשרות החטיפה מתאפשרת כאשר מגיע תהליך חדש ל- Ready Queue בזמן שתהליך אחר רץ , התהליך החדש הינו בעל פרץ עיבוד נמוך יותר מאשר **זמן העיבוד שנישאר** לתהליך שרץ כעת ולכן האלגוריתם יבצע חטיפה.

דוגמא:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Preemptive SJF Gantt Chart



Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5

P1 יתחיל לרוץ בזמן 0 מכיוון שהוא היחיד בתור.

בזמן 1 P2 הגיע לתור , כך שנותר ל-P1 עוד 7 שניות עיבוד אז ע"פ האלגוריתם P2 יחליף את P1 במעבד (חטיפה).

בזמן 2 P3 מגיע לתור אבל פרץ העיבוד שלו גדול מאשר של P1 (7) ושל p2 (3) ולכן לא תתבצע חטיפה.

בזמן 3 , P4 מגיע לתור עם פרץ עיבוד של 5 ולכן קטן יותר מP1 (7) וגם מ-P3 (9) אבל גדול יותר מ-P2 ולכן הוא יחכה עד ש- P2 יסיים.

לבסוף סדר הרצת התהליכים הוא כזה: P1->P2->P4->P1->P3 .

כאשר נחשב את זמן ההמתנה הממוצע נצטרך לכלול את הזמן הכולל שכל תהליך חיכה (גם לאחר שנחטף לו ה-CPU). לכן:

זמן ההמתנה של P2 הוא 0 – כי לא המתין כלל לזמן מעבד.

זמן ההמתנה של P4 הוא 2 -כי היה עליו לחכות רק 2 יחידות זמן עד ש-P2 יסיים.

זמן ההמתנה של P1 מתחלק ל-לפני החטיפה ולאחרי החטיפה, בפעם הראשונה הוא לא חיכה ואילו בפעם השניה הוא חיכה עד ש-P2 וגם P4 יסיימו ולכן סה"כ 9 יחידות זמן.

זמן ההמתנה של P3 הוא 15 כי הוא התבצע אחרון , אם נסכם את כל זמני ההמתנה ונחלק ב-4 (כי יש 4 תהליכים) נקבל 6.5 .

(באיור למעלה זה מחושב באופן הבא : על כל כניסה של תהליך למעבד מחושב כמה זמן הוא המתין עד כה פחות זמן הגעתו לתור, מובן שהשמיטו את הכניסה הראשונה של P1 כי הוא הגיע בזמן 0 וחיכה 0 שניות אז זה 0=0-0 וזה חסר משמעות לסכום ההמתנות).

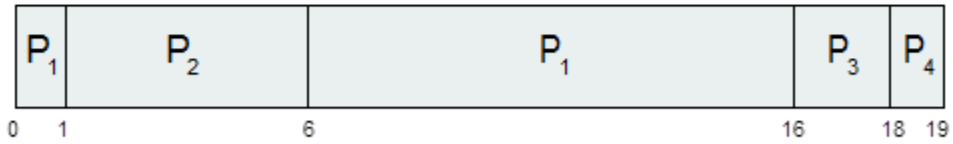
כעת נכיר עוד אלגוריתם:

Priority Scheduling-תזמון עדיפויות:

הרעיון הכללי , ניתן לכל תהליך עדיפות (מספר) באופן כלשהו ונקצה את המעבד לתהליך שעדיפותו מרבית.
אם יש תהליכים בעלי אותה עדיפות נשתמש באלגוריתם FCFC עבורם.
נשים לב שאלגוריתם SJF הוא מקרה פרטי של תזמון עדיפויות.
תזמון עדיפויות יכול להיות חוטפני או לא.
ישנה בעיה של "**הרעבה**" גם פה או חסימה אינסופית של תהליך מסוים במצב שבו התהליך אף פעם לא מתקדם בתור כי הוא נחסם ע"י תהליכים שהעדיפות שלהם נמוכה יותר.
פתרון אפשרי, **Aging-הזדקנות**: קדימותו של תהליך תעלה ככל שהוא מזדקן.
(# באיור יש טעות במספור התהליכים)

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2 msec

עוד אלגוריתם יהיה:

Round Rubin (RR)- סבבי תזמון:

הרעיון, FCFS עם מדיניות חטיפה: קובעים לכל תהליך פרק זמן ("**Time Quantum**") אחרי הפרק זמן הזה (בדר"כ 10-100 מילי שניות) מתבצעת חטיפה לטובת התהליך הבא בתור, כאשר הנגזל עובר לסוף ה-Ready Queue (יכול להיות מצב שפרק הזמן שהוקצה לתהליך הספיק לו כדי להסתיים ואז לא תתבצע חטיפה).
באלגוריתם הנ"ל אין את בעיית ה-השיירה (תזכורת: תהליך אחד גדול שמגיע לפני כמה תהליכים קצרים ויוצר מצב שבו זמן ההמתנה של התהליכים הקצרים ארוך מאוד).

כיצד נבחר את גודל יחידת הזמן?

-אם נקבע את t להיות זמן גדול, בעצם מימשנו תור רגיל (FIFO), אין תועלת.
-אם נקבע את t (גודל יחידת הזמן שכל תהליך מקבל) כזמן קטן מאוד אז ייווצר מצב שנבזבז הרבה זמן על החלפת תהליכים (Context Switch) .
-אם נקבע את t להיות בערך כמו הזמן של Context Switch אז נקבל תקורה (Overhead- היא שימוש במשאבים באופן שאינו תורם באופן ישיר לביצוע משימה) גבוהה כי חצי מהזמן הוא יהיה עסוק בהחלפות.
לכן נקבע את t לפי הכלל הבא: עבור n תהליכים שנמצאים ב-Ready Queue כל תהליך יקבל 1/n , במצב כזה זמן ההמתנה המרבי הינו t(n-1).
חשוב: בשיטה זו לא תיתכן הרעבה.

נכיר עוד אלגוריתם:

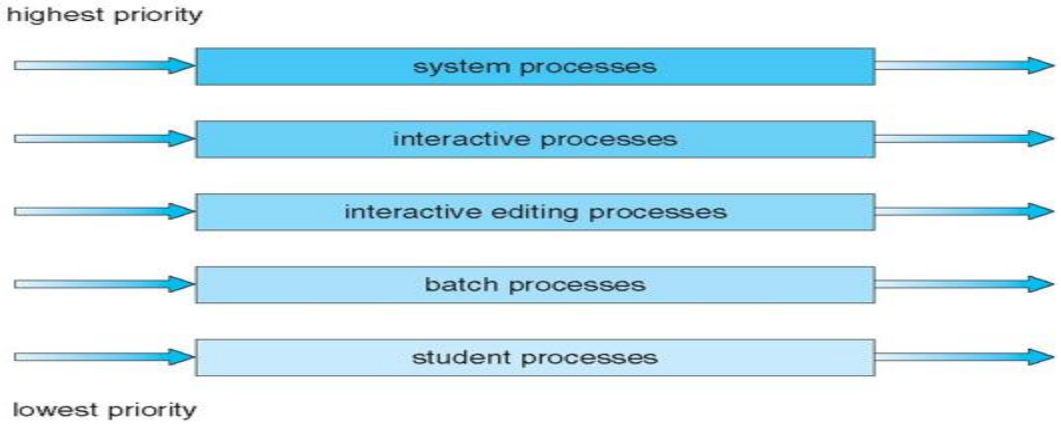
<u>Multilevel Queue- תזמון תור רב-רמתי</u>:
לעיתים ניתן לחלק את התהליכים בקלות לקבוצות נפרדות, למשל:

- Foreground- תהליכי חזית, דורשים אטרקציה עם המשתמש.
- Background-תהליכי רקע (אצווה).

לכל אחד מהקבוצות יש דרישות "זמן תגובה" שונות ולכן גם צרכי תזמון שונים (בנוסף ייתכנו גם עדיפויות שונות בין הקבוצות) ולכן נוח לחלק אותם לתורים.
ובכן האלגוריתם מחלק את התהליכים שנמצאים בתור ה-Read למספר תורים (בכך דואג לתקורה נמוכה) ועל כל תור מופעל אלגוריתם אחר (לדוגמא RR על תור תהליכי החזית ו- FCFS על תור תהליכי הרקע).

בנוסף צריכים לקבוע מתזמן בין התורים השונים, השיטה המקובלת היא, תור שקדימותו גבוהה יותר זוכה לעדיפות מלאה וחוטפנית על פני תור שקדימותו נמוכה יותר.
משמעות הדבר היא שתהליכים בתוק שקדימותו נמוכה ירוצו רק כאשר התור שקדימותו גבוהה יותר ריק.
יש עוד אפשרות לחלק זמן לכל תור כמו ב-RR.
באלגוריתם זה תיתכן הרעבה כי תורים בעליי עדיפות נמוכה יכולים להיתקע מאחור.

איור המדגים חלוקה לתורים:

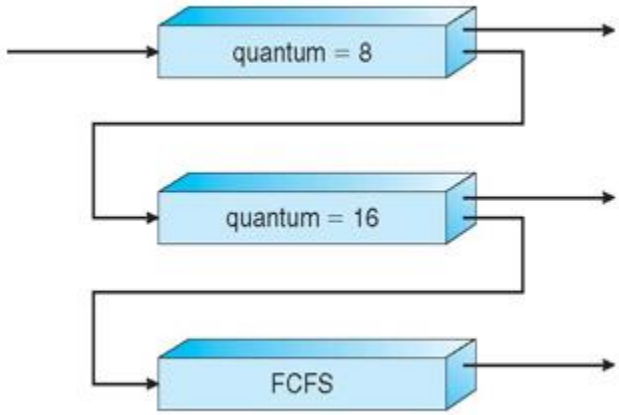


כעת נכיר עוד אלגוריתם:

Multilevel Feedback Queue-תזמון תור רב-רמתי עם פיקוח:
בשיטה זאת תהליך עשוי לנדוד לאורך חיים בין תורי התזמון השונים.
משתמשים בו כדי למנוע הרעבה של תהליכים שנמצאים בתורים בעלי עדיפות נמוכה.
המעבר בין תורים על פי צריכת המעבד של התהליך:
-מי שצורך הרבה זמן עיבוד, עובד לתורים בעלי עדיפות נמוכה.
-תהליך שנמצא הרבה זמן בתורים בעלי עדיפות נמוכה, משודרג לתורים בעלי עדיפות גבוהה יותר (ובכך נמנעת הרעבה- זהו מנגנון aging -הזדקנות).

באופן כללי, אלגוריתם זה מוגדר ע"פ הפרמטרים הבאים:

- מספר התורים.
- אלגוריתם התזמון עבור כל תור.
- השיטה על-פיה מעלים תהליך לתור בעל עדיפות גבוהה.
- השיטה על-פיה מורידים תהליך לתור בעל עדיפות נמוכה.
- השיטה למיפוי תהליכים לתורים השונים.



אחרי שראינו את האופן בו מתוזמנים התהליכים, נדבר על האופן בו מתוזמנים תהליכונים (תזכורת: כל תהליך מורכב מתהליכון או כמה).

Thread Scheduling- תזמון תהליכונים:

בעבר כבר הבדלנו בין 2 סוגי תהליכים: תהליכים ברמת המשתמש (user-level thread) ותהליכים ברמת מערכת ההפעלה (kernel-level thread) .

כאמור, תהליכים ברמת ה-user אינם מתוזמנים ע"י מערכת ההפעלה אלא ע"י ספרייה (שאנו טוענים בקוד) כך שאין למערכת ההפעלה התערבות בעניין, אבל מצד שני כדי להריץ תהליכונים של המשתמש ב-CPU חייבים לקשר אותם לתהליכונים ברמת מערכת ההפעלה (כזכור, רק מערכת ההפעלה נוגעת ב-CPU).

הקישור הנ"ל יכול להיות ישיר או עקיף באמצעות תהליך קל (**LMP- lightweight process**).

אחד ההבדלים בין תהליכונים של המשתמש לתהליכונים של מערכת ההפעלה הוא, הדרך שבה הם מתוזמנים, שכן התחרות על זמן עיבוד מתרחשת בין התהליכונים השונים השייכים לאותו התהליך.

בכל תהליך של המשתמש, הספרייה (שכללנו בקוד) עושה תזמון בין התהליכונים ובעזרת מנגנון many to many וגם many to one מקשרת את התהליכונים המתוזמנים למערכת ההפעלה (LMP) , הסכמה הזאת נקראת **"PCS" .**

כעת, תהליכוני המשתמש מופו לתהליכון (או כמה) של מערכת ההפעלה, והיא תהיה אחראית להריץ אותם ב-CPU, עכשיו נותר להחליט אילו תהליכונים ברמת מערכת ההפעלה ירוצו, מערכת ההפעלה משתמשת בסדרן שנקרא **System** "**Contention Scope- (SCS)**" כדי להחליט.

(סה"כ נקרא בשם כולל לסדרן של תהליכוני המשתמש, PCS ולסדרן של תהליכוני מערכת ההפעלה, SCS)

במחשבים החדשים לפעמים יש ארכיטקטורה של כמה ליבות במעבד (כמה מעבדים קטנים המחוברים בניהם) ואז התזמון מורכב יותר.

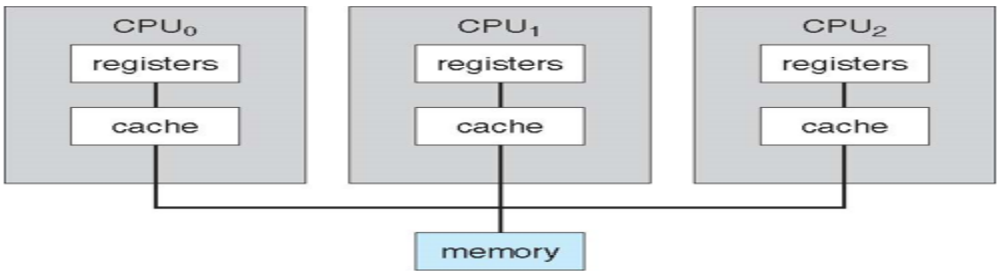
מה נרוויח מהארכיטקטורה הזאת?

- תפוקה גדולה יותר (יותר תהליכים מתבצעים בזמן X)
- חיסכון במשאבים
- אמינות- עמיד בפני תקלות

כעת נצטרך לענות על השאלה, איך מחלקים את העבודות בין היחידות השונות של ה-CPU? ובכן יש 2 שיטות:

1. **Asymmetric Multiprocessing-חלוקה לא סימטרית:** מעבד אחד (ראשי) אחראי אל התזמון וגם על פעולות ה-I/O ושאר המעבדים עובדים רק על הרצת הקוד.
   המימוש הזה פשוט היות ורק למעבד אחד יש גישה לזיכרון (מנהל טוב שמחלק את העבודה).
2. **Symmetric Multiprocessing-חלוקה סימטרית:** בגישה הזו כולם עושים את הכל.
   המימוש של השיטה יכול להכיל רק Ready Queue אחד משותף או אחד לכל אחד.

Symmetric Multiprocessing Architecture



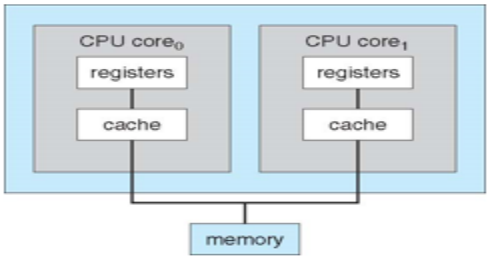
כאמור, בשיטה הסימטרית כל המעבדים קשורים לאותו הזיכרון (SMA) ואם כן , נוצרת מורכבות בחלוקת התהליכים למעבדים (לכן שיטה זו יקרה יותר).

צריך גם לעשות הבחנה האם כל המעבדים זהים מבחינה פונקציונלית, ניתן להריץ בכולם כל תהליך שנמצא ב- Ready Queue, מושג זה נקרא **"Homogeneous Processors".**

מבחינת התהליכים, יכולה להיות "זיקה" של תהליך למעבד מסוים: **"Processor Affinity"** :

- **Soft affinity-** לתהליכון יש זיקה למעבד מסוים (או לקבוצה של מעבדים) ואם לא הוקצה לשם אז מהגר למעבד אחר (גמיש).
- **Hard affinity-** התהליך מציין ע"י System Call מעבד או קבוצה שרק עליהם ירוץ (לא גמיש).

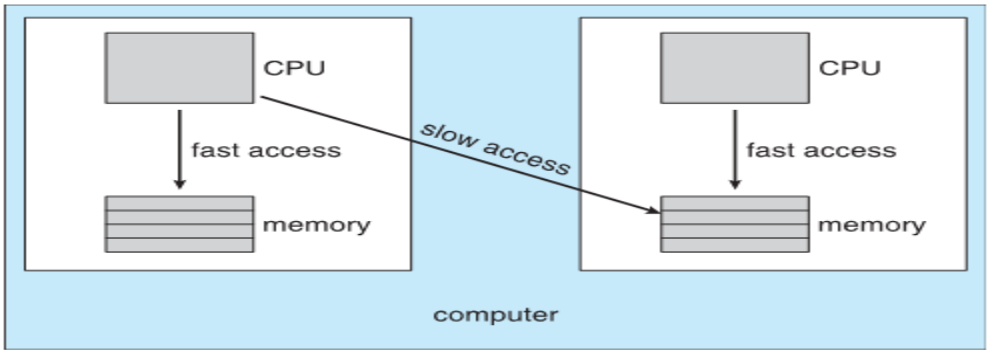
<u>Dual-Core Design – מעבד עם שתי ליבות</u>:
מעבד מרובה ליבות הוא רכיב מחשוב **יחיד** עם שתי יחידות עיבוד עצמאיות או יותר (Multicore Processors) הנקראות ליבות, אשר קוראות ומבצעות תהליכים (מעבד מרובה ליבות מיישם עיבוד מרובה בחבילה פיזית אחת).
מערכת ההפעלה מתייחסת לכל ליבה כמעבד בפני עצמו.



באופן כזה אנו מבטיחים מהירות גבוהה יותר וריבוי תהליכונים גם גבוהה יותר , בנוסף צורך פחות כוח (רכיב אחד).
נשאלת השאלה, איך הם עובדים במקביל (מה החלוקה):

- **<u>Data Parallelism- מידע משותף</u>:** חלוקה של הנתונים על כל הליבות, (הפצת הנתונים על פני צמתים משותפים שונים) הנתונים שייכים למשימה אחת.
- **<u>Task parallelism-משימות משותפות</u>**: חלוקה של תהליכונים ששייכים לכמה משימות שונות בין כל הליבות.

גישה לא סימטרית לזיכרון (MUMA): חלקים שונים של זיכרון מחוברים בהעדפה למעבד מסוים.
דבר זה יכול ליצור גישה איטית בין מעבד (ליבה) שרוצה לגשת לזיכרון שמחובר בהעדפה למעבד אחר.



פרק 6: סנכרון תהליכים

סנכרון תהליכים- Process Synchronization:

בפרק הקודם דיברנו על מצב שיש כמה מעבדים או כמה ליבות, מצב זה מאפשר עבודה **מקבילית אמתית** בין תהליכים (לא כמו ב- Multi-Programing שהמעבד בזמן נתון יכול להריץ רק תהליך אחד אז המקביליות היא שבזמן X המעבד יעבוד על Y תהליכים לסירוגין).

המקביליות האמתית מביאה אתה בעיה של סנכרון, מה יקרה עם 2 תהליכים שרצים ב-2 מעבדים שונים ירצו כעת לגשת לאותו מקום בזיכרון? מי מהם קובע?

בעיית סנכרון זו קיימת גם כשמדובר במעבד אחד כאשר משתמשים באלגוריתם התזמון Round Rubin- היות ותהליך יכול לגשת למשאב מסוים ולעדכן אותו ותוך כדי העדכון יגמר לו זמן הסבב שהוקצה וכעת תהליך אחר ייגש לעדכן את אותו המשאב.

כשמדובר בתהליכים של מערכת ההפעלה, היא אחראית לסדר בניהם, הבעיה מתעוררת התהליכים שאנו כותבים (תוכניות) היות וטעות של סנכרון יכולה להתגלות רק כאשר התוכנית הפכה לתהליך שרץ (רק בריצה).

לכן צריך שיהיה תיאום מסוים בין תהליכים (שמונע מ-2 תהליכים לגשת לאותו משאב משותף).

על מנת להמחיש את הבעיה , נזכר בבעיית "הצרכן- יצרן" :

תזכורת:

"במדעי המחשב, בעיית הצרכן- היא דוגמה קלאסית לתיאור בעיית סנכרון שני תהליכים או יותר הנגרמת במערכות המאפשרות ריבוי משימות.

הבעיה נגרמת כאשר שני תהליכים או יותר חולקים חוצץ המוגבל בגודלו. חלק מהתהליכים אחראים להכניס מידע לחוצץ (יצרנים) וחלק אחראים לשלוף ממנו מידע (צרכנים).

פתרון הבעיה דורש מנגנון שימנע משני תהליכים לעדכן בו-זמנית את אותה רשומה ,בעזרת שימוש בסנכרון בין התהליכים."

לשם הדוגמא, נשתמש ב- int שנקרא לו counter במקום ב-buffer.

נניח שזה הקוד של היצרן:

Producer

```
while (true) {
/* produce an item in next produced */

while (counter == BUFFER_SIZE) ;
        /* do nothing */
buffer[in] = next_produced;
in = (in + 1) % BUFFER_SIZE;
counter++;
}
```

ונניח שזה הקוד של הצרכן:

Consumer

```
while (true) {
     while (counter == 0)
             ; /* do nothing */
     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
        counter--;
     /* consume the item in next consumed */
}
```

הבעיה שלנו מתחילה בפקודה ++counter ובפקודה --counter.
כידוע אנו כותבים בשפה עילית, והקומפיילר ממיר את הפקודות לשפת אסמבלר כך שלמשל בדוגמא שלנו הפקודות --/++ counter מתרגמות כל אחת ל-3 פקודות:

- **counter++** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **counter--** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

מובן כעת מה יכולה להיות הבעיה, תהליך X (של היצרן) התחיל את הפקודה ++counter ובאמצע אחת מה-3 פקודות אסמבלי (שאליהן תורגמה הפקודה ++counter) התרחשה פסיקה, זה אומר שהתהליך יושהה.
כעת נכנס תהליך Y (של הצרכן) וביצע —counter, יוצא שקיבלנו ערך לא עקבי של ה-counter (עוד לא סיים את גדילתו וכבר הפחיתו ממנו):

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter`	{register1 = 5}
S1: producer execute `register1 = register1 + 1`	{register1 = 6}
S2: consumer execute `register2 = counter`	{register2 = 5}
S3: consumer execute `register2 = register2 - 1`	{register2 = 4}
S4: producer execute `counter = register1`	{counter = 6 }
S5: consumer execute `counter = register2`	{counter = 4}

בעיה זו נקראת **"בעיית הקטע הקריטי (Critical Section Problem)".**
לכל תהליך יש קטע קריטי בו הוא משנה משאבים, אם כן מה נעשה כדי שתהליך שנכנס לקטע קריטי יהיה מוגן?
כדי לפתור את הבעיה הזו אנו זקוקים לפרוטוקול, לכן:

- כל תהליך יצטרך לבקש רשות על מנת להיכנס לקטע הקריטי שלו. כניסה לקטע הקריטי-**Entry section**.
- ביציאת תהליך מהקטע הקריטי הוא מעדכן את מערכת ההפעלה ורק אז ממשיך בקוד (יתר הקוד נקרא remainder section) . יציאה מקטע קריטי **-Exit section** .

General structure of process P_i

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

הפתרון הנ"ל חייב לענות על 3 דרישות:

1. **<u>Mutual Exclusion-בלעדות הדדית</u>**: אם תהליך Pi נמצא בקטע הקריטי שלו, אף תהליך אחר לא יוכל להיכנס למצב של קטע קריטי.

2. **<u>Progress-התקדמות</u>**: יש התקדמות, אין קיפאון. התהליכים כל הזמן מתקדמים ובפרט התהליך שכעת בקטע הקריטי שלו.
אם יש מצב שאף תהליך לא בקטע הקריטי שלו ויש כמה תהליכים שמעוניינים להיכנס לקטע הקריטי שלהם אז רק התהליכים שלא נמצאים במצב remainder section (שעוד לא היו במצב הקריטי) ישתתפו בהחלטה מי יכנס קודם.

3. **<u>Bounded Waiting-הגבלת זמן המתנה</u>**: חסם עליון למספר הפעמים שתהליכים אחרים יכולים להיכנס לקטע הקריטי שלהם מהרגע שתהליך אחר רוצה להיכנס לקטע הקריטי שלו.

נשים לב שהפרוטוקול הזה לא מאפשר הרעבה.

כעת נראה אלגוריתמים שאמורים לתת פתרון לקטע הקריטי, כל אלגוריתם מניח שיש רק 2 תהליכים:

<u>אלגוריתם ראשון</u>:

```
do {

    while (turn == j);
            critical section
    turn = j;
            remainder section
} while (true);
```

בקוד הנ"ל יש בעיה של Progress (התקדמות)- כי גם במצב שבו הגיע תורו של תהליך i , (trun=Pi) ואותו תהליך לא זקוק לכניסה לקטע הקריטי שלו, האלגוריתם לא יאפשר לתהליכים אחרים להיכנס לקטע הקריטי שלהם תהליך אחר להיכנס למצב של הקטע הקריטי שלהם (אלא רק שיגיע תורם).

אלגוריתם שני:

```
repeat
        flag[i] := true;
        while flag[j] do no-op;
          critical section
        flag[i]:= false;
              remainder section
until false
```

באלגוריתם הנ"ל יש מערך בוליאני כך שעבור כל תהליך קיים דגל , אם תהליך רוצה להיכנס לקטע הקריטי שלו הוא משנה את הערך של הדגל ל-True .
הבעיה באלגוריתם, במצב ש-2 התהליכים רוצים להיכנס לקטע הקריטי באותו הזמן (אחד הדליק את דגלו->החלפת הקשר-> השני הדליק את דגלו).
הם ישנו את ערכי דגליהם ל-True ואז לפי האלגוריתם אף אחד מהם לא ייכנס למצב הקריטי.
האלגוריתם הזה שומר על מניעה הדדית אבל את 2 התנאים הנוספים אינו מקיים.

אלגוריתם שלישי:

```
repeat
        while flag[j] do no-op;
            flag[i] := true;
          critical section
        flag[i]:= false;
              remainder section
until false
```

אותו סיפור של המערך דגלים כמו מהאלגוריתם השני.
הלולאה של ה- while (רק שורה אחת) מונעת מתהליך Pi להיכנס למצב הקריטי שלו כל עוד שיש תהליך אחר Pj שגם רוצה.
הבעיה בפתרון, שאם תתבצע פסיקה בדיוק לפני השורה flag[i]=true ותהליך שכעת קיבל את זמן המעבד ישנה את ערך הדגל שלו ל-true (יכול כי Pi עוד לא הספיק לשנות את ערך דגלו) אז 2 התהליכים יוכלו להיכנס לקטע הקריטי גם יחד, אין כאן שמירה על מניעה הדדית.

טיפ מאברהם טרייסמן- תמיד לכתוב את הקוד פעמיים פעם עם i ופעם עם J וכך אפשר לבדוק את התנאים יותר טוב.

אם כן הפתרון האולטימטיבי יהיה לשלב 2 תנאים:

1.תהליך Pi יכנס למצב הקריטי רק אם אף תהליך Pj לא רוצה להיכנס

2. לא תורו של Pj:

בשביל זה נחזיק מערך בוליאני ועוד משתנה שיציין של מי התור.

```
do{
    flag[i]=true;
    while (flag[j] && turn==j);
    //do critical section
    flag[i]=false;
    //do remainder section
}
```

כל זה בהקשר של תהליכים ברמת ה-user , אבל איך מערכת ההפעלה מתמודדת עם בעיית הקטע הקריטי? ובכן יש 2 גישות למערכת ההפעלה:

1. Preemptive-זכות מנע: מערכת ההפעלה מתערבת (יכולה להחליף בין תהליכים לגבי הזמנים הקריטיים).
2. Non-preemptive- אין החלפה של תהליכים, כל עוד הוא רץ מניחים לו.

ישנה עוד אפשרות לפתור את בעיית הקטע הקריטי, ע"י חומרה:

Synchronization Hardware- סנכרון ע"י חומרה:

ישנה אפשרות למנוע את בעיית הקטע הקריטי באמצעות חומרה, אם לא נאפשר שתגיע פסיקה ברגע שמשתנה משותף מעודכן אז לא אמורה להיות לנו שום בעיה (פתרון זה תומך בעיקר במערכות עם מעבד אחד כי לוקח זמן להודיע לכמה מעבדים לא לקבל פסיקות כרגע).

לכן מערכות רבות מספקות לנו "**הוראות חומרה-lock**" (פקודה שמתבצעת באופן אוטומטי בלי אפשרות להפריע לה **Atomic**) שמאפשרות לנו לבדוק או לעדכן תוכן של משתנה. כמובן שנעזר בזה לפתרון בעיית הקטע הקריטי.

זהו המבנה של הפתרון:

```
do {
    acquire lock
            critical section
    release lock
            remainder section
} while (TRUE);
```

כעת נגדיר את פקודת החומרה– "test and set" :

מציבה ב-target את הערך True ומחזירה את הערך המקורי של target

```
boolean test_and_set (boolean *target)
  {
      boolean rv = *target;
      *target = TRUE;
      return rv:
  }
```

את הפונקציה "test and set" נשים ב-acquire lock :

```
do {
      while (test_and_set(&lock))
      ; /* do nothing */
          /* critical section */
   lock = false;
          /* remainder section */
 } while (true);
```

בהתחלה lock= False עד שתהליך Pi רוצה להיכנס לקטע הקריטי שלו, אז הוא יפעיל את הפונקציה של החומרה- "test and set" וכעת lock=True, כעת הוא יכנס לקטע הקריטי ורק כשיסיים lock יחזור להיות False, כעת תהליך אחר יכול להיכנס לקטע הקריטי שלו. (צריך לשים לב שהwhile הוא רק שורה אחת, מונע מתהליך אחר להיכנס).

עוד פתרון לסנכרון בקטע הקריטי בין תהליכים, יהיה:

Semaphore-סמפור:
מנגנון לסנכרון תהליכים הפועלים יחד במחשב (מניעה הדדית), למעשה זאת הרחבה של מנגנון ה-lock .

ויקיפדיה: "כיום נהוג לפתור בעיה זו על ידי שימוש בסמפור :מעין דגל (flag) אשר כדי לשנות את ערכו התהליך נדרש לבקש זאת ממערכת ההפעלה. במידה והדגל כבר "נתפס" על ידי תהליך אחר, מערכת ההפעלה תעביר את התהליך ל"מצב שינה", ו"תעיר" אותו על ידי משלוח פסיקה ברגע שהדגל "משוחרר" על ידי התהליך שתפס אותו.

בצורה זו מתאפשר סנכרון ללא בזבוז זמן מעבד, וכן מובטח לנו כי מערכת ההפעלה מפקחת על מצב הדגל ושינוי של ערכו מבוצע בצורה אטומית (לא ניתן לחטוף לתהליך את המעבד בזמן שינוי ערך הדגל).

אופן הפעולה: הסמפור הוא משתנה היכול לקבל ערכים שלמים לא שליליים (במקרה של סימפור בינארי הוא יכול לקבל שני ערכים בלבד, 0 ו-1).

על הסמפור מוגדרות שתי פעולות אטומיות:

1. אם ערך הסמפור גדול מ-0 הקטן אותו ב-1 והמשך, אחרת המתן עד שיהיה גדול מ-0, ואז הקטן אותו ב-1 והמשך.
2. הגדל את ערך הסמפור ב-1.

דוגמה לשימוש בסמפור בינארי על מנת לפתור את בעיית המניעה ההדדית כאשר קיימים שני תהליכים החולקים משאב משותף. הסמפור יאותחל ל-1. לפני כניסה של כל תהליך לקטע הקריטי תתבצע הפעולה הראשונה, וביציאה ממנו הפעולה השנייה. "

2 הפעולות האוטומטיות שקראנו מויקפדיה הן:

- Wait()- מקבל את הסמפור (S), אם קטן או שווה לאפס- מחכים. אחרת עושים --S.

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Signal()- ++S.

```
signal(S) {
    S++;
}
```

נדגיש שרק בעזרת 2 הפעולות האטומיות (לא ניתן להפריע להן באמצע ע"י פסיקות) אפשר לשנות את ערכו של S.

כפי שקראנו בויקפדיה ישנם 2 סוגים של סמפור:

- **Counting semaphore**- בסמפור מניה יש משתנה counting (S) ובנוסף יש List של תהליכים שרוצים להיכנס לקטע הקריטי שלהם, והערך של S קטן מ-0 אז אותם תהליכים יכנסו למצב שינה (sleep) וכאשר הערך של S יחזור להיות חיובי הם יכנסו (נשים לב שמספר התהליכים שרוצים להיכנס לקטע הקריטי הוא בדיוק |S-|).
כאמור Wait() דואג "להרדים ולהעיר" (במידת הצורך) את התהליכים שדורשים קטע קריטי.
ו-Signal() דואג לעדכן יציאה של כל תהליך מהקטע הקריטי (S++).
- **Binary semaphore**-בסמפור בינארי ערך הסימפור יכול להיות רק 0/1 .
אפשר להסתכל על זה כמו תא שירותים יחיד במקום הומה אנשים, כאשר תהליך נכנס לקטע הקריטי (נכנס לתא השירותים) הוא נועל אחריו את הדלת (S=0) וכאשר יוצא מהקטע הקריטי משאיר את הדלת פתוחה לתהליך הבא (S=1).

Semaphore Implementation- יישום הסמפור:

```
wait(s);
//כאן נמצא הקטע הקריטי
signal(s);
```

עד כה הפתרונות שראינו למניעה הדדית השתמשו בלולאת while שדאגה למנוע מתהליך Pj להיכנס לקטע הקריטי שלו עד שתהליך Pi יסיים- דבר זה נקרא, **" המתנה פעילה-Busy waiting".**
הבעיה בהמתנה פעילה היא, שכל עוד שתהליך Pi רץ, Pj יהיה תקוע בלולאת ה-while (של שורה אחת) וידרוש זמן מעבד (למרות שלא מתקדם).
גם כאן נשארנו עם בעיית ה-Busy waiting.
כדי להתגבר על בעיית ההמתנה הפעילה, צריך לעדכן את הפעולות האטומיות (wait, signal) כך:
נממש את הסימפור כמבנה:

```
typedef struct{
int value;
struct process *list;
} semaphore;
```

לכל semaphore יש ערך מספרי ורשימת תהליכים.

את פעולת ה-wait נשדרג בכך שנוסיף לה פעולה בשם **"block()"** שהיא תהיה אחראית להעביר את התהליכים שרוצים להיכנס לקטע הקריטי (אבל אי אפשר כי יש כבר אחר) אל ה-List, ובכך הם לא דורשים זמן מעבד (ממלא לא יכולים להתקדם).

```
wait(semaphore *S) {
   S->value--;
   if (S->value < 0) {
      add this process to S->list;
      block();
   }
}
```

את פעולת ה-signal נשדרג בכך שנוסיף לה פעולה בשם **"wakeup()"** שהיא תהיה אחראית להוציא מה-List את אחד מהתהליכים החסומים ולהעביר אותו ל-Ready Queue.

```
signal(semaphore *S) {
   S->value++;
   if (S->value <= 0) {
      remove a process P from S->list;
      wakeup(P);
   }
}
```

Deadlocks and Starvation-בעיית קיפאון והרעבה:

Deadlock-קיפאון: מבוי סתום, 2 (או יותר) תהליכים שכל אחד מהם משתמש במשאב משותף שונה ובנוסף כל אחד מהם גם רוצה להשתמש במשאב שהשני מחזיק,

אז נוצר מצב של קיפאון כלומר, אף תהליך לא יכול לבצע את מטרתו כי הראשון מחכה עד שהוא יקבל את המשאב שהשני משתמש בו כרגע. והשני לא יסיים להשתמש במשאב כי הוא צריך את המשאב שהראשון משתמש בו כדי לסיים לעשות את מטרתו.

מצב זה מתאפשר בגלל המניעה ההדדית שמקנה לכל תהליך בלעדיות בקטע הקריטי שלו.

```
P0                    P1
   wait(S);                  wait(Q);
   wait(Q);                  wait(S);
...                   ...
   signal(S);                signal(Q);
   signal(Q);                signal(S);
```

הרעבה: תיתכן במצב שבו ,תהליך נתקע בעקבות הסמפור בתור המתנה (ע"י פעולת ()block) ולא יכול לצאת .

Priority inversion: יכולה להיווצר בעיה כאשר יש תהליך עם עדיפות נמוכה בתור ההמתנה מחזיק במשאב שתהליך עם עדיפות גבוהה צריך, מכיוון שזה תוקע את הרצת התהליכים עושים "ירושת עדיפות" והתהליך שהיה בעל העדיפות הנמוכה מקבל את העדיפות של התהליך בעל העדיפות הגבוהה שזקוק לו.

כעת נציג מספר בעיות סנכרון שנפתרות ע"י שימוש בסמפור:

Readers-Writers Problem- בעיית הקוראים-כותבים:

ויקיפדיה: "בעיית הקוראים כותבים נוצרת כאשר יש מידע ושני סוגי תהליכים שנגשים למידע. תהליך קורא - קורא את המידע מבלי לשנות אותו. תהליך כותב - יכול לקרוא את המידע וגם לשנות אותו. כאשר הכותב ניגש למידע אז אסור לשום תהליך אחר לגשת למידע במקביל כדי שלא יתקבל מידע שגוי. כאשר הקורא ניגש למידע אז ניתן לאפשר גם לקוראים אחרים לגשת למידע והקריאה לא מהווה קטע קריטי ביחס לקוראים אחרים. תתכן כאן בעיה של הרעבת הכותבים כי ברגע שיהיו הרבה קוראים אז הכותב לא יצליח לגשת אל המידע כי סביר שיהיה לפחות קורא אחד שמשתמש כרגע במידע."

על מנת לפתור את הבעיה נגדיר 3 משתנים (2 סימפורים ומונה):

- rw_mutex- הוא הסימפור המאפשר גישה של הכותבים ל-Buffer, מאותחל ב-1.(אישור הכניסה של הכותבים)
- mutex- הוא הסימפור המאפשר כניסה לקוראים (כאמור, אין בעיה שכמה קוראים יקראו יחד) .
- read_count- מונה שסופר כמה קואים יש כעת ב-Buffer.

כעת נציג את הקוד של הקוראים ואת הקוד של הכותבים:

הקוד של הכותבים

```
do {
      wait(rw_mutex);
          ...
      /* writing is performed */
          ...
    signal(rw_mutex);
} while (true);
```

כאשר כותב נכנס (ניגש ל-Buffer) הוא נועל אחריו את הדלת Wait(rw_mutex) כדי שכותב אחר לא יכנס (אחרי שחיכה אם היה צורך).
וכאשר הוא מסיים הוא משאיר את הדלת פתוחה ע"י signal(rw_mutex).

הקוד של הקוראים

```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
         wait(rw_mutex);
      signal(mutex);
          ...
        /* reading is performed */
          ...
      wait(mutex);
        read count--;
        if (read_count == 0)
      signal(rw_mutex);
      signal(mutex);
} while (true);
```

כשהקורא נכנס מגדיל את המונה (יכול להיות שלפני הוא חיכה) אם המונה שווה ל-1 סימן שהוא הקורא היחיד שניגש ל-Buffer כך שסביר להניח שהדלת של הכותבים פתוחה, אז הקורא ינעל אותה עי" wait(rw_mutex) וכעת יכולים להיכנס כמה קוראים גם יחד.
כאשר נותנים עדיפות לקוראים יתכן מצב של הרעבה לכותבים.

עוד בעיה שנפתרה ע"י סמפור:

Dining Philosopher Problem-בעיית הפילוסופים הסועדים:
ויקיפדיה: "חמישה פילוסופים יושבים סביב שולחן עגול ומבלים את זמנם בחשיבה ובאכילה .במרכז השולחן ישנה קערת ספגטי גדולה, ולכל אחד מהפילוסופים יש צלחת .בין כל שתי צלחות נמצא מזלג ,כך שישנם בסך הכול חמישה מזלגות.

על מנת לאכול את הספגטי, כל פילוסוף זקוק לשני מזלגות (בניסוחים אחרים של הבעיה המזלגות מוחלפים במקלות אכילה .(כל פילוסוף יכול להרים רק את שני המזלגות הקרובים אליו (אלו שמימין ומשמאל לצלחת שלו) ואינו יכול להרים את שניהם בו-זמנית - כלומר, עליו להרים את אחד מהם, ורק לאחר מכן את השני. תנאי נוסף הוא שלא מתקיימת כל תקשורת בין הפילוסופים.

שתי הבעיות הבסיסיות שעלולות להתהוות הן קיפאון והרעבה .מצב של קיפאון מתרחש כאשר כל הפילוסופים נתקעים במצב כלשהו ממנו אינם יכולים לצאת. למשל, אם הכלל שעל פיו הפילוסופים פועלים אומר כי כאשר הם רוצים לאכול עליהם להרים את המזלג שמימינם ולחכות עד שהמזלג משמאלם יתפנה, עלול להיווצר קיפאון אם כל הפילוסופים ירצו לאכול בו-זמנית - כולם ירימו את המזלג שמימינם ולאחר מכן יחכו לנצח למזלג שמצד שמאל (שכן מזלג זה מוחזק בידי פילוסוף אחר, שגם הוא מחכה שיתפנה המזלג משמאלו, וכן הלאה בצורה מעגלית).

הנמשל מובן, תיאור של מצב קיפאון.

איך נפתור מצב שכזה בעזרת סמפור?
ניצור מערך של 5 Semaphores שנקרא לו [5]chopstick (כמו המזלגות- כל תא i במערך מאשר לתהליך i לקחת את המזלגות).
כעת כל פילוסוף שרוצה לאכול (כל תהליך שרוצה לגשת למשאב המשותף אבל זקוק למשאבים של 2 התהליכים הסמוכים לו) יעשה ()Wait ל-2 התהליכים שסמוכים לו (כך יכול להשתמש במשאבים שהם תפסו והוא היה זקוק להם), הקוד:

- The structure of Philosopher *i*:

```
do {
     wait (chopstick[i] );
   wait (chopStick[ (i + 1) % 5] );

              //  eat

   signal (chopstick[i] );
   signal (chopstick[ (i + 1) % 5] );

                  //  think

} while (TRUE);
```

אומנם באופן כזה תיתכן בעיה של קיפאון, במצב שבו כולם ניסו לאכול ביחד ואז כל אחד תפס מזלג אחד ומחכה לנצח למזלג השני.

אם כן, נגדיר מספר חיזוקים לפתרון:

- ✓ נאפשר רק ל-4 פילוסופים לשבת ליד השולחן (רק 4 תהליכים ב-Multiprogramming בהנחה שיש 5 תהליכים במצב הנ"ל)
- ✓ לאפשר לפילוסוף להרים מזלג רק במצב ששני המזלגות פנויים.
- ✓ לבצע את הרמת המזלגות הסמוכים באופן א-סימטרי (פעם מימין ופעם משמאל).

כעת נראה את הבעיה האחרונה שהסמפור פתר:

<u>Bounded-Buffer Problem- בעיית היצרן צרכן</u>:
תזכורת: במדעי המחשב, בעיית הצרכן- היא דוגמה קלאסית לתיאור בעיית סנכרון שני תהליכים או יותר הנגרמת במערכות המאפשרות ריבוי משימות. הבעיה נגרמת כאשר שני תהליכים או יותר חולקים חוצץ המוגבל בגודלו. חלק מהתהליכים אחראים להכניס מידע לחוצץ (יצרנים) וחלק אחראים לשלוף ממנו מידע (צרכנים).

נניח שיש לנו n – Buffers וניתן לשמור בהם רק n פריטים.
נגדיר 3 משתני סמפור:
mutex- הוא הסימפור שמאפשר גישה ל-Buffer מאותחל ב-1 (מסנכרן הין היצרן לצרכן).
full- הוא הסימפור שציין את כל ה-Buffers המלאים מאותחל ל-0.
empty- הוא הסימפור שמציין את כל ה-Buffers הריקים מאותחל ל-n.

קוד היצרן

```
do {
      ...
      /* produce an item in next_produced */
      ...
    wait(empty);
    wait(mutex);
      ...
      /* add next produced to the buffer */
      ...
    signal(mutex);
    signal(full);
 } while (true);
```

רואים בהתחלה (שורה ראשונה) שהיצרן יוצר item וכעת הוא צריך לאחסן אותו באחד מה-Buffers אז הוא יעשה wait(empty) כי צריך לעדכן שיש Buffer אחד פחות, אם כן הוא צריך לחסום את הגישה ל-Buffer (כדי לאחסן) ולכן הוא יעשה wait(mutex) וכעת הוא יאחסן ואז יפתח את הגישה ל- Bufferולכן יעשה signal(mutex) ואז נעשה signal(full) כדי להעלות את מספר המלאים ב-1.

קוד הצרכן

```
Do {
    wait(full);
    wait(mutex);
      ...
      /* remove an item from buffer to next_consumed */
      ...
    signal(mutex);
    signal(empty);
      ...
      /* consume the item in next consumed */
      ...
    } while (true);
```

נסה להבין את הצרכן לבד.

פרק 7: קיפאון

Deadlocks-קיפאון(חסימה הדדית):
תזכורת: מבוי סתום, 2 (או יותר) תהליכים שכל אחד מהם משתמש במשאב משותף (Cpu, סמפור, זיכרון, התקן קלט פלט וכו') שונה ובנוסף כל אחד מהם גם רוצה להשתמש במשאב שהשני מחזיק, אז נוצר מצב של קיפאון כלומר, אף תהליך לא יכול לבצע את מטרתו כי הראשון מחכה עד שהוא יקבל את המשאב שהשני משתמש בו כרגע. והשני לא יסיים להשתמש במשאב כי הוא צריך את המשאב שהראשון משתמש בו כדי לסיים לעשות את מטרתו.
מצב זה מתאפשר בגלל המניעה ההדדית שמקנה לכל תהליך בלעדיות בקטע הקריטי שלו.

עוד ניסוח יהיה: קיפאון הוא מצב שבו קיימת קבוצה של n תהליכים, וכל תהליך בקבוצה מחכה (חסום) לאירוע שיכול להיגרם רק על ידי מי מחברי הקבוצה.

בררו לנו שבמצב קיפאון תיתכן גם הרעבה.

תהליך רשאי לבקש מספר בלתי מוגבל של משאבים והוא האחראי לשחרר אותם בתום השימוש, אם לא ניתן להפנות את המשאב לתהליך הוא ימתין עד לקבלתו.
כעיקרון תהליך יכול להמצאות ב-3 סטטוסים שונים בהקשר של משאבים:
Request-התהליך מבקש משאב, אם לא יקבל הוא יכנס ל-Waiting Queue עד שיקבל.
Use- התהליך משתמש/ רץ על גבי המשאב.
Release- התהליך משחרר את המשאב בסיום השימוש.

כל אחד מ-3 הפעולות הללו מתבצעות בעזרת System Call.

כעת נדבר על התנאים ההכרחיים למצב קיפאון:
Deadlock Characterization:
ישנם 4 תנאים הכרחיים (שקורים במקביל) כדי שנגיע למצב של קיפאון:

1. **Mutual Exclusion-בלבדיות**: צריך שיהיה משאב אחד לפחות שלא ניתן להקצאה במקביל לכמה תהליכים. (רק ככה תיווצר המתנה).

2. **Hold And Wait-החזקה והמתנה**: צריך שתהליך יחזיק במשאב X וימתין למשאב Y שמוחזק בידי תהליך אחר. (שוב רק ככה ייווצר מעגל של ממתינים).

3. **No preemption-אי חטיפה**: לא ניתן לחטוף משאב מתהליך המחזיק בו- על המחזיק לשחררו ביוזמתו.

4. **Circular Wait- המתנה מעגלית:** עבור קבוצה {P0……Pn} של תהליכים שממתינים למשאבים, צריך שייתקיים ש-Pn ממתין למשאב שנמצא אצל P1 , ו- P1 ממתין למשאב שנמצא אצל P2 וכן הלאה.
(אחרת Pn יוכל לסיים ואחריו Pn-1 וכן הלאה).

System Resource Allocation Graph- גרף הקצאת משאבים:

גרף הקצאת משאבים משמש לתיאור מצב המערכת (תהליכים, משאבים, בקשות תלויות, הקצאות).
בהינתן גרף ניתן יהיה לזהות האם המערכת במצב של קיפאון או לא.
הגרף מורכב מקדקודים ומצלעות כאשר יש שני סוגי קדקודים:

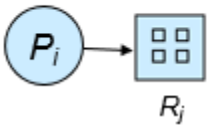
- Process

- Resource Type with 4 instances

הקדקודים שבצורת עיגול הם תהליכים ואילו הקדקודים שבצורת ריבוע אלו המשאבים.
כל ריבוע קטן (שבתוך הריבוע שמאפיין משאב) הוא מופע של המשאב- **"instance"** (ז"א שלמשל בריבוע הנ"ל יש 4 ריבועים קטנים וזה אומר שיש 4 מופעים של המשאב (כניסות) הנ"ל שיכולים להתחלק ל-4 תהליכים).

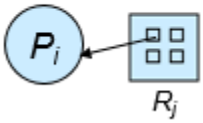
כאשר תהליך מבקש משאב תיווצר "קשת בקשה" כזאת:

P_i requests instance of R_j



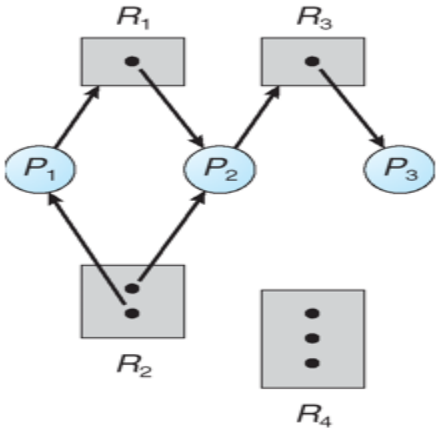
כאשר תהליך מקבל את המשאב אליו הוא הצביע "קשת הבקשה" הופכת להיות "קשת הקצאה" כזאת:

P_i is holding an instance of R_j

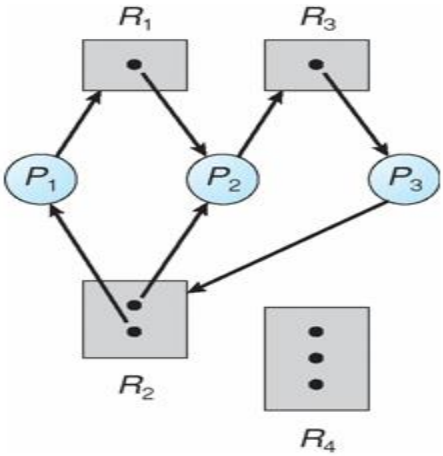


כל זה יפה, אבל איך זה עוזר לנו לגלות מצב של קיפאון?
בכלליות נאמר שתנאי הכרחי לdeadlock הוא קיומו של מעגל בגרף, אם קיים מעגל נבדוק האם ההמתנה היא מעגלית, אם כן הרי לנו מצב של קיפאון.

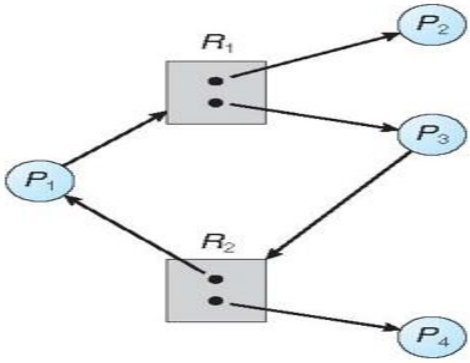
נסתכל במצב הנ"ל:



אומנם קיים מעגל אבל ההמתנה היא לא מעגלית, כי P3 יכול לססים ואז P2 ואז P1.
אבל מה יקרה אם P3 יזדקק למשאב R2 (אין למשאב R2 עוד Instances פנויים) ?
ובכן תיווצר המתנה מעגלית והמערכת תיכנס למצב של קיפאון:



נשים לב שאין המתנה מעגלית בגרף הנ"ל ולכן לא יהיה כאן מצב של קיפאון:



במקרה שבו יש רק משאב יחיד מכל סוג (instance אחד) ובנוסף יש מעגל בגרף בוודאי יש קיפאון.

<u>שיטות לטיפול ב-Deadlock:</u>
קיימות 3 דרכים לטפל בבעיית הקיפאון:

1. מניעה/ הימנעות- פיתוח פרוטוקולים שימנעו מראש מהמערכת (מתהליכים שבה) להיכנס למצב של קיפאון, פרוטוקול **מניעה (Deadlock prevention)** -דואג שלפחות אחד מהתנאים למצב קיפאון לא יתקיים. ואילו פרוטוקול **הימנעות (Deadlock avoidance)-** דואג שמערכת ההפעלה תדע מראש מהם המשאבים הנצרכים לכל תהליך ובעזרת מידע זה תוכל למנוע את מצב הקיפאון.
2. התאוששות-התהליכים רצים באופן חופשי, מערכת ההפעלה עוקבת אחרי "המצב" .
כאשר מתברר בדיעבד שנכנסנו למצב של קיפאון – מתאוששים מהמצב ע"י העפת חלק מהתהליכים.
3. התעלמות- מתעלמים מהבעיה (מניחים שהמשתמש ישים לב שהתהליך תקוע ויעיף אותו ידנית) זאת השיטה המקובלת כיום.

נרחיב קצת בעניין פתרון מניעה (Deadlock prevention), ונראה איך אנחנו יכולים לפתור לפחות אחד מהתנאים (שכאמור צריכים להתקיים בו זמנית) של כניסה למצב קיפאון:

- <u>Mutual Exclusion-בלבדיות</u>: (כאמור, תנאי זה אמר שצריך שלפחות משאב אחד לא יהיה ניתן לחלוק) תנאי זה מתייחס למשאבים אותם ניתן לחלוק, למשל: מדפסת לא יכולה להדפיס 2 דפים ביחד והמעבד לא יכול להריץ 2 תהליכים בו זמנית, **מנגד** מספר הקוראים כן יכול לקרוא קובץ במקביל.
לכן המסקנה שלא ניתן למנוע את תנאי הבלבדיות באופן מלא.

- <u>Hold And Wait- החזק והמתן</u>: (כאמור תנאי זה אמר שעל תהליך להחזיק במשאב x וימתין למשאב y שמחוזק ע"י תהליך אחר) כדי להבטיח שתנאי זה לא יקרה יש 2 פתרונות:
"הכל או כלום"- תהליך נדרש לבקש מראש את כל המשאבים שהוא צריך בו זמנית ורק אם כולם זמינים בעבורו, יקבלם.
"החלשת הדרישה"- תהליך יורשה לבקש משאב רק אם הוא לא מחזיק ברשותו משאבים כל שהם.
חסרונות:
-התהליך צריך לבקש משאב בלתי צפוי.
-חשש להרעבה של תהליך הזקוק למשאב "מבוקש".
-ניצול המשאבים עלול להיות נמוך משום שיכול להיות מצב שמשאבים רבים הוקצו אבל התהליכים עדיין לא משתמשים בהם.

- <u>Preemption- אי חטיפה</u>: (כאמור תנאי זה אמר שאסור לחטוף משאבים מתהליכים אחרים) כדי להבטיח שתנאי זה לא יקרה נשתמש בשיטה הבאה, אם תהליך מסוים מבקש משאב בזמן שהוא מחזיק בכמה משאבים אחרים, אך המשאב שביקש אינו זמין, הרי שיילקחו לו גם שאר המשאבים בהם החזיק.
לאחר מכן המשאב עובר לתור המתנה עבור אותם משאבים ששחרר ורק מתי שיוקצו לו כל המשאבים ששחרר **וגם** המשאב אליו היה זקוק רק אז ירוץ.

- <u>Circular Wait-המתנה מעגלית</u>: (כאמור , תנאי זה אמר שעל תהליך Pn לחפוץ במשאב שמוחזק ע"י תהליך P1, ועל תהליך P1 לחפוץ במשאב שמוחזק ע"י תהליך P2 וכן הלאה) כדי לדאוג שתנאי זה לא יקרה, נסדר את כל המשאבים בסדר מסויים וניתן לכל משאב מספר ייחודי (index), כעת נאפשר שכל תהליך יוכל לבקש אך ורק משאבים שהם באינדקס גדול יותר מהמשאבים שהתהליך הקודם ביקש.
כלומר אם תהליך Pi דרש משאבים עד אינדקס t אז תהליך Pi+1 יוכל לדרוש משאבים בעלי אינדקס הגדול שווה מ-t+1.

כעת נרחיב קצת בעניין פתרון ההימנעות (Deadlock avoidance) ונראה כציד ניתן לממשו.
כפי שאמרנו שיטה זו דרשה לדעת מיהם המשאבים אותם כל תהליך צריך לפני שהם מוקצים לו:

1. כל תהליך יצהיר בתחילת ריצתו מיהם המשאבים בהם הוא עשוי לעשות שימוש.

2. מערכת ההפעלה תפיל אלגוריתם חכם שמשתמש במידע זה כדי להימנע ממצב קיפאון בהמשך, כלומר תהליך P עלול לא לקבל משאב R מפני שההקצאה שלו עלולה להכניס את המערכת למצב קיפאון **בהמשך**.
החיסרון פה הוא הטלת תנאים קשים מידי על התהליכים (חשש שתהליכים לא יצליחו לרוץ) לכן:

3. נגדיר "**מצב בטוח"-** " אני בטוח שתהליכים שכרגע רצים לא יגיעו לחסימה הדדית" (פירוט מיד).

<u>Safe State-מצב בטוח</u>:
בכל תרחיש אפשרי, יש ברשות המערכת די משאבים לכך שהיא תוכל לאפשר לתהליכים שרצים לקבל את מה שהם הצהירו שהם עשויים לרצות (לא דווקא בו זמנית אלא, אולי בזה אחר זה).
כלומר בצורה פורמלית:
מערכת שמריצה n תהליכים נמצאת במצב בטוח ביחס למשאבים שכבר הוקצו, אם ורק אם קיימת סדרה של תהליכים <P1…….Pn> כך שלכל Pi בסדרה מתקיים שכמות המשאבים ש-Pi ראשי לדרוש **קטנה או שווה** מכמות המשאבים הפנויים + כמות המשאבים שמוקצים לתהליכים שלפניו בסדרה.

המחשה, נניח שיש 2 תהליכים {P1,P2} ובנוסף נניח שיש רק 2 סוגי משאבים {R1,R2} שלכל סוג יש כמות של 2.
נתון ש- P1 רוצה 2 יחדות מ- R2 ויחידה אחת מ-R1,
P2 רוצה 2 יחידות מ- R1 ויחידה 1 מ-R2.
עוד נתון שהמערכת כבר הספיקה להקצות ל-P1 יחידה אחת מ-R2 ול- P2 יחידה אחת מ-R1 :

מצב נוכחי			
תהליך	משאב	עשוי לבקש	מוקצה כרגע
P1	R1	1	0
	R2	2	1
P2	R1	2	1
	R2	1	0

תמונת המשאבים			
משאב	כמות	כרגע מוקצה	כרגע פנוי
R1	2	1	1
R2	2	1	1

האם במקרה זה המערכת במצב בטוח?
התשובה היא **ברור שכן**, כי המערכת יכולה להקצות ל-P1 את היחידה הנוספת של R2 ואת היחדה מ-R1 שלהם הוא זקוק, וכשיסיים המערכת תפנה את המשאבים ל-P2 (או להיפך).

מה היה קורה אם מערכת ההפעלה הייתה מקצה ל- P1 יחדה מכל משאב ול- P2 גם יחדה מכל משאב?

מצב נוכחי			
תהליך	משאב	עשוי לבקש	מוקצה כרגע
P1	R1	1	1
	R2	2	1
P2	R1	2	1
	R2	1	1

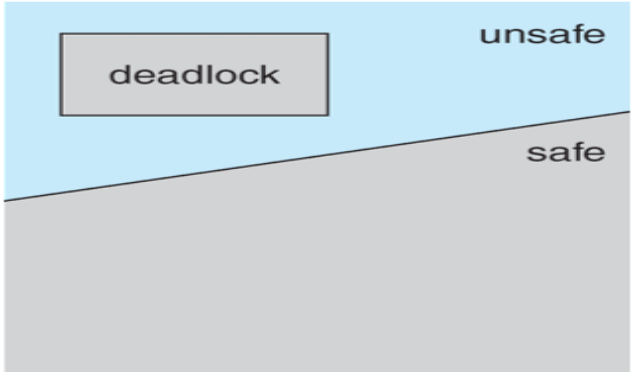
תמונת המשאבים			
משאב	כמות	כרגע מוקצה	כרגע פנוי
R1	2	2	0
R2	2	2	0

במצב כזה ברור, שהמערכת לא במצב בטוח כי P1 עלול להזדקק לעוד יחידה מ-R2 ולמערכת לא תהיה אפשרות לספק לו (כנ"ל לגבי P2).

עובדות:

- אם המערכת "במצב בטוח" לא יהיה קיפאון (בהכרח 4 התנאים לקיפאון לא יתקיימו).
- אם המערכת במצב לא בטוח **ייתכן** מצב קיפאון.
- הימנעות מבטיחה שהמערכת לא תיכנס למצב לא בטוח.

להלן הדיאגרמה שממחישה את העובדות:

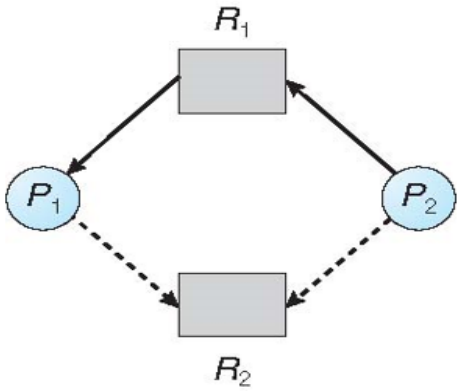


כעת נתאר אלגוריתם למימוש פתרון ההימנעות, נשתמש בגרף שתיארנו קודם ונוסיף עוד סוג של צלע:
Claim edge- צלע היתכנות: Pi->Rj מציינת ש-Pi **עשוי** לבקש את Rj במהלך ריצתו.
את קשת ההיתכנות מציירים בקו מקווקוו:

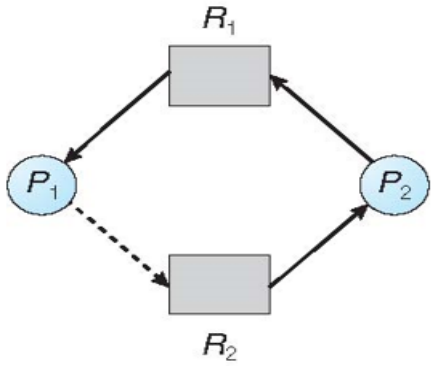
- תהליך מבקש משאב- קשת ההיתכנות הופכת לקשת בקשה
- תהליך מקבל משאב- קשת הבקשה הופכת לקשת הקצאה
- תהליך משחרר משאב- קשת ההקצאה הופכת לקשת היתכנות

כעת כאשר תהליך מבקש משאב, ההקצאה תאושר רק אם הפיכת צלע הבקשה לצלע הקצאה לא תיצור מעגל בגרף .

קצת המחשה:



P1 ו- P2 עשויים לרצות את R2.



P1 עשוי לרצות את R2, אם קשת ההיתכנות תהפוך לקשת בקשה וייצר מעגל ולכן אם P1 ידרוש את R2 נהיה במצב לא בטוח .

אלגוריתם זה מתאים לסיטואציה שיש רק instance (כניסה/ יחידה /מופע) אחד לכל משאב.
כעת נכיר אלגוריתם שמתאים למצב שיש לכל משאב כמה instances:

<u>The Banker's Algorithm-אלגוריתם הבנקאי:</u>
השם של האלגוריתם נובע מהדימוי, האם הבנק יכול להקצות אשראי/ משאב נוסף ועדיין להישאר במצב בטוח?
סימונים:
n- מספר התהליכים.
m- מספר סוגי המשאבים.

מבני הנתונים:
[m]work- מערך ששומר כמה יחידות נותרו מכל משאב בידי המערכת.
[m][n]max- מציין עבור תהליך P כמה משאבים מסוג R הוא עשוי לצרוך (לכל היותר).
[m][n]allocation- מציין עבור תהליך P כמה משאבים כבר הוקצו לו.
[m][n]need- מציין עבור תהליך P לכמה משאבים מסוג R הוא עוד עשוי להזדקק (מיותר אפשר לחשב).
[n]finish- מציין האם תהליך הוסף כבר לסדרה בטוחה (כלומר אפשר להניח שהוא סיים או יסיים בבטחה).

הרעיון באלגוריתם הבנקאים הוא לוודא שכל הקצאה מותירה את המערכת במצב בטוח, לכן האלגוריתם מכיל בתוכו תת אלגוריתם **"Safety Algorithm"** שבודק האם הקצאת משאבים לתהליך משאירה את המערכת במצב בטוח.
חשוב להבין שהבדיקה מתבצעת **טרם** שמאשרים את ההקצאה:
נשנה את מצב המערכת **כאילו** המשאבים כבר הוקצו ונבצע את הבדיקה ע"י תת האלגוריתם, אם תת האלגוריתם יחזיר תשובה שהמערכת במצב בטוח נקצה את המשאבים לתהליך, אם לא אז לא.

הסבר אינטואיטיבי לאלגוריתם: רצים בלולאה כל עוד יש התקדמות:
1. נחפש תהליך שכל המשאבים שהוא עשוי לבקש נמצאים בידי המערכת ואפשר להקצתם.
2. אם נמצא תהליך כזה נסמן אותו כ-"סיים" (למעשה יוכל לסיים) ולפיכך לשחרר את משאביו.
אם בסיום הלולאה כל התהליכים מסומנים ב-"סיים" אז המצב בטוח, אחרת לא.

הקוד (כאמור n-תהליכים, m- משאבים):

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work = Available

   Finish [i] = false **for** ***i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish [i] = false***

   (b) $\boldsymbol{Need_i \leq Work}$

   If no such ***i*** exists, go to step 4

3. $\boldsymbol{Work = Work + Allocation_i}$
   Finish[i] = true
   go to step 2

4. If ***Finish [i] == true*** for all ***i***, then the system is in a safe state

בפסקה 1, איחול, כל המשאבים מסומנים כזמינים וכל התהליכים מסומנים שעוד לא "סיימו".
בפסקה 2, מחפשים את התהליך שעונה על הדרישות שהסברנו. אם מצאנו עוברים לפסקה 3, אם לא לפסקה 4.
בפסקה 3, מוסיפים את המשאבים שהתהליך שיכול "לסיים" תפס לרשימת המשאבים הזמינים וחזרה לפסקה 2.
בפסקה 4, בדיקה האם כולם "סיימו" (מצב בטוח) או שמא לא.

המחשה:

3 resource types:

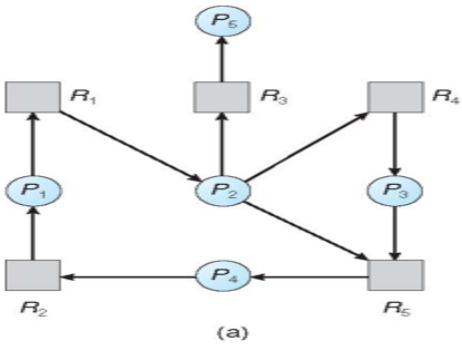
A (10 instances), *B* (5instances), and *C* (7 instances)

- Snapshot at time T_0:

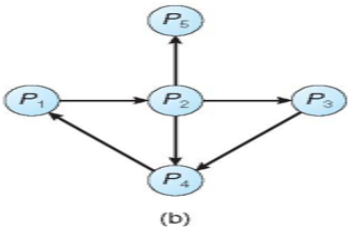
	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

יש שלושה סוגי משאבים שלכל אחד מספר instances שונה.
בזמן t0 , לתהליך P0 מוקצים 0 יחידיות של משאב A ת בתור ה-Max רואים שתהליך P0 עשוי לדרוש 7 יחידות ממשאב A ובתור ה-Available רואים שנותר רק 3 יחידות של משאב A בידי המערכת.

כדי להכיר את הפתרון הבא נכיר גרף מיוחד, לעיתים מתארים את הגרף קצת אחרת , זה נקרא "wait-for-graph"- במקום להקדיש קדקוד מיוחד גם למשאב , מקדישים קדקודים רק עבור התהליכים כך שכל תהליך שמחכה שחברו ישחרר איזה משאב יצביע על חברו:



Resource-Allocation Graph



Corresponding wait-for graph

למשל, P1 מחכה ש-P2 יסיים עם R1 אז הוא יצביע על P2.

עד כה ראינו אלגוריתמים שהתמודדו עם הבעיה בצורה של למנוע ממצב לא בטוח להגיע , הם בדקו האם ההקצאה תכניס את המערכת למצב לא בטוח, גישה חלופית תהיה:

<u>Detection Algorithm-אבחון חסימה הדדית</u>:

הגישה אומרת, ניתן לתהליכים לרוץ כפי רצונם וכשנגלה שהגענו למצב קיפאון , נעיף חלק מהתהליכים על מנת לשחרר את מצב ה-Deadlock.

כדי לאפשר פתרון בשיטה הזאת, על מערכת ההפעלה לדעת לזהות מצב של קיפאון וגם עליה לדעת להתאושש ממנו, לכן נבחין בין 2 מצבים:

1. לכל משאב יש instance אחד:
   נשתמש בגרף wait-for שהכרנו, המערכת נמצאת בקיפאון אם ורק אם הגרף כולל מעגלים.
   לכן על מערכת ההפעלה לתחזק כזה גרף ולבדוק מידי פעם העם מכיל מעגלים.
2. לכל משאב יש כמה instances:
   דרוש אלגוריתם מורכב יותר, נשתמש באלגוריתם הבנקאים אבל עם שינויים קלים, האלגוריתם אינו מורץ רק כאשר תהליך מבקש משאבים אלא מעת לעת (כדי לבדוק האם המערכת נתונה במצב Deadlock).

מבני הנתונים:
work[m]- מערך ששומר כמה יחידות זמינות מכל משאב בידי המערכת (שלא הוקצו).
allocation[n][m]- מערך ששומר כמה יחידות מסוג j הוקצו לתהליך i.
request[n][m]–מערך ששומר עבור כל תהליך i במערכת, לכמה משאבים מסוג j התהליך ממתין.
(איננו שומרים עוד את שאר השדות שהיו באלגוריתם הבנקאים).

הקוד:

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:
   (a) ***Work = Available***
   (b) For ***i* = 1,2, …, *n***, if ***Allocation***$_i \neq$ **0**, then ***Finish*[i] = *false***; otherwise, ***Finish*[i] = *true***

2. Find an index ***i*** such that both:
   (a) ***Finish*[*i*] == *false***
   (b) ***Request***$_i \leq$ ***Work***

   If no such ***i*** exists, go to step 4

3. ***Work = Work + Allocation***$_i$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then P_i is deadlocked

בפסקה 1, מאתחלים את רשימת המשאבים לזמינה.
עוברים על כל התהליכים: אם הוקצו לתהליך משאבים מסמנים את שדה ה-"סיים" שלו ב-False, אחרת מסמנים את שדה ה-"סיים" ב-True.
בפסקה 2, מחפשים תהליך שהוגדר כ"לא סיים" (לא מועמד לסיים) וגם שהמשאבים שהוא צריך נמצאים בידי המערכת.
אם לא מצאנו כזה תהליך יוצאים מהלולאה (break) והמערכת מצויה במצב קיפאון ,עוברים למצב 4.
בפסקה 3,אם מצאנו כזה תהליך מסמנים אותו כ-"סיים" (מועמד לסיום) ומוסיפים את המשאבים שתפס לרשימת המשאבים הזמינים. וחוזרים לפסקה 2.
בפסקה 4, בסוף לאחר שעברנו על כל התהליכים (אם לא היה break לפני) בודקים האם קיים תהליך שהוגדר כ-"לא סיים", אם כן המערכת במצב קיפאון.

כעת נשאלת השאלה, לפי מה נחליט אילו תהליכים להעיף במידה וכבר קפאנו?
מערכת ההפעלה בוחרת על פי הקריטריונים:

- קדימות התהליכים השונים.
- כמה זמן הושקע בתהליך וכמה זמן עתיד להיות מושקע בו.
- כמות המשאבים בהם משתמש התהליך (לפעמים גם הסוג- אם מבוקשים).
- לכמה משאבים עוד צפוי להזדקק התהליך.
- כמה תהליכים נוספים יהיה צריך להעיף פרט לתהליך זה.
- האם מדובר בתהליך אינטראקטיבי או אצווה.
- אילו פעולות בלתי הפיכות התהליך כבר עשה.
- כיצד נשמר מהרעבה.

כשאומרים להעיף תהליך, הכוונה להעביר אותו ל-safe mode והוא ירוץ אחרי ההתאוששות.
ההתאוששות מתאפשרת ב-3 בחינות:

1. בחירת קורבן כפי שהסברנו.
2. מערכת ההפעלה מדווחת למפעיל על קיפאון והוא יבחר אילו תהליכים להעיף
3. לא הורגים תהליכים רק חוטפים מהם משאבים.

פרק 8: זיכרון ראשי

Memory Management-ניהול הזיכרון הראשי:

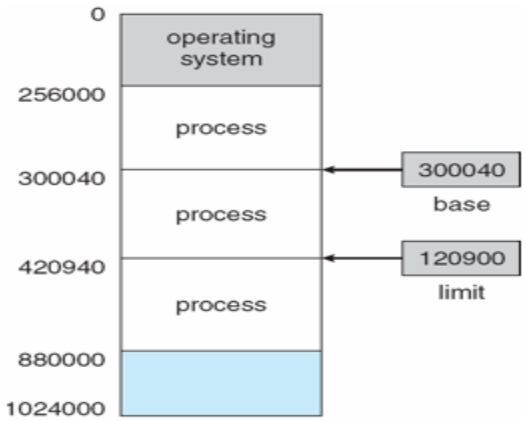
הנחה לפרק זה – כל הזיכרון של התהליך נמצא בזיכרון הראשי , כאמור ה- Cpu יכול לגשת רק לזיכרון הראשי ולזיכרון של הרגיסטרים.

אחד מהדרישות שלנו מה-Cpu הייתה, שיעבוד בצורה רציפה מבלי להימצא בזמן סרק, לכן הכרנו את המושג של Multiprogramming, אם כן ישנה חשיבות רבה לניהול הזיכרון הראשי, ניהול טוב יוכל למצות את תפוקת ה-Cpu בצורה המכסימלית.
עוד מטרה הייתה הגנה על החומרה-תהליכים שנמצאים יחד בזיכרון הראשי מפני גישה של תהליך Pi לנתונים של תהליך Pj, לשם כך מערכת ההפעלה מקצה לכל תהליך טווח כתובות (בזיכרון הראשי) שאליהם יוכל לגשת, כלומר כל תהליך יוכל לגשת רק ל-PCB (מבנה הנתונים שמאחסן אותו) שלו ובכך נגן מפני חדירה של תהליך מסוים לחברו.

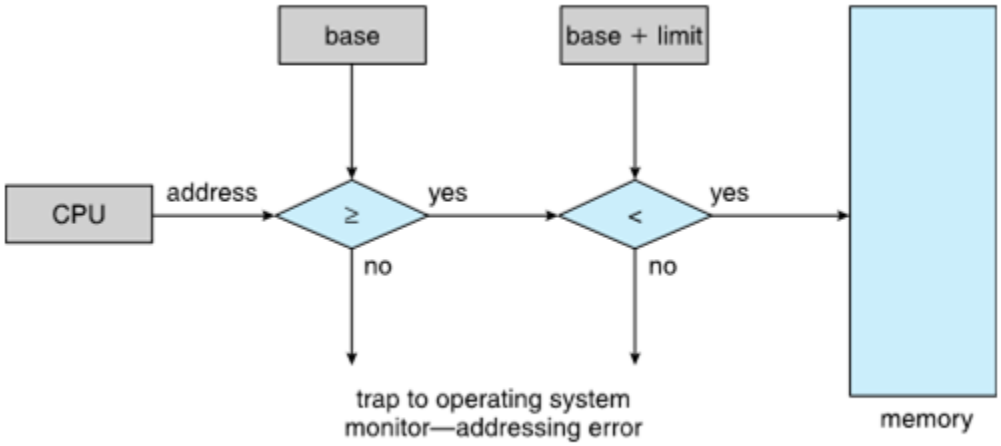
כזכור, תוכניות נמצאות על הדיסק הקשיח (או התקן חיצוני אחר לא נדיף) ובכדי שיהפכו לתהליכים הם צריכים לעבור לזיכרון הראשי (Swap in) .
כאשר תהליך מאוחסן בזיכרון הראשי יש לו כתובת פיזית "Physical address" בה הוא מאוחסן.
הגישה של ה-Cpu לזיכרון הראשי (RAM) היא איטית ביחס לגישה של הרגיסטרים ולכן ה-Cpu משתמש ברגיסטרים על מנת להעביר את התהליכים מהזיכרון הראשי לעיבוד:
Cache- מתאם בין הזיכרון הראשי ובין הזיכרון של הרגיסטרים .

במהלך ריצה של תהליך ב-Cpu ייתכן שאותו תהליך ירצה לעדכן שדה או נתון, בפועל ה-Cpu יצטרך לכתוב את השינוי ב-PCB שנמצא כעת בזיכרון הראשי (כלומר בתחום הכתובות שאותו תהליך קיבל מהמערכת הפעלה).
מי מבטיח לנו שה-Cpu לא יכתוב את הנתון בכתובת אחרת בזיכרון הראשי שאינה שייכת לאותו תהליך?

לשם כך נכיר את המושג : "**כתובת לוגית-Logical address**" :
הכתובת הלוגית מורכבת מ-2 שדות : **<base, limit>** , כאשר תהליך עובר מהזיכרון הראשי למעבד (ע"י ה-cache) , יש 2 רגיסטרים , הראשון base שאליו מוכנסת הכתובת הראשונה שהתהליך תופס בזיכרון הראשי (כתובת ההתחלה שלו), השני limit אל רגיסטר זה מוכנס הגודל של התהליך.
אם נחבר את הכתובת ההתחלתית של התהליך (בזיכרון הראשי) יחד עם גודלו נקבל את הכתובת הסופית של אותו תהליך בזיכרון הראשי.



בצורה כזאת נוכל להגן על כתובות החומרה (להגן שתהליך אחד לא ישתמש בכתובות של תהליכים אחרים בזיכרון הראשי).
ה-Cpu מריץ את התהליך, כעת אם צריך לגשת לאחד הכתובות שבחומרה (שנמצאות בזיכרון הראשי שבו מאוחסנים עוד תהליכים) בשביל לעדכן נתון או מסיבה אחרת, המעבד בודק האם הכתובת שאליה צריך להגיע נמצאת בטווח שבין ה-base ל-limit , אם כן , תתבצע הפעולה. אחרת, לא.



כפי שמתואר באיור, ה-Cpu פולט כתובת במהלך הרצת התהליך אליה הוא צריך לגשת על מנת להביא נתון או לכתוב שם.
הבדיקה הראשונה האם כתובת הנתון גדולה או שווה מכתובת ה-base ? אם כן ממשיכים.
הבדיקה השנייה, האם כתובת הנתון קטנה ממש מחיבור של כתובת ה-base וגודל ה-limit? אם כן אפשר לגשת.

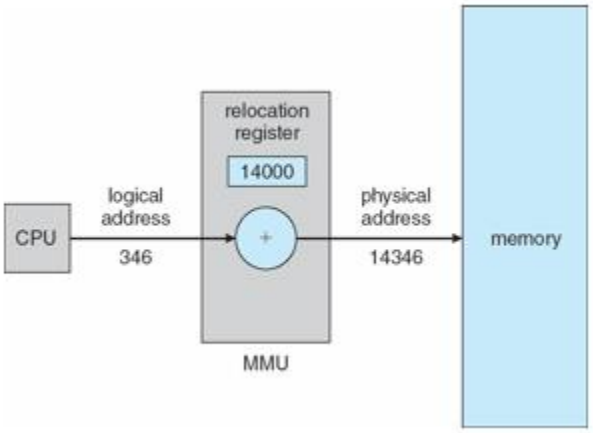
לכל תהליך יש 3 שלבים: טעינת הנתונים, קומפילציה, הרצה.

כאשר תוכנית עוברת קומפילציה (בדרכה להיות תהליך) ישנו הליך שנקרא link שבו מקשרים את הספריות (שעשינו להם include) לקוד שכתבנו.
ישנם 2 סוגי ספריות:

1. ספריות סטטיות- בקומפילציה מועתק כל הקוד של הספרייה לקוד שאנו כתבנו (קוד מנופח בכל תוכנית).
2. ספריות דינאמיות- יש תג (סימניה) איפה שיש link וכאשר התהליך רץ ומגיע לתג הספרייה מיובאת.

יש יתרון לספריה הדינאמית בכך שהקוד המקומפל הרבה יותר קטן (המחיר זה מעט מהירות שאנו מפסידים לגשת לספריה בעת ריצה).
צריך למצוא גשר (התאמה) בין הזיכרון הראשי (שם מאוחסן-הכתובת הפיזית) לבין הכתובת הלוגית (וירטואלית- כאשר נמצא בעיבוד) כדי שהמעבד יוכל לגשת לכתובות הפיזיות, לשם כך יש רכיב שנקרא :

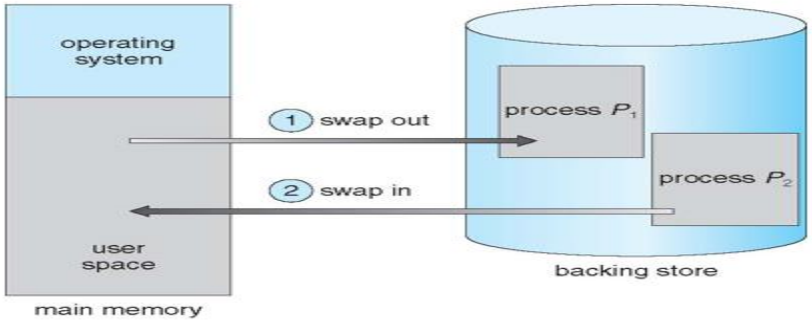
Memory-Management Unit (MMU):
רכיב חומרה שמתאם בזמן ריצה בין הכתבות הלוגיות לבין הכתובות הפיזיות.
אחת הדרכים לממש את ה-MMU היא ע"י רגיסטר ה-base שמעתה נקרא לו "**relocation register**" מהשם החדש ניתן להבין שהוא יהיה אחראי אל התזוזה בתוך ה-Cpu.
כאמור בתוך הרגיסטר יש את הכתובת ההתחלתית של התהליך בזיכרון הראשי (הכתובת הפיזית) .
כעת כאשר המעבד צריך לגשת לאחת הכתובות הפיזיות של התהליך ה-MMU יחבר את הכתובת שיש ברגיסטר עם הכתובת שהמעבד פלט ויחד נגיע לכתובת המדויקת של הנתון (הכתובת הפיזית שלו).



מרחב הכתובות הלוגיות והפזיות תלוי הכמות הסיביות שיש ב-Bus שמחבר בין המעבד ל-MMU .
אם המעבד מחובר ל-Bus של 32 סיביות (יש גם 64) אז אפשר לתאר כתובת רק עד 32 סיביות שגם אותם נצטרך לחלק ל-limit ול-base.
אם נחלק לbase 16 סיביות ז"א שה-limit (שזה הגודל זיכרון המקסימאלי שמוקצה לתהליך) יהיה בגודל 16 ביט שזה לא הרבה, לכן משחקים עם החלוקה (מצד אחד רוצים כתובות ומצד שני זיכרון גדול לכל תהליך).

<u>Swapping-העברת מידע</u>:
כאמור תוכנית צריכה לעבוד לזיכרון הראשי על מנת להפוך לתהליך, ובמערכת שמשתמשת בעיקרון ה-Multiprogramming יש הרבה החלפות של תהליכים במעבד.
אם כן נוצר מצב שיש הרבה תהליכים שרוצים לרוץ "במקביל" ואין מספיק מקום לאחסנם בזיכרון הראשי.

<u>Backing store</u>: העברת התוכנית בין הזיכרון המשני (או דיסק מיוחד אחר שמיועד לכך) לזיכרון הראשי ולהיפך ע"י אלגוריתמים הנקראים swap (swap in- העברה מהמשני לראשי, swap out- מהראשי למשני).



<u>Roll in/out</u>: שיטה נוספת של Swap in/out , שממומש ע"י "priority-based scheduling" או בקיצור עדיפויות, ברגע שתהליך בעל עדיפות גבוהה מגיע , מנהל הזיכרון מוציא החוצה תהליך בעל עדיפות נמוכה (מהזיכרון הראשי אל הזיכרון המשני) וכאשר התהליך בעל העדיפות הגבוהה מסיים, התהליך שהוצא יכול לחזור.

ברוב המקרים התהליך שהוצא יחזור לאותה כתובת פיזית כי ההוצאה שלו הייתה כבר לאחר שמערכת ההפעלה הקצתה לו כתובת פיזית.

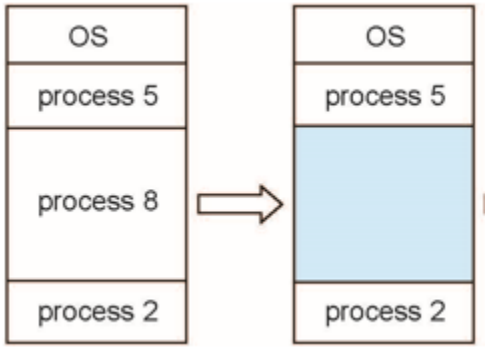
החלפת תהליכים במעבד (Context switch) במקרים כאלו היא גדולה מאוד, (כל החלפה יכולה לקחת הרבה זמן לדוגמא: יש 2 תהליכים שגודל כל אחד 100 mb כשקצב התעבורה הוא 50 mb לשנייה, הוצאת התהליך מזיכרון הראשי ייקח 2 שניות וטעינת החדש ייקח שוב 2 שניות סה"כ 4 שניות.) לכן נדרוש שזמן שכל תהליך יקבל יהיה גדול משמעותית מהזמן שלוקח לעשות Swap (Timesharing) .

Contiguous Allocation-הקצאה רציפה:

מערכת ההפעלה הינה תוכנית בפני עצמה, ולכן זקוקה גם היא לזמן מעבד, אבל המעבד יכול לגשת רק לזיכרון הראשי או לרגיסטרים, אם כן איפה מאוחסנת מערכת ההפעלה? כמובן שבזיכרון הראשי (כי זיכרון הרגיסטרים קטן מידי) בכתובות הנמוכות ביותר.

הזיכרון הראשי הוא מוגבל בגודלו לכן חייבים לעשות הקצאה מדויקת וחכמה של המשאב החשוב הזה.
לכן אנו דורשים שהתהליכים יאוחסנו בזיכרון הנותר (מהזיכרון הראשי שמערכת ההפעלה לא משתמשת בו) באופן רציף, ז"א שלא יהיו חורים בזיכרון הראשי שלא משתמשים בהם (לא רציף).
במצב לא רציף יהיה קשה לנהל את הזיכרון ויהיה בזבוז.

Multiple-partition Allocation-הקצאה מרובה:
כאמור בכל רגע נתון יש מספר תהליכים שנמצאים בזיכרון הראשי ומחכים לזמן מעבד.
נדון בסיטואציה שיש חור בזיכרון ,איך זה יכול להיות? הרי אנחנו דואגים להקצאה רציפה?
ובכן, תהליך שאוחסן בזיכרון הראשי בגלל והסתיים, כעת יש חור במקום שהיה:



נניח שכעת הגיע תהליך חדש, היכן נמקם אותו? באמצע (בחור-hole)? או אחרי התהליך האחרון?
ובכן, יש 3 גישות:

First fit- במקום הראשון הפנוי (בחור הראשון) שגדול מספיק כדי להכיל אותו (סורקים כל פעם מתחילת הווקטור).

Best fit – נמקם את התהליך בחור הכי קטן שמוצא שיוכל להכיל אותו (נצטרך לעבור על כל הווקטור)

Worst fit- נמקם את התהליך החדש בחור הכי גדול שקיים (גם כאן נצטרך לעבור על הכל).

(ה- best fit וה- first fit טובים יותר מה-worst fit כי ב- worst fit יכול להיות מצב שיישארו רק חורים קטנים שלא יהיה אפשר לדחוס שם תהליכים חדשים).

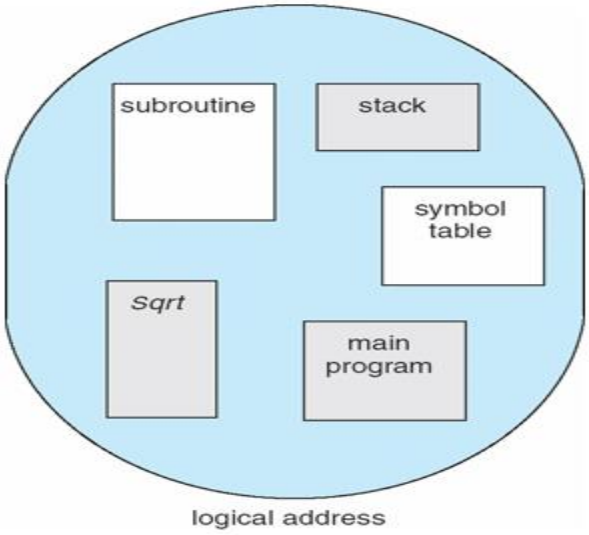
Next First fit- כמו First Fit אבל יש מצביע למקום האחרון בזיכרון שבו בוצעה הקצאה וכשצריך לחפש מקום נוסף אז סורקים החל מהמצביע.

כעת נדבר על מספר פתרונות לניהול חורים בזיכרון:

איחוד-Fragmentation :
כאשר יש המון יחידות של חורי זיכרון כך שאין מקום מספיק גדול באף חור לדחוף עוד תהליך
מצב זה נקרא-**"External fragmentation"** . (קיטוע חיצוני- יש בזבוז מחוץ לתהליך).
עושים איחוד של חורי הזיכרון ומאפשרים כניסה של עוד תהליך (האיחוד מחייב הזזה של שאר התהליכים).

מאידך ישנה סיטואציה שבה מקצים לכל תהליך יותר בתים ממה שהוא צריך בפועל (גם הקצאות חוזרות בבקשות דינמיות של התהליך), ויוצא מצב שבממוצע יש בזבוז של חצי בלוק לכל הקצאה (החלק הלא מנוצל "מבוזבז")
מצב זה נקרא-**"Internal fragmentation"** (קיטוע פנימי- יש בזבוז בתוך התהליך).

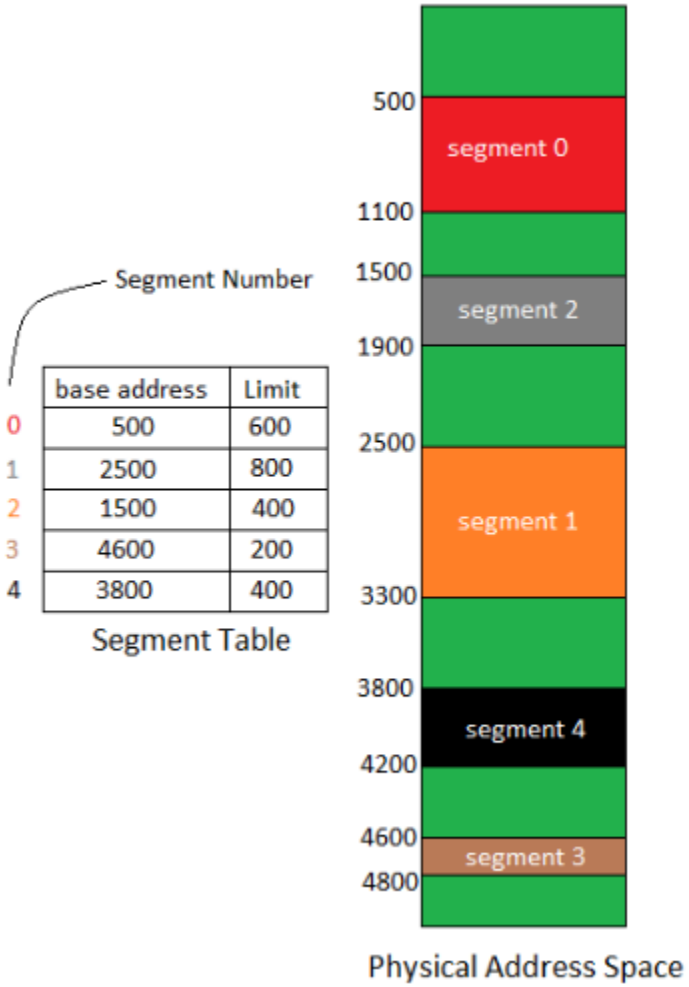
חיתוך Segmentation:
מחלקים את קוד התוכנית לסגמנטים רבים (כל פונקציה בסגמנט נפרד- מפה החשיבות לריבוי פונקציות במקום קוד ארוך ומסובך) וכל סגמנט רץ כתהליך בנפרד ואפשר לדחוף אותו בחורי זיכרון קטנים כך שהסיכוי להיכנס לזיכרון הראשי גודל.



עוד יתרון לרציפות הוא מהירות, כאשר התהליכים ממוקמים באופן רציף בזיכרון הראשי – יותר קל למעבד לבצע גישה לזיכרון ע"י הגדלה ב-1 (בכל פקודה) כי זאת פעולה פשוטה מאשר לחשב כתובת רחוקה יותר.

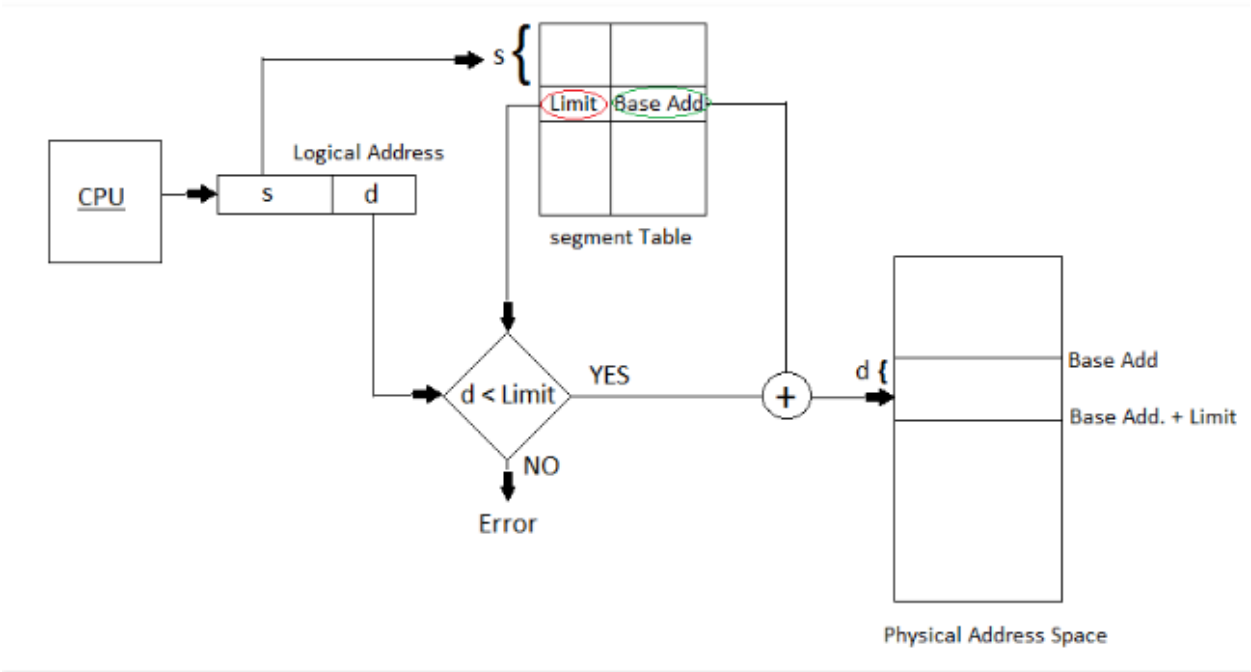
Segmentation Architecture-מבנה הסגמנט:

כפי שראינו, בגישה הנ"ל מחלקים כל תוכנית למספר סגמנטים, לכן עבור כל תהליך ישנה טבלה של סגמנטים.
הטבלה מאוחסנת בזיכרון הראשי (כלומר עבור כל תהליך שנמצא בזיכרון הראשי, קיימת טבלה שמאוחסנת שם),
הטבלה מורכבת מ-2 עמודות (שמרכיבות את הכתובת הלוגית של כל סגמנט) : **base** -הכתובת ההתחלתית של הסגמנט בזיכרון הראשי (הכתובת הפיזית) ו-**limit**- הגודל של הסגמנט.
כדי לשמור את הטבלה בזיכרון הראשי אנחנו זקוקים ל-2 משתנים: כתובת בסיס של הטבלה (**STBR**) וגודל הטבלה (**STLR**).



לעומת זאת הכתובת הלוגית של כל סגמנט מכילה: **<מספר סידורי (segment number), הזחה (offset)>**
המספר הסידורי הוא מספר השורה של כל סגמנט בטבלה וההזחה זה המרחק מראש הסגמנט עד לנתון המבוקש בתוכו.
מסמנים את הכתובת הלוגית כך <s, d>.
כעת, אם נרצה לעבור מכתובת לוגית לכתובת פיזית ,במקום להשתמש במנגנון הrelocation כמו שראינו מקודם, נשתמש בטבלת הסגמנטים באופן הבא:
נניח שתהליך רץ במעבד, והמעבד צריך להביא נתון או לכתוב בזיכרון , אז המעבד יפלוט כתובת לוגית של הנתון <s, d> ,
דבר ראשון לפני שהמעבד מתאמץ הוא יבדוק האם מספר הסגמנט שהוא קיבל בכתובת הלוגית אכן נמצא בטבלה ע"י בדיקה פשוטה –s < STLR , כעת אפשר להמשיך.
כעת מתבצעת בדיקה האם d < limit , כלומר האם הנתון נמצא בתוך סגמנט s ? (בודקים האם ההזחה מראש הסגמנט עדיין משאירה אותנו בתוכו).

אם לא, תיזרק פסיקה מסוג Trap.
אם כן, המעבד ייגש לטבלת הסגמנטים של אותו התהליך וייקח משם את כתובת הבסיס (base) של אותו הסגמנט ויחבר לה את ההזחה (d), התוצאה- הנתון המבוקש.



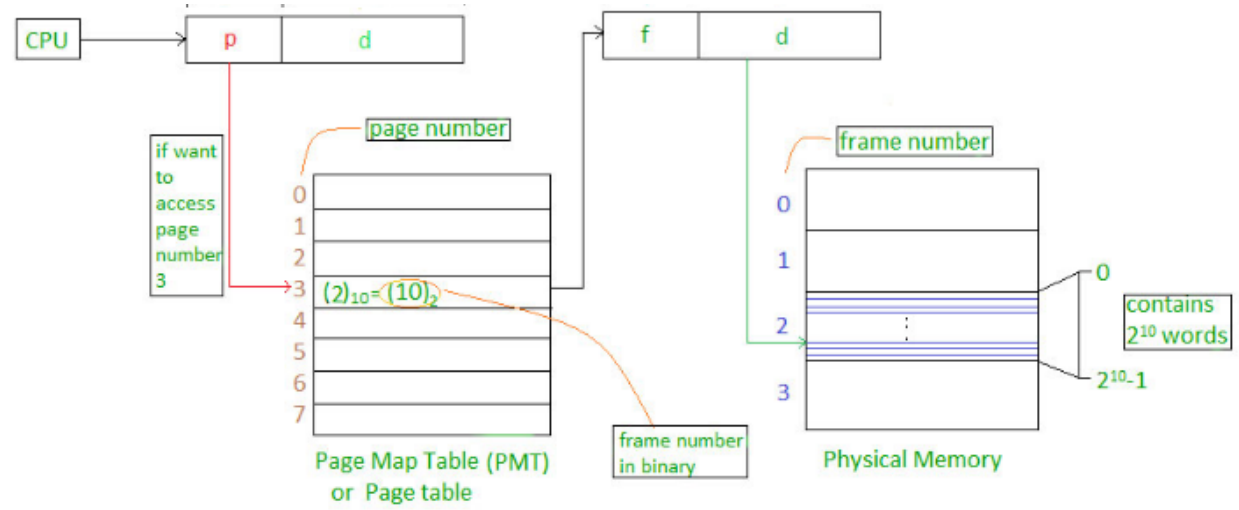
מה שאנחנו מרוויחים בשיטה זו, הימנעות מ- Internal fragmentation (אבל לא מ- External).
חיסרון: על מנת לכתוב או לקרוא יש לגשת לזיכרון פעמיים: פעם אחד כדי לבצע את הבדיקה (d<limit) וגם כדי לחבר את ה-base עם ההזחה, ופעם שנייה כדי לכתוב או לקרוא.

עוד פתרון לטיפול בחורים יהיה:

<u>Paging -חלוקה לדפים:</u>

בגישה הזאת מחלקים את הזיכרון הראשי למספר חלקים שווים שנקרא להם "**פריימים-Frames**" .
(משל-אם יש 64 kb בזיכרון הראשי, נחלק ל-4 חלקים שווים כך שכל Frame יהיה 16 kb).
בצד של הזיכרון הלוגי, מחלקים את הכתובות למספר חלקים שווים שנקרא להם "**דפים-Pages**" .
כך ש- Page יהיה באותו הגודל של Frame.
אם צריך n פריימים כדי לאחסן תוכנית בזיכרון הראשי , אז בצד הזיכרון הלוגי צריך n דפים.
במידה ויש שימוש רק בחלק מהפריים האחרון, לא נקטין אותו אלא יוקצה כל הפריים, יכול להיות בזבוז ובכך לא הצלחנו לפתור את הבעיה של internal fragmentation אבל את external fragmentation כן פותר (בדיוק הפוך מ-Segmentation) אם כן נקבע פריימים לא כ"כ גדולים על מנת להימנע מinternal fermentation.

המנגנון עובד כך:
עבור כל תהליך נחזיק **"page table"** בעלת עמודה אחת שבה יאוחסנו הכתובות של ה-Frames ביחס ל-Pages של אותו התהליך.
ה cpu פולט כתובת לוגית שמורכת מ-2 אלמנטים **<p, d>** :
p-page number: מספר השורה בטבלה בה מאוחסן ה-Page.
d-page offset: ההזחה בתוך הדף.
כעת, נניח שהמעבד מריץ איזה תהליך ובאמצע הוא רוצה לקרוא או לכתוב לזיכרון הפיזי, לכן המעבד פולט כתובת לוגית <p, d> כעת צריך להתבצע תיאום בין ה-Page לבין ה-Frame (ניתן לחשוב על זה כך, התהליך מאוחסן בזיכרון הראשי בתוך יחידות זיכרון שנקראות "Frames" וכאשר התהליך עובר למעבד הוא "מועתק" ליחידות זיכרון באותו הגודל שנקראות "Pages" , המעבד מבצע את זממו על נתוני התהליך המועתק שמאוחסן ב-Pages וכעת הוא רוצה לעדכן את המקור , שכאמור נמצא בזיכרון הפיזי- בזיכרון הראשי) .
אם כן, המעבד ייגש ל-page table וישלוף את הכתובת של ה-Page בזיכרון הראשי.
כעת הוא יחבר לכתובת ששלף את ההזחה (d) בתוך ה-frame ויקבל את המקום הרצוי:



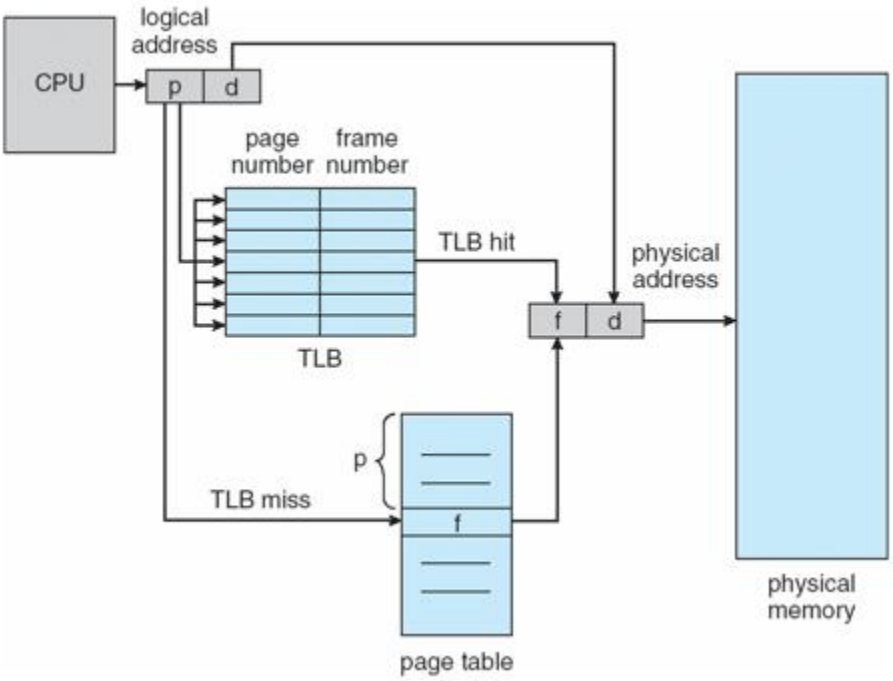
כאן אין בדיקה האם ההזחה משאירה אותנו בתוך ה-Frame כי הזיכרון הפיזי מחולק לחלקים שווים (הoffset בין כל pages קבוע) כך שא"א לגלוש.

כדי לאחזן את טבלת הדפים בזיכרון דרושים לנו 2 פרמטרים:

1. **PTBR-** מצביע לטבלת ה-page
2. **PTLR-** הגודל של טבלת ה-pages.

הטבלה שמורה בזיכרון הראשי (בכתובות הנמוכות) ואם כן יש בעיה , שכל פעם שנרצה לגשת לפריים נצטרך לפנות פעמיים לזיכרון (גם בפתרון הקודם) : פעם אחת כדי לתרגם את ה-Page ל-Frame ופעם שניה גישה לזיכרון להגיע לנתון.

כדי לפתור את הבעיה ,נאחסן טבלה בעלת 2 עמודות (Page, Frame) שהיא אחראית לתרגום ביניהם, בזיכרון שנקרא associative memory נקרא גם **"Translation Look-Aside Buffer(TLBs)"-** זהו זיכרון שהגישה אליו מהירה מאוד.
כך נוכל להקטין את הבעיה מ-2n ל-אפסילון ועוד n .



המנגנון עובד ככה: ה-Cpu פולט כתובת לוגית ואז ניגש לטבלה שנמצאת ב-TLB, אם ה- Page נמצא שם אז הרווחנו וחסכנו זמן.
אם לא, נפנה לזיכרון הראשי ונתרגם את ה-Page ל-Frame (2 גישות לזיכרון).

כעת נוכל לחשב את זמן הגישה האפקטיבי לזיכרון: **"Effective Access Time(EAT)"**:

- **Effective Access Time (EAT)**

$$EAT = (1 + \varepsilon)\,\alpha + (2 + \varepsilon)(1 - \alpha)$$
$$= 2 + \varepsilon - \alpha$$

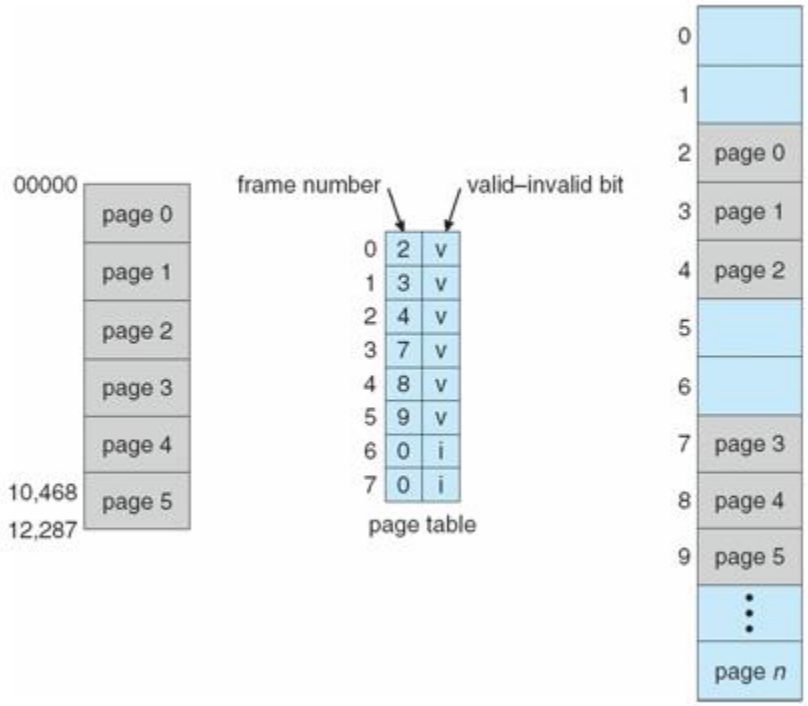
הערה: הנוסחה היא ספציפית לדוגמא : במבחן יחידת זמן לא חייבת להיות 1 ויכולה להיות n ויש גם משחק עם יחידות הזמן יכול להיות גם שיש כמה רמות לטבלת הדפים ואז מספר הגישות לזיכרון יגדל.
לכן צריך להבין את העיקרון והתאם לנתוני השאלה להרכיב את נוסחת ה-EAT.

אפסילון- זמן גישה לTLB.
1-יחידת זמן (גישה לזיכרון)
a – הסיכוי שה- page יהיה בTLB.
לכן ככל שה-a גדול הזמן גישה הכולל יהיה קטן יותר.

מה קורה עם ה-TLB בזמן שיש context switch?
שתי גישות:

1. תמשיך מהמקום הבא הלא תפוס בTLB
2. למחוק את כל הTLB ולייבא את הנתונים של התהליך החדש.

חוץ מהתרגום של דף לפריים נוסיף ביט Valid שאומר אם אפשר לגשת ל-Frame או לא (הגנה נוספת על הזיכרון-נניח שהמעבד פולט כתובת לוגית, בודקים עם הביט של ה Page היא valid או לא ורק אם כן פונים לזיכרון של התהליך.)



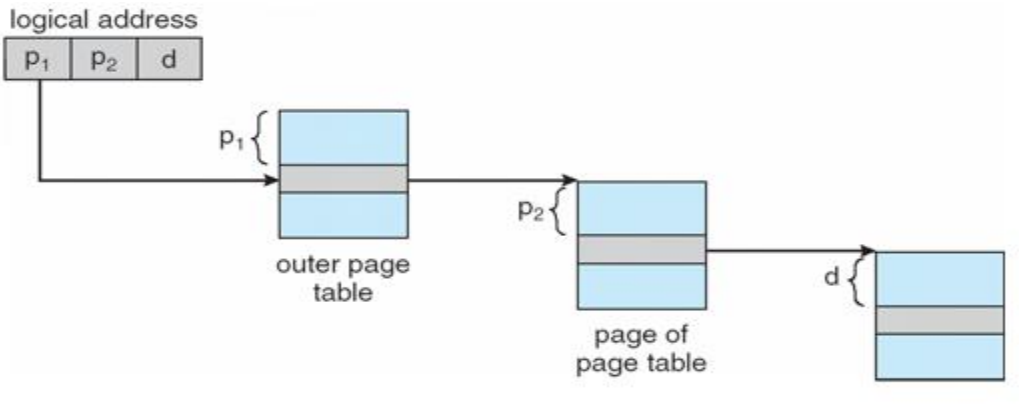
לעיתים Pages של תהליכים שונים חולקים את הקוד בניהם ולעיתים לא תלוי בגישה:

<u>Shared Code</u>- יש עותק אחד (עם הרשאות קריאה בלבד-reentrant) שמשותף בין כמה תהליכים, דומה לתהליכונים שחולקים את הקוד של ה-process שמכיל אותם (למשל פונקציה שנצרכת ע"י כמה תהליכים).
<u>Privat code and data-</u> לכל Page יש עותק של הקוד והנתונים.

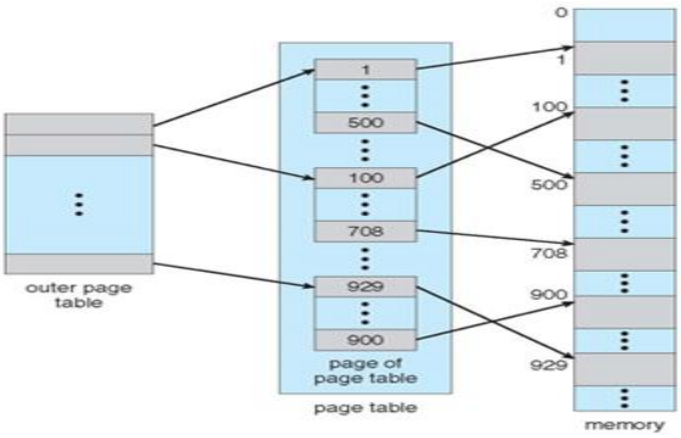
כעת נדבר על:
מבנה ה-Page Table:

נדון בסיטואציה שבה יש מרחב כתובות לוגיות שמוקצה ע"י מחשב של 32 ביט (ה-Bus שמחובר ל-Cpu בעל 32 יציאות) ז"א שיש 2 בחזקת 32 כתובות אפשריות.
נניח שגודל כל Page הוא 4 Kb , אם נחלק זה בזה נקבל מיליון כתובות! , אם כן, מתעוררת הבעיה של מרחב עצום של כתובות שזה גורר שה-Page Table יהיה גדול מאוד (4 Mb).

לכן משתמשים בשיטה של ההיררכיה **Hierarchical page table-Two Level Page Table** :
מחלקים את הטבלה הראשית לכמה טבלאות פנימיות ,כעת הכתובת הלוגית תראה כך <p,p,d> .
ה- P הראשון מפנה לטבלה שבה מאוחסן הPage.
ה-P השני מפנה לאיפה בטבלה נמצא ה-Page.
ה- d זה ההזחה בתוך הפריים.



ככה אפשר לנהל זיכרון רחב יותר.
אפשר לחשוב על זה שהדירות הפרטיות (כל שורה בטבלת הדפים) הפכה להיות בניין (טבלה פנימית).



מובן כעת, ש-32 הסיביות יחולקו ל-3 פרמטרים במקום ל-2 :

- Thus, a logical address is as follows:

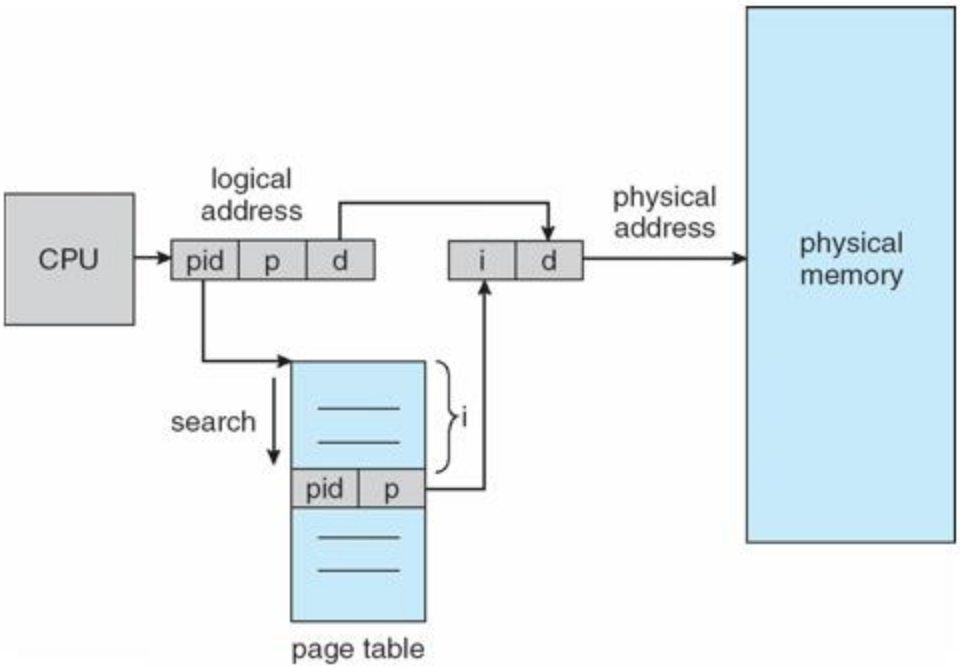
page number		page offset
p_1	p_2	d
12	10	10

כל מערכת הפעלה עושה את החלוקה באופן שונה.
באופן כזה על כל תהליך יש 3 פניות לזיכרון (זה המחיר שאנו משלמים על מרחב העצום הזה של הכתובות)
ולכן החשיבות של ה-TLB גדולה מאוד כאן.

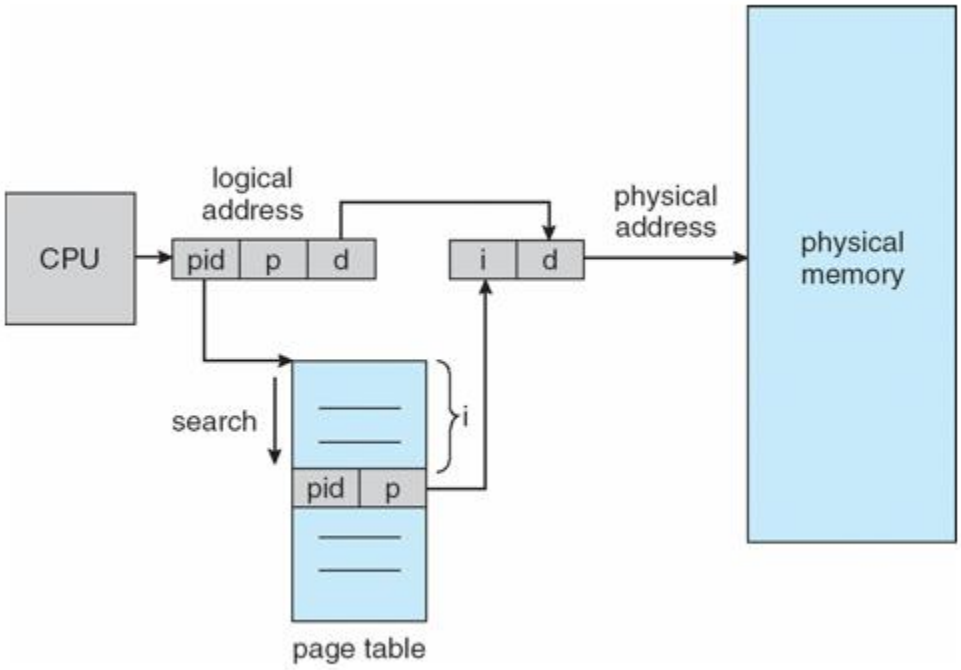
השיטה אינה יעילה עבור 64 ביט בגלל ריבוי הרמות.

ישנה עוד גישה לשליטה במרחב כתובות עצום:

Inverted Page Table:
Page Table אחת לכמה תהליכים במקום לכל אחד בנפרד, בגישה הזאת הכתובת הלוגית מחולקת ל-3 חלקים:
<p, d, pid> יש מספר תהליכים שמשתמשים באותם משאבים (שנמצאים בpage table)
המעבד פולט את הכתובת עם 3 החלקים:
עם ה-P וה-Pid הוא מוצא את ה-Page המתאים בטבלה (Pid זה המפתח גיבוב עבור פונקציה משותפת מסוימת).
העומק שאליו ירדנו בטבלה נקרא i (הכתובת הפיזית של ה-Frame) , המעבד מחבר את i עם ה-d וכך הוא מוצא את הנתון.



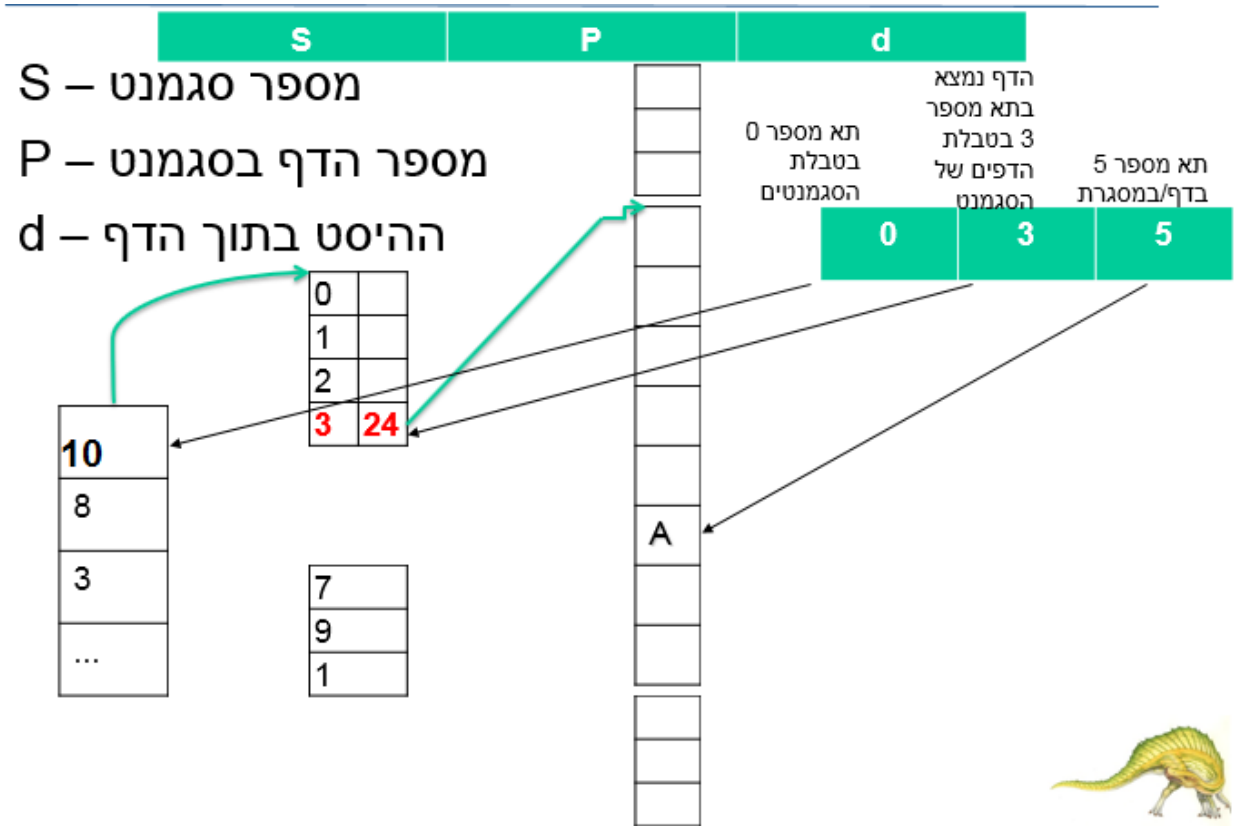
בשיטה זו אנו חוסכים כפילויות כי כל פונקציה מאוחסנת בPage table רק פעם אחת ומקבל מספר Pid וכל פעם שתהליכים שונים רוצים להשתמש בו מוצאים אותו בעזרת הPid.



לסיכום, ראינו את שיטת ה-Segmentation שפתרה לנו את בעיית ה-Internal fragmentation אבל לא את ה- External fragmentation, וראינו גם את שיטת ה-Paging שפתרה בדיוק הפוך.
לכן יש שיטה שמשלבת את שניהם ובכך נצמצם את 2 הבעיות:

Segmented Paged Memory:

הכתובת הלוגית מורכבת מ-3 פרמטרים: <s ,p ,d>



בתוך כל סגמנט יש טבלת Pages.

פרק 9: זיכרון וירטואלי

Virtual Memory-זיכרון וירטואלי:

בפרק הקודם הנחנו שכל התהליכים נמצאים בזיכרון הראשי אבל בשנים האחרונות התברר שהתהליכים גדלו וקשה לאחסן את כולם בזיכרון הראשי לכן חברת "Microsoft" המציאה את עיקרון הזיכרון הווירטואלי המאפשרת ל-Cpu לגשת לזיכרונות אחרים (משני, אינטרנטי).
בנוסף מספר הסיביות של המעבד גדלו (64 סיביות) ולכן מספר הכתובות שאפשר להפיק הוא עצום יותר ממספר הכתובות הפיזיות בזיכרון הראשי , אם כן איך נתאם בין הזיכרון הלוגי לזיכרון הפיזי?
בפרק הזה נטפל בבעיה הנ"ל (שהכתובת הלוגית היא מעבר לזיכרון הראשי ועלינו להביא את הנתונים משם).

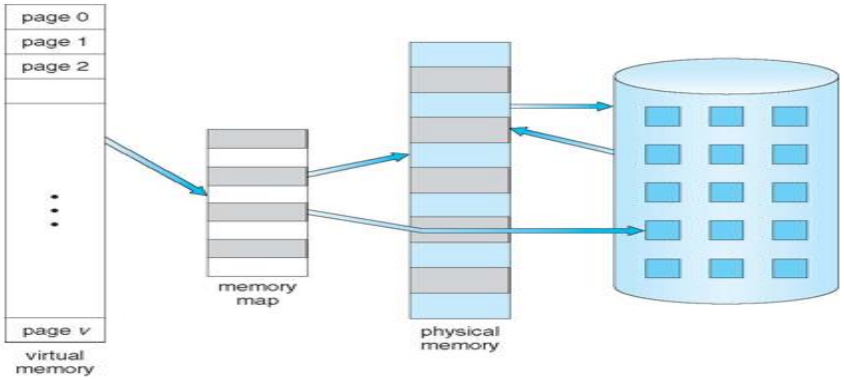
היתרונות בזיכרון וירטואלי:

- ניתן להריץ תוכנית שגודלה עולה על גודל הזיכרון הפיזי.
- ניתן להקצות פחות זיכרון לתהליך מאשר אם היינו טוענים אותו בשלימותו לזיכרון הראשי.
- Context switch משתלם לפעמים.
- יותר תוכניות יכולות לרוץ בו זמנית.

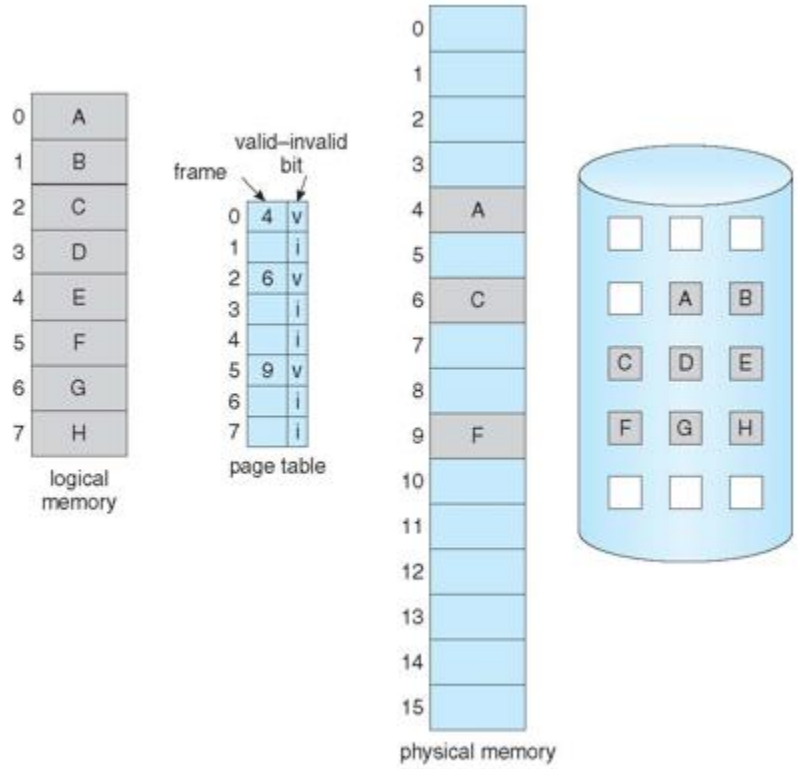
על כל נתון שאנו צריכים לייבא (Caching) מתבצעת פעולת Swap in (בד"כ לוקח זמן - פעולת כתיבה).

אם יש לנו זיכרון לוגי (וירטואלי) פונים קודם ל Page table שכאמור, תפקידו להמיר את ה Page ל-Frame .
ע"י רכיב ה-MMU , ניגשים בזיכרון הפיזי (הראשי) אל ה-Frame, אם הכתובת הלוגית שנפלטה מה-MMU הגיעה אל זיכרון חופשי בזיכרון הפיזי (ריבוע אפור באיור) , אז צריך לייבא את הנתונים מהזיכרון המשני ע"י Swap-in.

האפשריות הן כאלו :
יש מספיק מקום פנוי להעביר את הזיכרון נבצע Swap-in
או שהכל תפוס , נבצע Swap-out לאחד מהתהליכים ואז Swap-in לתהליך שרוצים לייבא.



לכל פריים בזיכרון הראשי מותאם Valid-bit שאומר האם התהליך נמצא בפריים או לא.
(אם לא נמצא נעשה Swap)

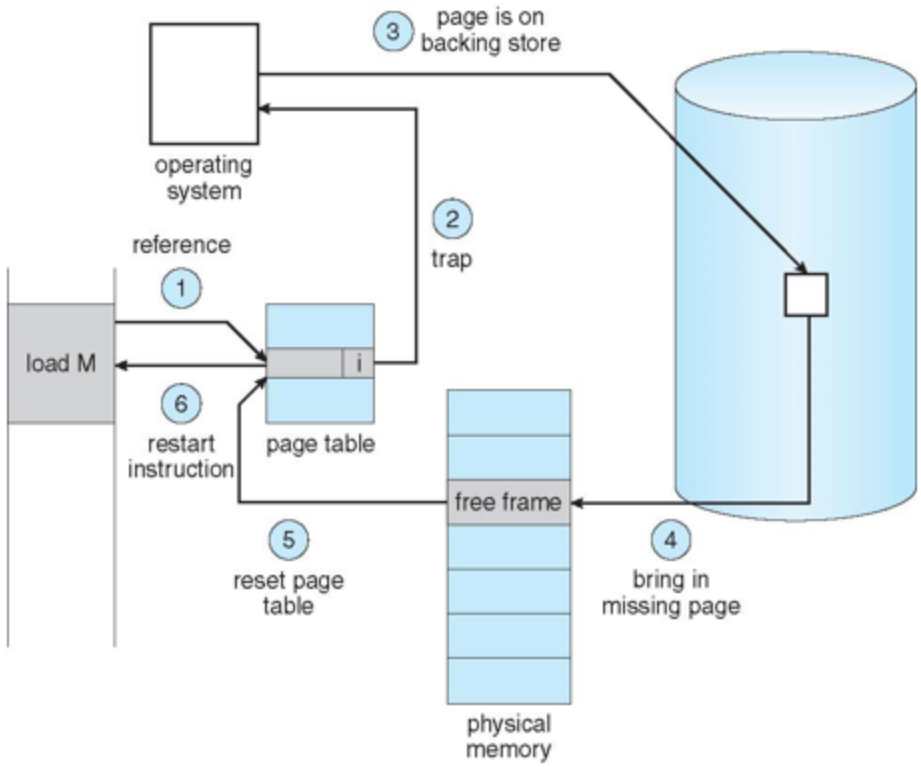


ברגע שיש invalid מתבצעת פסיקה (trap) שאומרת "התהליך לא נמצא בכתובת הלוגית שהופנתה לכן צריך לחפש בזיכרון המשני".

זה נקרא, "**Page fault**" , ומפה התהליך הוא כזה:

1. מערכת ההפעלה מסתכלת בטבלה אחרת כדי להחליט האם היה חישוב לוגי לא נכון של הכתובת או שהתהליך לא בזיכרון.
2. מוצאת פריים פנוי
3. עושה Swap-in
4. מעדכנת את ה Valid-bit ב-V
5. מאתחלת את ההליך מחדש.

Steps in Handling a Page Fault



מה אנו משלמים בתמורה? כמובן שזמן גישה :
נגדיר משתנה P שמשמעותו :
P=1 – יש Page fault
P=0 אין פספוס.

0<P<1 – בעצם P הוא הסיכוי ל-Page fault.

כעת נוכל לחשב את הזמן הגישה האפקטיבי (Effective Access Time-EAT):

EAT=(1-P) *Memory Access +P*(Swap-out +Swap-in)

חשוב: Swap-out לא תמיד קורה(לדוגמא צריך לייבא Page ויש מקום בשבילו בזיכרון הראשי).
דוגמא:

Memory Access Time : 200 nanoseconds

Average page-fault service Time: 8 millisecond
לכן:

EAT=(1-P)*200+P(8,000,000)=200 + 7,999,800

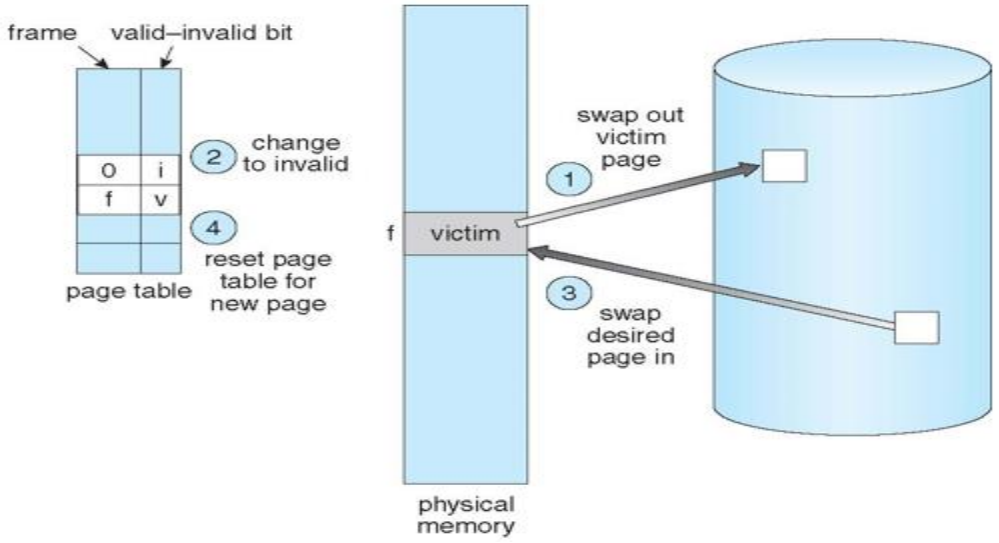
מובן שכדי שהזמן האפקטיבי יהיה טוב צריך P קטן, ז"א פחות סיכוי לפספוס (Page fault).
השימוש במודל הזיכרון ה-וירטואלי אפקטיבי רק כאשר מתקיימת **"מקומיות"** ז"א שמתקיים שהמידע הנדרש נמצא באותו ה-Page (כך לא נצטרך להחליף Page ונמנע מ- Page fault).
מכאן נובעת חשיבות ניהול הזיכרון, באם הוא מסודר התגובה (סיום התהליך) יהיה מהיר הרבה יותר.

מה קורה אם בזיכרון הפיזי (הזיכרון הראשי) הכל תפוס?
נצטרך לתפוס "קורבן" (Page מסוים) ולהעתיק אותו (Swap out) לזיכרון הווירטואלי ובכך נפנה מקום ל- Page הרצוי .
כמובן שנצטרך לבחור את הקורבן בצורה חכמה.
הינה 2 עקרונות למניעת Page fault:

<u>Over-allocation</u> – צריך למצוא איזון בהקצאת הזיכרון הראשי שמקבל כל תהליך היות ואם תהליך P1 יקבל המון מהזיכרון הראשי אז הוא יוכל לשמור על עיקרון המקומיות (ולמנוע page fault) ,אבל בגלל שהקצנו לו המון מהזיכרון הראשי , לתהליך P2 נשאר קצת זיכרון ראשי (כדי לרוץ עליו) והוא לא יוכל לשמור על עיקרון המקומיות (וה-page fault אצלו יהיה באופן נרחב) אם כך יצא שכרנו בהפסדנו, לכן צריך איזון.

<u>modify bit</u>- לעיתים ניתן למנוע העתקות שלמות של תהליך שלם, לדוגמא אם שיננו רק מידע קטן בתוכו, אז נשנה רק את ה-Page הספציפי מבלי להעתיק את כל התהליך, לכן קיים bit שמוצמד לכל Page שאם הוא דלוק ז"א שצריך לעדכן רק את ה-Page הנ"ל (ע"י System Call flash).

תהליך החלפת page:



כפי שמתואר באיור לאחר פעולות ה-Swap in, Swap out מעדכנים את הביט שבשורת טבלת הפריימים ל-Valid.

כעת נרצה להתמקד באלגוריתמים שבוחרים את הקורבן בצורה הכי טובה:

First -In-First-Out (FIFO) Algorithm:

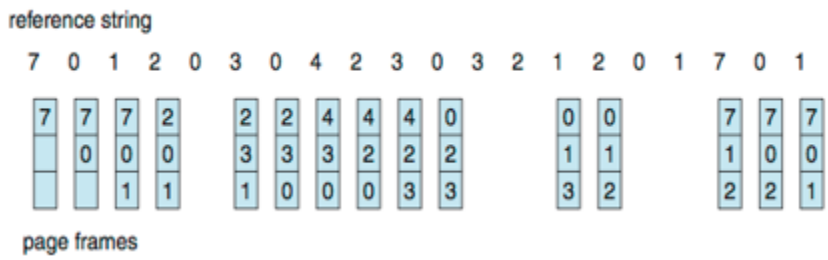
נכיר את המושג "**Reference String**" - סדרת מספרים שנקראת משמאל לימין, משמעות של כל מספר הוא מספר ה- Page שהתהליך היה צריך (לפי הסדר).

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

בשיטה זו נזכור את הסדר שה-Pages הגיעו, כי אם הזיכרון הראשי יהיה מלא ונצטרך לייבא דף אחר, אז נוציא את מי שהגיע ראשון.

נתון ש-3 פריימים יכולים להימצא בזיכרון הראשי בכל זמן.
נתרגל:
בהתחלה צריך את 7 לכן נעשה Swap in
כעת צריך את 0 שוב נעשה Swap in
כעת צריך את 1 , שוב נעשה Swap in

לפי הנתונים הזיכרון כעת מלא, וצריך את 2 , אז לפי עיקרון ה-FIFO נוציא את 7 Swap out, ונייבא את 2 Swap in .
וכן הלאה (צריך לזכור את סדר הגעת הפריימים כדי ליישם את עיקרון ה-FIFO).

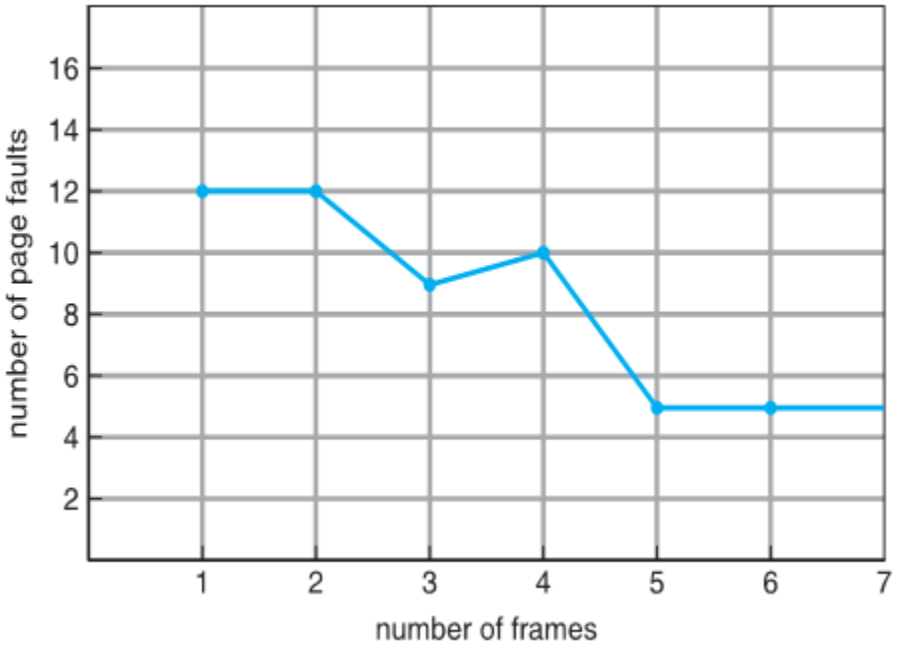


בדוגמא הספציפית הזו הוקצו 3 פריימים לתהליך בזיכרון הראשי, מה יקרה אם נקצה יותר?
האם מספר ה-Page Fault יקטן?
התשובה: לא בהכרח!

תופעה זו נקראת: **Bélády's anomaly** כפי שניתן לראות באיור :

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest page			3	2	1	0	3	3	3	2	4	4
Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest page				3	3	3	2	1	0	4	3	2

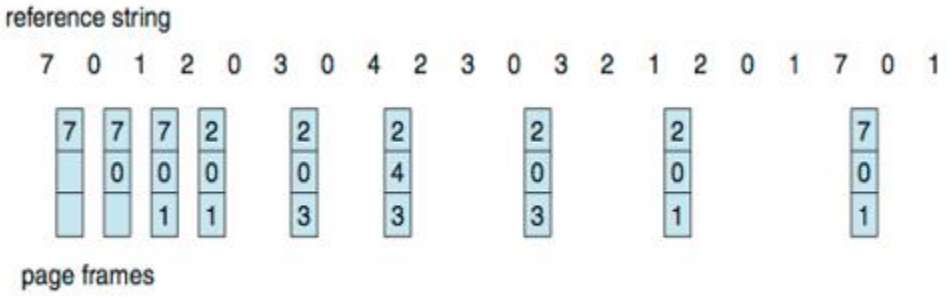
העקומה הבאה מתארת את גדילת ה-Page fault לעומת גדילת מספר הפריימים בכל תהליך:



עוד אלגוריתם (לא ניתן למימוש):

<u>Optimal Algorithm- אלגוריתם אופטימלי:</u>

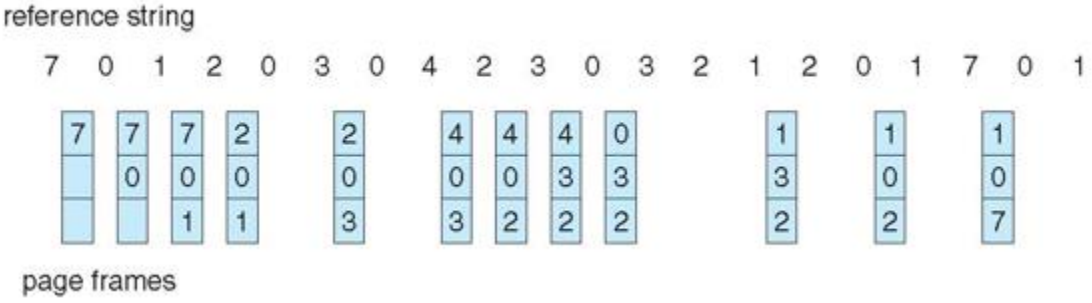
האלגוריתם מניח שמערכת ההפעלה יודעת את העתיד, כלומר יודעת לאילו Pages נזדקק בעתיד.
כמובן שזאת שיטה תיאורטית ולא יישומית , לפי השיטה כאשר הזיכרון מלא וצריכים לייבא דף, מוציאים את הדף שלא היה בשימוש הכי הרבה זמן.



השיטה התיאורטית הנ"ל יכולה להוות מודל לפיתוח שיטה שמבוססת על חיזויים (צריך לדעת להריץ את האלגוריתם למבחן).

עוד אלגוריתם:

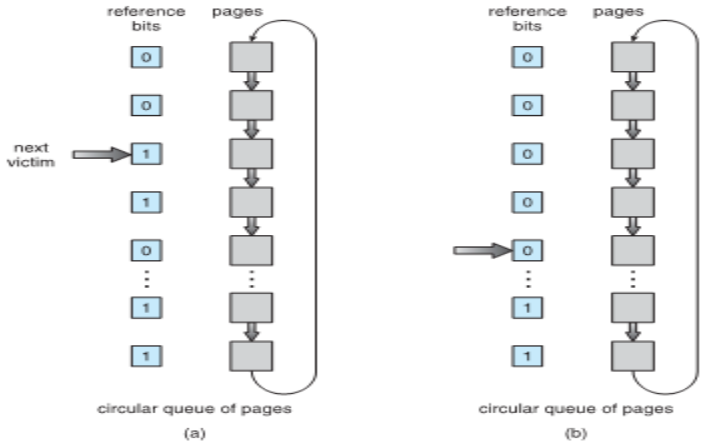
<u>Least Recently Used (LRU) Algorithm</u>:
מוציאים את מי שהשתמשנו בו הכי מזמן.
במבחן אם לא כתוב להניח שהתאים ריקים.



שיטה טובה יותר מאשר FIFO.
(שים לב למעבר בין 0 ל-3).

עוד אלגוריתם:

<u>Second-Chance (clock) Page Replacement Algorithm</u>:
האלגוריתם מפעיל 2 שיקולים:
השיקול הראשון: מימוש FIFO
השיקול השני : מחזיק Bit **(Reference bit)** עבור כל Page שאומר האם כבר נגשנו אליו (אם כן יהיה ערכו 1).
המנגנון עובד כך: בהתחלה כל ה-Reference bits מעודכנים להיות 0.
לכל Page שאליו פונים משנים את ערך ה-Reference bit ל-1 (גם אם נגשנו מספר פעמים)
כעת אם צריך לפנות מקום ל- Page אחר הולכים לראש התור (FIFO) ובודקים: אם הביט של הראשון ערכו 1 אז נותנים לו עוד צ'אנס ומשנים את הביט שלו ל-0 ומעדכנים את **זמן ההגעה שלו לשעה הנוכחית (אחרון בתור כעת)**, בצורה כזאת הוא יקבל הזדמנות שניה כי הוא ייצא רק לאחר כל השאר.
כעת,עוברים לPage השני **שהגיע לתור** (ז"א הם לא חייבים להיות דווקא אחד אחרי השני, למשל כבר הוחלף מישהו ביניהם) אם ערכו 0 אז הוא הקורבן (אם ערכו היה אחד גם לו היינו נותנים הזדמנות שנייה וכו' - ז"א נוציא רק אם הביט הוא 0).
הערה: ברגע שנכנס Page לתור ערך הביט הוא 1.



עוד אלגוריתם יהיה:

Counting Algorithm:

כמו האלגוריתם האחרון , השינוי הוא שבמקום bit אחד שמונה את מספר הפניות יש כמה ביטים לכל Page שמסמנים כמה פעמים פנינו אליו, יש 2 גישות להחלפה של Pages :

- **Lease Frequently Used (LFU) Algorithm-** נחליף על פי FIFO+ ה- Page שאליו ניגשנו הכי מעט פעמים.
- **Most Frequently Used(MFU) Algorithm-**נחליף על פי FIFO+ ה- Page שאליו ניגשנו הכי הרבה פעמים.

כעת נדבר על אופן חלוקת הפריימים לתהליכים (כמה פריימים ירכיבו כל תהליך), כמובן שאופן החלוקה משפיעה המון על רמת ה-Page fault שאנו חווים.

Fixed Allocation:

- **Equal allocation-** שוויוני- מחלקים את מס' הפריימים במספר התהליכים שיש.
- allocation **Proportiona**l – מחלקים באופן יחסי, תהליך גדול יקבל יותר פריימים:

- Dynamic as degree of multiprogramming, process sizes change
  - s_i = size of process p_i
  - $S = \sum s_i$
  - m = total number of frames
  - a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$m = 64$

$s1 = 10$

$s_2 = 127$

$a_1 = \frac{10}{137} \times 64 \approx 5$

$a_2 = \frac{127}{137} \times 64 \approx 59$

m מסמל את מספר הפריימים שיש בזיכרון הראשי.
יש התחשבות בגודל התהליך ,
באיור רואים שתהליך a2 קיבל יותר פריימים כי הוא גדול יותר.

Priority Allocation:

הקצאת הפריימים נעשית על פי עדיפות לתהליך כזה או אחר ולאו דווקא לפי הגודל.

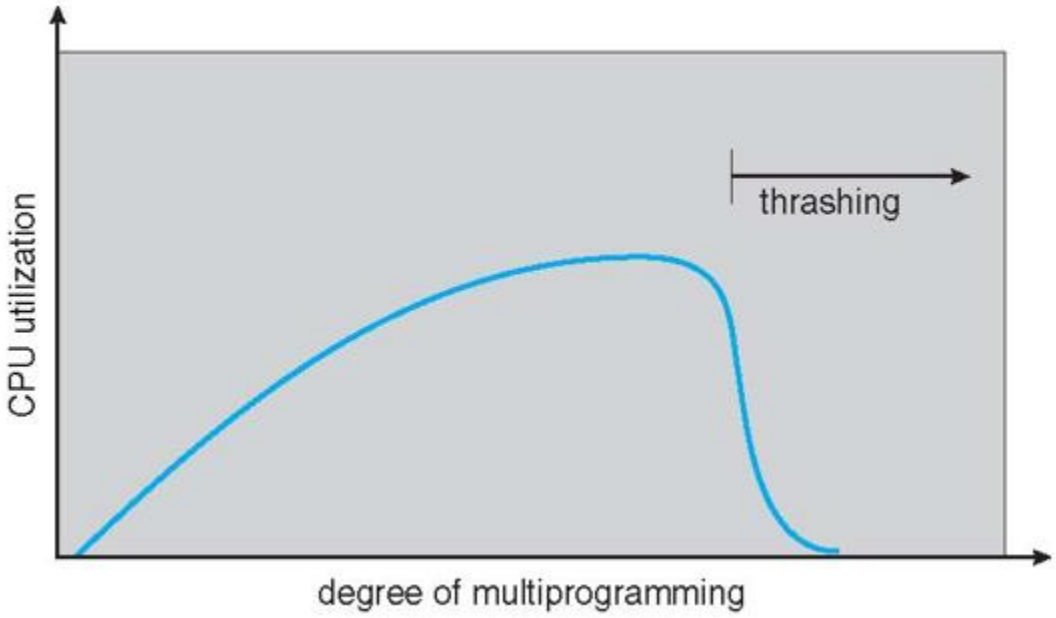
הערה- במבחן אוהבים לתת אלגוריתמים שלא סוקרו בהרצאה (כך לפחות עשו ב-תשע"ח) ולכן יש ללמוד גם על LFU,MFU, מימוש של optimal algorithm ושאר אלגוריתמים שיש.

כעת שראינו את השיטות לחלוקת פריימים לכל תהליך נכיר 2 גישות להחלפת פריימים ב-Cpu:

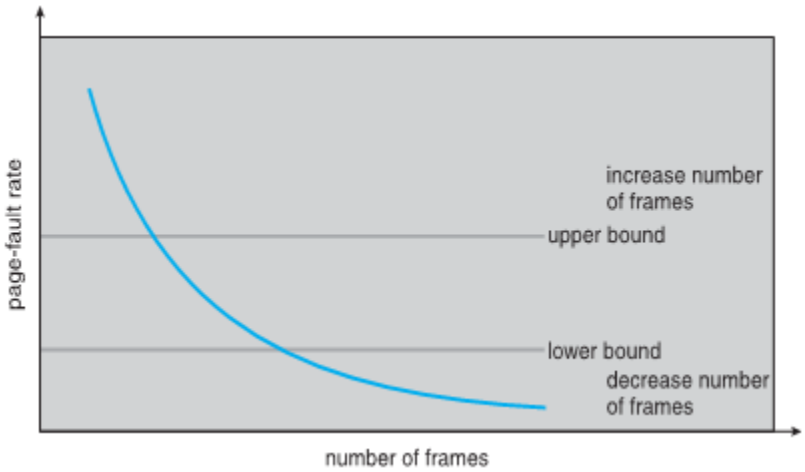
Global replacement- אפשר להכניס פריים של תהליך Pi לפריים שסיים ב-Cpu השייך לתהליך Pj אע"פ שיש עוד פריימים השייכים לPj וכעת באמצע ריצה.

Local replacement- אפשר להכניס פריים חדש רק על חשבון מספר פריימים שהוקצו לPi שכעת בריצה.

החלפת דפים מרובה גורמת לאבסורד, כי כל הזמן ה-Cpu עסוק בהחלפה –"**Thrashing**" ולא מתקדם עם ה-Processes.



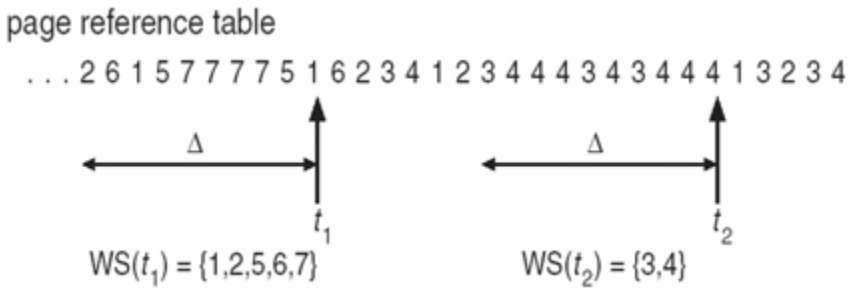
בעיית ה-Trashing (דישה) נוצרת כאשר לכל תהליך מוקצים מעט פריימים, באופן כזה אי אפשר לשמור על תכונת המקומיות, זאת אומרת ש-"החברים" (חלק מהפונקציות) של תהליך מסוים לא נמצאים לידו וכל פעם הוא יצטרך לקרוא להם מחדש (Page fault).



אם כן איך נפתור את הבעיה?
נוריד את כמות העבודות המוטלות על ה-Cpu ובכך נפחית את רמת ה-Multiprogramming וממלא מספר ה-Swaps יקטן והבעיה תעלם.
נרצה לדעת האם מערכת ההפעלה יכולה לחזות מראש שהיא עומדת להיכנס למצב של Trashing.
כדי למנוע את המצב נצטרך שעיקרון ה-"מקומיות" יהיה אופטימלי, ולכן על מנת לחזות Trashing שמתקרב נצטרך לעמוד את רמת המקומיות:

Working-Set Model:

כל תהליך שרץ מריץ מספר Pages ,נקבע זמן מסוים (דלתא) שבו נעקוב אחרי ה-Pages **שמתחלפים** :



ה-Working set הוא ה-Pages השונים בכל מדידה, לדוגמא במדידה הראשונה {1,2,5,6,7} ואילו במדידה השנייה {3,4} ,
כדי לדעת אם יש Trashing בזמן מסוים נבצע מדידה ונבדוק האם Working set size >m
(כאשר m מספר הפריימים הנתונים לשימוש התהליך בזיכרון הראשי) אם כן, אז יש Trashing .
(Working set size- מספר הדפים שבכל Working set).

לסיכום, אם מקצים יותר זיכרון לכל תהליך ה- page fault יקטן, היות ויש יותר סיכויים שלא נצטרך לייבא Page כי יש יותר חלקים מהתהליך שנמצאים כעת בזיכרון הראשי(אבל בכל זאת צריך לשמור על איזון בהקצאה כדי לא לפגוע בתכונת המקומיות של תהליכים אחרים).

Page fault יכול להיגרם גם מכתיבת קוד שאינה תקינה.
כל שפת תכנות יש לה את הייחוד מבחינת ניהול הזיכרון (איך מאחסנת מערכים וכו').
יש חשיבות לאופן כתיבת הקוד , למשל נשים לב בדוגמאות הבאות את ההבדל ב- Page fault כאשר אנו סורקים מטריצה לפי העמודות או לפי השורות (כאשר כל שורה של המטריצה מאוחסנת בדף אחד):

- Program structure
  - int[128,128] data;
  - Each row is stored in one page
  - Program 1

```
for (j = 0; j <128; j++)
    for (i = 0; i < 128; i++)
        data[i,j] = 0;
```

128 x 128 = 16,384 page faults

  - Program 2

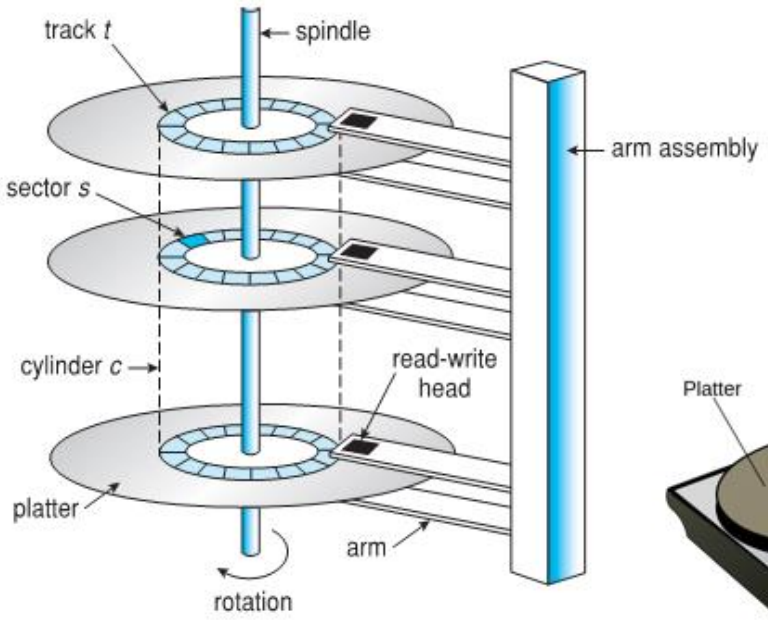
```
for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i,j] = 0;
```

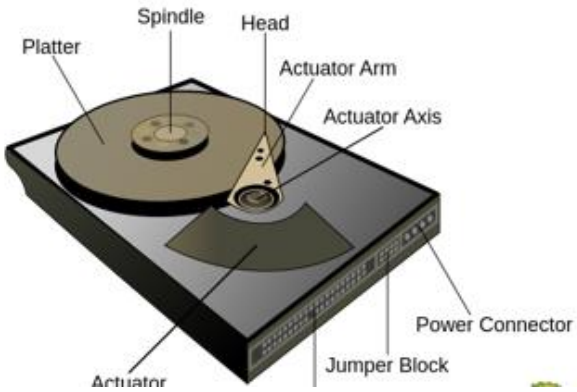
128 page faults

ברור שכאשר רצים על השורות ([i,0],[i,1]…….,[i,127]) כל שורה i זה Page אחד ולכן עדיף לעבור בסדר הזה .
אבל כאשר רצים על העמודות ([0,i],[1,i],……..[127,i]) כל [j,i] נמצא ב- page אחר.

פרק 10: אחסון

דיסק קשיח – יש מערך ארוך שמחולק לטרקים (הדיסק העגול) המחוברים בניהם :
היחידות שמחלקות את הטרק נקראות סקטורים.





לכל דיסק יש קורא וכותב שלעיתים יכולים לעשות זאת בו זמנית.
זהו מנגנון מכני – ולכן רגיש למכות לכן החברות המייצרות שמות בולם זעזועים ובנוסף צריך לקרר אותו (כי יש חיכוך שיוצר חום) לכן יש מאוורר.

הדיסקים הקשיחים מגיעים בכל מיני גדלים, ומסוגלים להכיל המון מידע.
יש מושג שנקרא **"RPM"** - כמה סיבובים בדקה עושה הדיסק, ככל שזה יותר מהיר הדיסק נחשב יותר טוב (כי אפשר להגיע ליותר מידע בזמן t).

Rotational speed [rpm]	Average latency [ms]
15,000	2
10,000	3
7,200	4.16
5,400	5.55
4,800	6.25

(From Wikipedia)

כדי לחשב את זמן הגישה הממוצע של הדיסק – "**Average Latency**" עושים 30,000/rpm .
Seek Time- הזמן שלוקח לזרוע הדיסק להזיז את הראש הקורא/ כותב לסקטור המתאים.
כדי לחשב את זמן הגישה –"**Access Latency**" עושים: (Seek Time / 2)+Average Latency

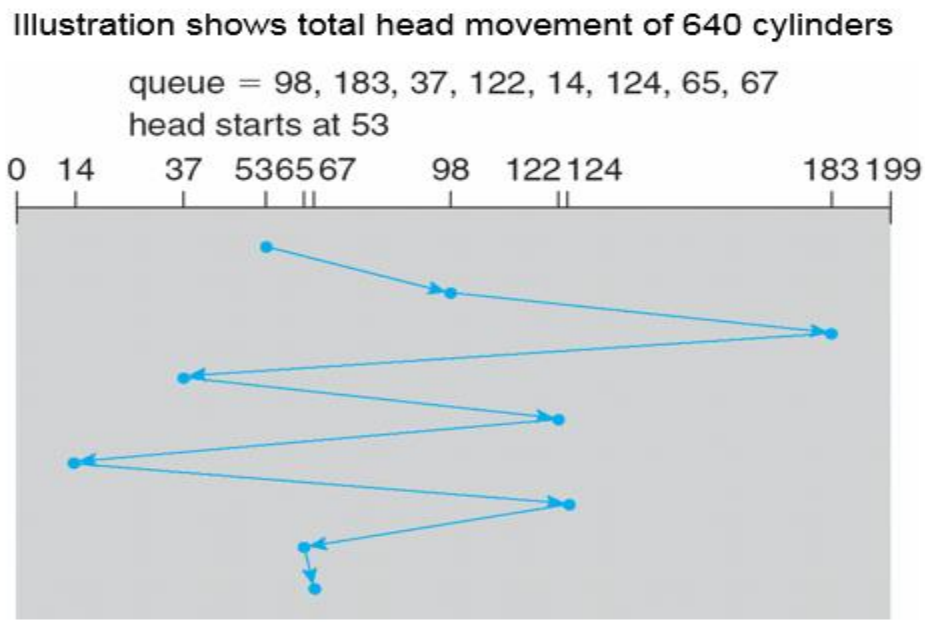
Solid-State Disk (SSD):
זהו דיסק מהיר יותר ולא מכני , לא מייצר חום (אין צורך בקירור) אך פחות עמיד וגדול מה-Hard-Disk.
ה-SSD יקר יותר מהדיסק הקשיח.
להלן טבלת השוואות בכמה פרמטרים:

Attribute	SSD	HDD
Start-up time	Few milliseconds	Several seconds
Random Access time	<0.1ms	~10ms
Noise	No noise	Notable, specially with fans
Temperature	-55°C to 70°C	0°C to 55°C (problems 35+°C)
Shock and vibration	Not sensitive	Susceptible
Weight	Lightweight	3.5" HDD weighs ~700g
Power outage	Sensitive	Stable
Cost per GB (9/2016)	$0.23	$0.03 for 3.5", $0.04 for 2.5"
Power consumption	30%-50% of HDD	2-5W(2.5"), 20W(3.5")

מבנה הדיסק:
יש בלוקים שמסודרים אחד אחרי השני (כמו מערך חד ממדי ארוך שכל תא מכיל בלוק).
יש מתזמן של הדיסק שתפקידו לצמצם עד כמה שאפשר את זמן החיפוש של נתונים בדיסק.
ישנם מספר אלגוריתמים למתזמן:

לאורך כל השיטות שנראה, הבקר מקבל רשימה של סקטורים שמהם צריך לשלוף מידע.
בנוסף נתון, המקום שבו הראש קורא נמצא.

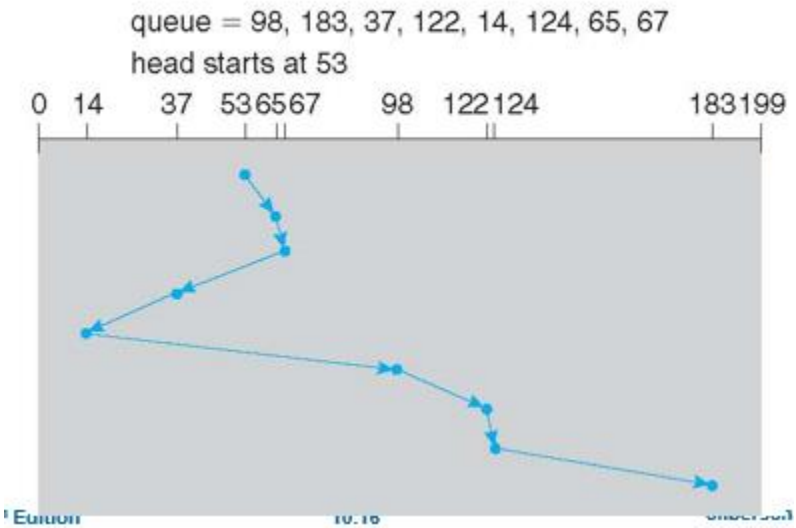
FCFS: נפנה לטרקים לפי הסדר שהגיעו



כדי לאפיין את היעילות מודדים את המרחקים בין כל הטרקים שפנינו אליהם (לפי הסדר של הטיול).
איך מודדים את המרחק?
לדוגמא ההינו ב-98 והלכנו ל- 53 אז מחשבים את הערך המוחלט של החיסור בניהם ואז מוסיפים את המשך המסלול -אח"כ מ-98 ל-183 אז נוסיף את הערך המוחלט של החיסור בניהם וכו' .

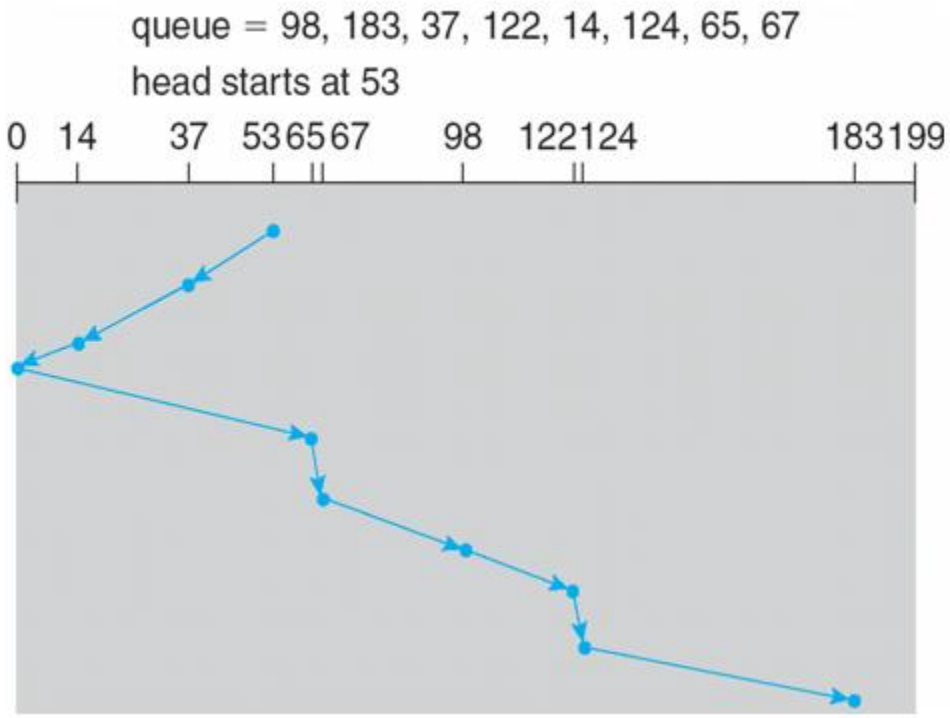
SSTF: ניגשים לטרק הכי קרוב מהנקודה שכעת נמצאים:

:



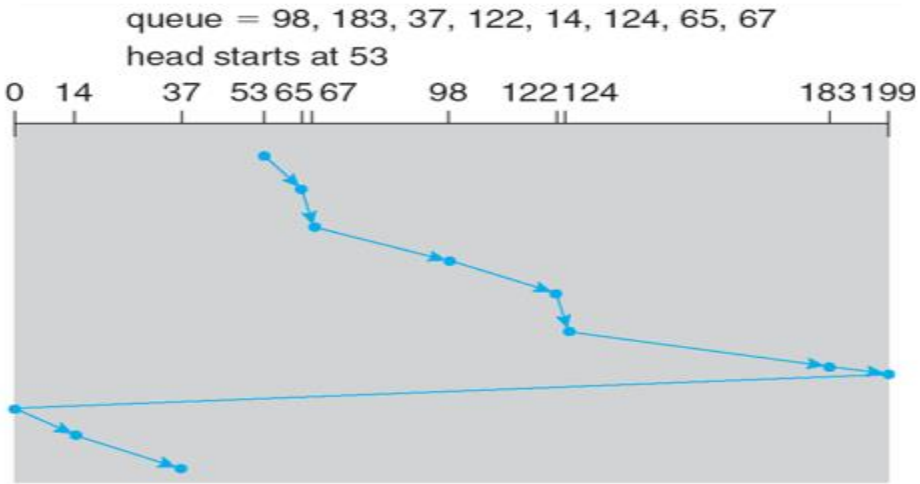
מובן המרחק הסופי יהיה נמוך יותר מהשיטה הקודמת

SCAN: סורקים בכיוון מסוים וכשנגיע לקצה נתחיל לסרוק לכיוון השני:



המרחק כאן נמוך יותר מהשיטה הקודמת.

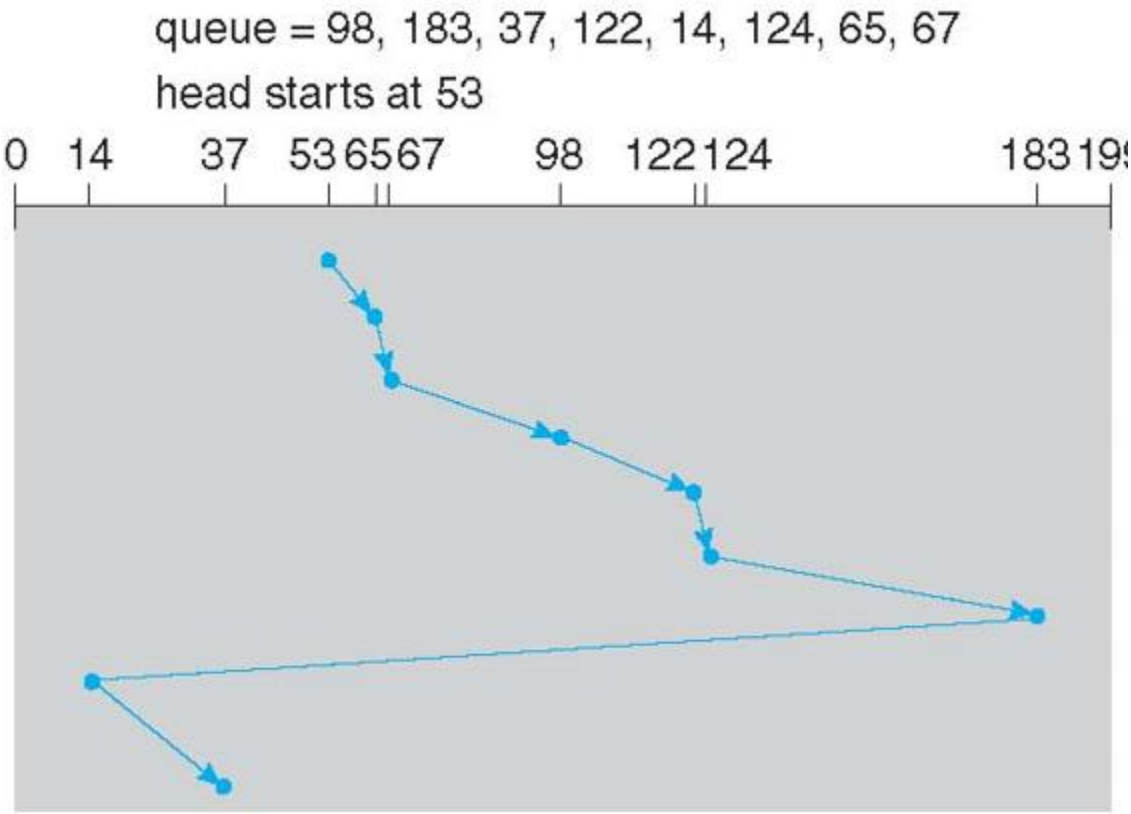
C-SCAN: סורקים לכיוון מסוים מהמיקום הנוכחי (סורקים את הטרקים שבדרך) כאשר נגיע לקצה נקפוץ לקצה השני (מבלי לסרוק) ושוב נסרוק לאותו הכיוון.



המרחק לא יעיל

LOOK:

LOCK זה גרסה של ה-SCAN – הולכים לכיוון מסווים עד **לטרק האחרון שצריך** בכיוון הזה ואז קופץ לטרק הכי נמוך שצריך בכיוון השני:



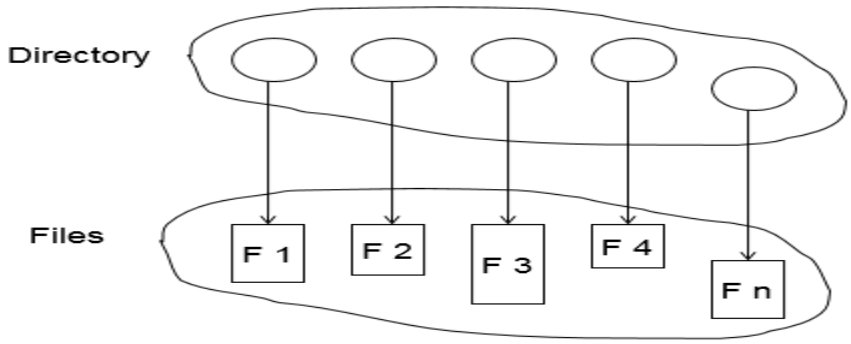
C-LOCK זה גרסה של ה-C-SCAN - סורקים בכיוון מסוים עד לטרק האחרון שצריך ואז קופצים לטרק האחרון שנמצא בכיוון השני (מבלי לסרוק) ומתחילים סריקה לאותו הכיוון החל ממנו.

לסיכום, ה-SSTF זאת השיטה הנפוצה ביותר.

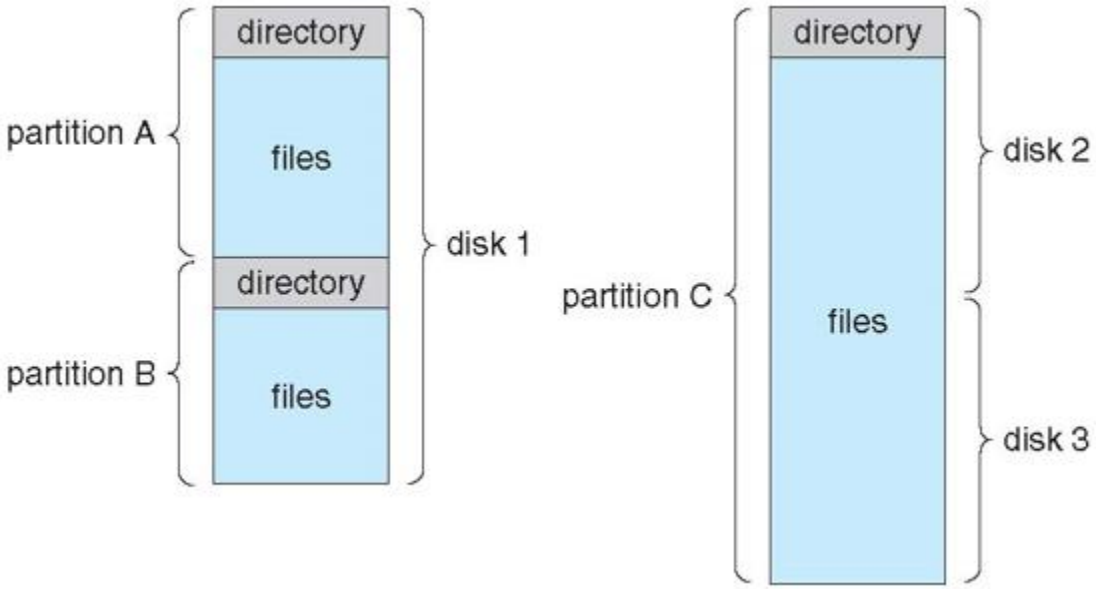
פרק 11-12: ניהול קבצים

כאשר מדברים על קבצים במחשב אפשר לסווג אותם לסוגים : מידע, תוכנית וכו'
לכל קובץ יש **"Attributes" -מידע על הקובץ** : גודל, שם, סוג (המידע על הקובץ שמור במבנה נתונים שמכיל את הקובץ שנקרא FCB).
הטיפוס של הקובץ בד"כ קובע איזו תוכנית תפתח אותו (מסתכלים על הסיומות- py exe, וכו').
מבנה הקבצים : יכול להיות מאוד פשטני ויכול להיות מורכב מאוד.

מערכת הקבצים מאפשרת לנו לסדר את הקבצים בתוך המחשב , המערכת משתמשת בתיקיות שמפנים **("Link")** לקבצים:



זיכרון המחשב מחולק למס' **"Partition"** (חלקים, כל כונן הוא Partition) לכן יכול להיות מצב שתיקיה שמכילה (מפנה ל) i קבצים תאוחסן על מס' Partitions:

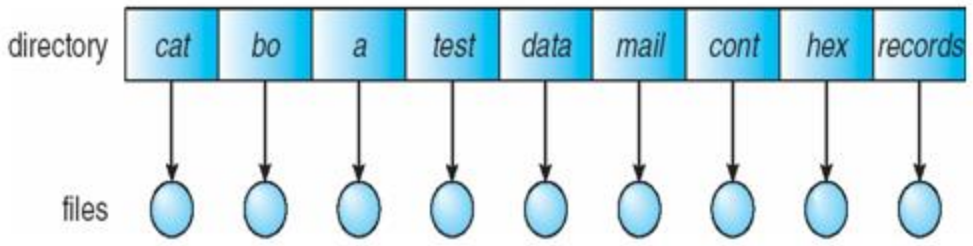


אפשר לומר שתיקיה היא גם קובץ, שמכיל בתוכו את שמות הקבצים שמיוחסים אליו ואת ההפניות אליהם.

בתוך התיקיות ניתן: לחפש קבצים, ליצור, למחוק, לסמן, לשנות את השם ושאר פעולות שמוכרות לנו מחיי היומיום.

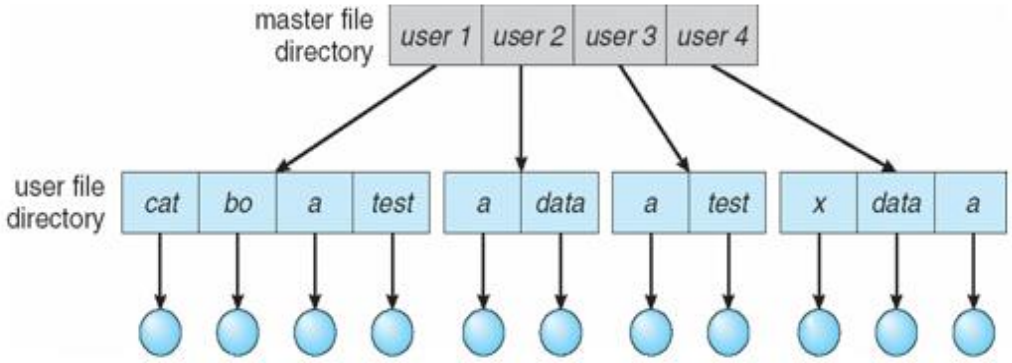
היתרון של התיקיות הוא צירוף המידע תחת אותה הגדרה כמו תיקיה למסמכים ותיקיה למוזיקה, ככה ניתן להבחין במידע לפי סוגיו ובנוסף אפשר להבדיל בין משתמשים.

במחשבים הישנים הייתה רמה אחת, ז"א יש מקום אחד שבו כולם מאחסנים (קיימת רק תיקיה אחת) :



הגישה לא טובה, למשל אי אפשר לשמור 2 קבצים בעלי אותו השם.

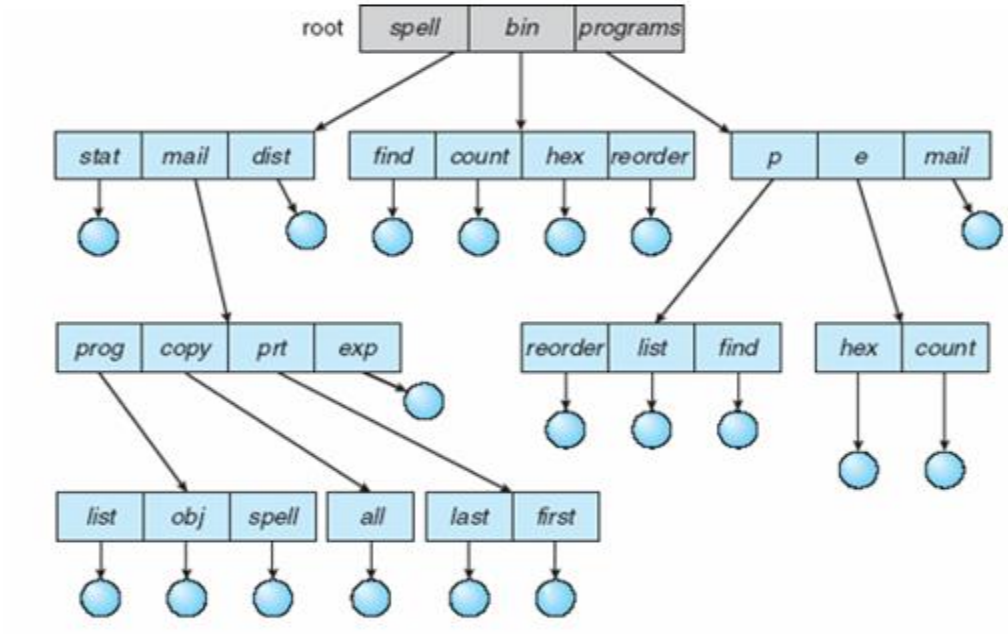
לכן התפתחה השיטה המודרנית שעושה הבחנה בין ה-Users:



בצורה הזאת יש חישוב של הנתיב **"Path"** והחיפוש והשימוש הרבה יותר יעילים.
נשים לב שיש כמה ערכים ב-Root (ז"א שלכל משתמש יש תחילת נתיב שונה).

2 השיטות הנ"ל נקראות "עץ תיקיות".
אבל ישנה בעיה של groping -שכן לא ניתן לחלוק קבצים בין המשתמשים השונים.
לכן השיטות היותר חדשות מאופיינות **בגרף:**

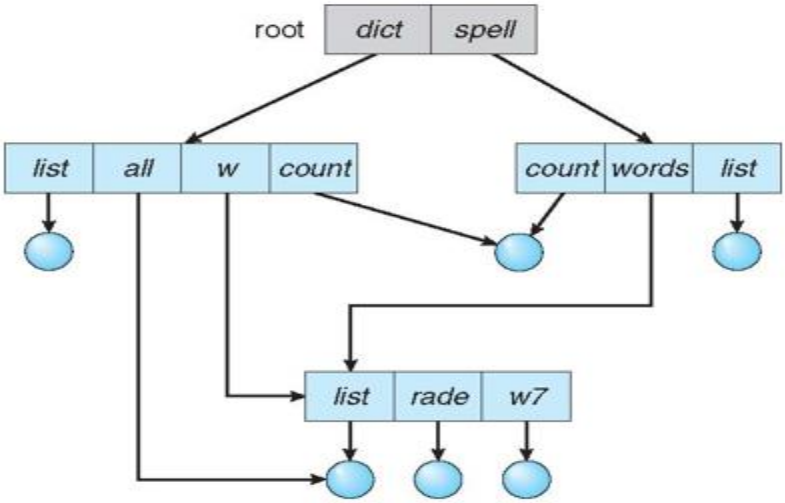
גרף של תיקיות וקבצים מתפרש לכמה וכמה רמות היררכיות:



צורת העץ מאפשרת לכווץ את הנתונים ובנוסף הטיול בעץ נוחה.
דרך אחת לנווט בעץ היא : **-"Path -כתובת"** – למשל: cd/spell/mail/prog .

בהקשר של ה-Path נכיר 2 מושגים:
"**Absolute Path**"- ה-Path מתאר את המסלול מראש העץ
"**Relative Path**"- ה-Path הוא בהתאם למקום יחסי (כמו ב-Unix שם אפשר לכתוב Some Path/. שזה אומר מהתיקייה הנוכחית תרד ל-)

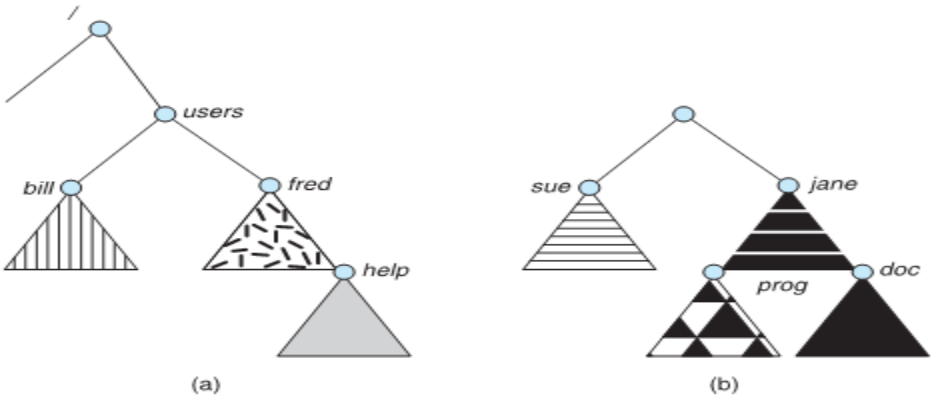
לעיתים ישנו קובץ שמיוחס ל-2 או יותר תיקיות:



בעיקר ממומש כאשר יש כמה Users על מחשב אחד ויש תיקיות משותפות.
העדכונים בתיקיות המשותפות נעשות ע"פ ההרשאות של ה-Users שחולקים אותן.

כאמור התיקיות מפנות מאחת לשנייה ולכן צריך להימנע ממצב של נתק בעץ או מצב שבו קיים מצביע למקום שנמחק-"**dangling pointer**" יכול לקרוא כש-2 משתמשים מצביעים לאותו הקובץ ואחד מהם מחק אותו.

<u>Mounting</u>:
לפעמים יש כמה מכונות שעובדות על אותו המחשב או כמה Hard Disks לאותו מחשב, אם כן איך נאחד את העצים של התיקיות ששייכות לישויות שונות? **(זה חובה במידה ונרצה להעביר מידע בניהם!)**
נקבע מקום (קדקוד) שבו יתבצע "החיבור בין העצים, בצורה כזאת נוכל לטייל ביעילות בין העצים השונים של התיקיות, הקישור הנ"ל נקרא "**Mounting**":



דוגמא נפוצה, כאשר מחברים Disk On key למחשב חייב להתבצע Mounting על מנת שנוכל להעביר לתוכו קבצים(אחרת איך נרשום Path?).

העשרה: כאשר מנתקים DOK בצורה בטוחה מתבצעת System Call "Unlock" שמנתק את ה- Link קודם הוצאת ה-DOK וככה לא נפגע בכבל שיש בו מתח.

File Sharing-שיתוף קבצים:

מערכת הפעלה שמחולקת למספר משתמשים חייבת לספק הגנה "Protection" על הקבצים (והתיקיות שכאמור גם הן קבצים).

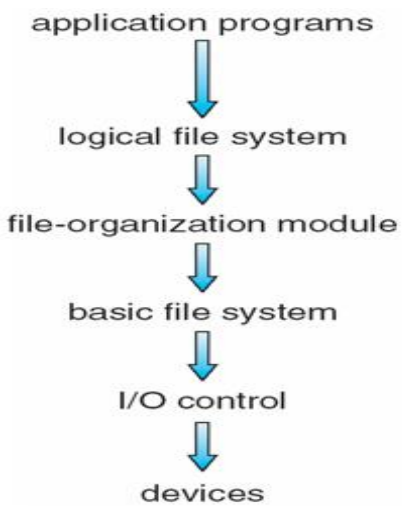
מערכת ההפעלה תאפשר גישה לנתיבים (תציג/ לא תציג קבצים) בהתאם למשתמש User ID או לקבוצת משתמשים Groups ID ותאפשר את הפעולות על הקבצים בהתאם.

(בקורס מדלגים על נושא חשוב שנקרא RAID , לידע כללי הוא חשוב מאוד וגם לראיונות עבודה).

File-System Structure-מבנה מערכת הקבצים:

ה-File System הוא מנגנון שעושר לנהל את הזיכרון המשני (או את ה-Hard Disk) שאותו מחלקים לבלוקים בהם מאחסנים את הקבצים השונים.
מבנה הנתונים שיאפשר לנו לנהל את הקבצים נקרא **"FCB"** (כמו ה-PCB לתהליכים), שע"י ה- **"Device driver"** ינוע בין הבלוקים.

המנגנון בנוי משכבות :



Logical file System- מנהל את המידע על הקובץ – **"Metadata"** (מיקום, שם וכו'), מנהל את התיקייה ודואג לאבטחה.

File organization module – מפענח את הבקשה הלוגית לפיזי.

Basic file system- נותן את הפקודה להתקנים לקרוא או לכתוב בלוקים במקום מסוים.

Device driver-נמצא בשכבה האחרונה, אחראי על שינוע הקבצים .

החלוקה של הקבצים לבלוקים משתנה בין הפורמטים השונים של הדיסק, מה גם שהניהול הוא אחר לכן צריך לקחת בחשבון איזה פורמט אנו בוחרים לדיסק.
צריך גם לקחת בחשבון על אילו מערכות הפעלה אנו חולשים, לא כל הפורמטים מתאימים לכולם.

Each with its own format (CD-ROM is ISO 9660; Unix has **UFS**, FFS; Windows has FAT, FAT32, NTFS as well as floppy, CD, DVD Blu-ray, Linux has more than 40 types,

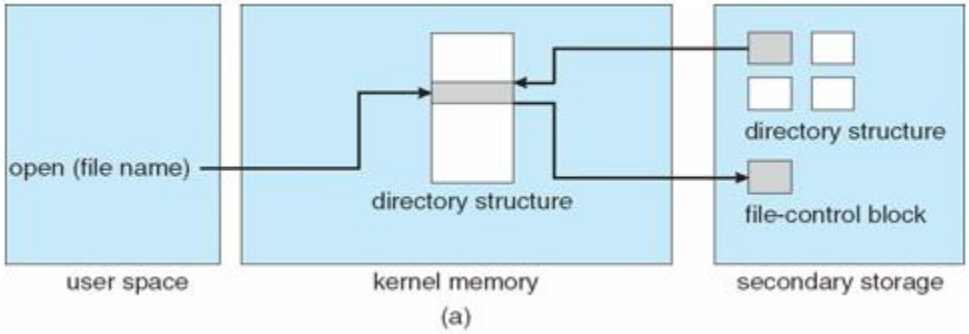
File Control Block(FCB):

מבנה הנתונים שמאחסן כל קובץ:

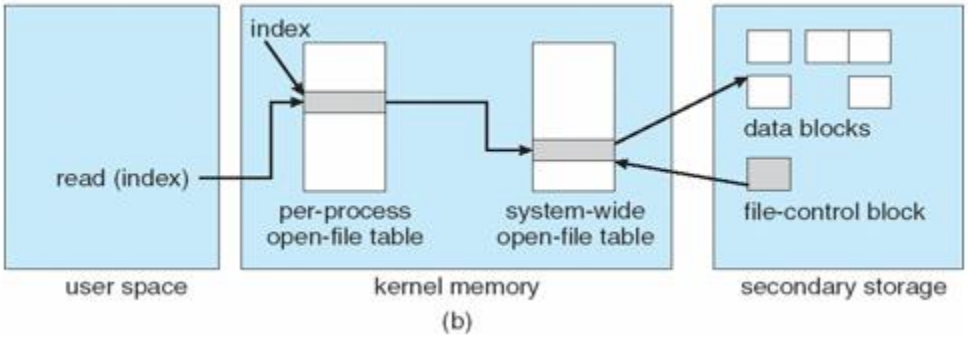
file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

בעזרת מידע זה אפשר לגשת לבלוקים:

למשל כאשר נרצה לפתוח קובץ פונים ל-directory המתאים בFCB וניגשים ל-File-control block שמפנה את מערכת ההפעלה לבלוק המתאים פעולה זו תחזיר index של הקובץ.



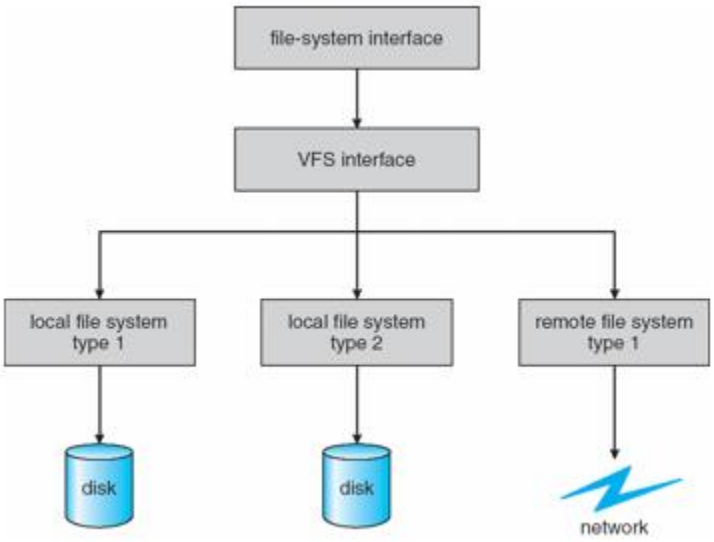
כעת אם נרצה לקרוא מקובץ נשתמש ב-System call שנקרא read(index) (כבר קיבלנו את ה-Index בפתיחה) וכעת נפנה לטבלאות של הקבצים שיפנו אותנו למקום הנכון (הקבצים ששייכים לתהליך הנתון) ונקרא.
הערך שיוחזר , כמה בתים קראנו.



Virtual File System:

גישה עתידנית, בונים שיכבה וירטואלית שמערכת ההפעלה פונה אליה על מנת להעביר מידע להתקנים, והשכבה הווירטואלית היא זאת שפונה להתקנים.

היתרון, חיבור מספר התקנים (מעולמות שונים, כלומר מפוזרים אפילו פיזית) והיכולת להסתכל עליהם כאחד.



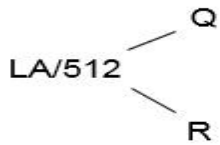
Allocation Methods-שיטות הקצאה:

כמו בניהול הזיכרון הראשי, גם כאן נרצה לנהל את הזיכרון בצורה שלא יהיו בו חורים.

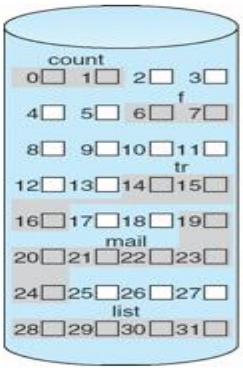
Contiguous-הקצאה רציפה:

הנתונים מוקצים בדיסק, לכל קובץ מקצים מספר בלוקים כך שהנתונים נשמרים בבלוקים עוקבים.
הערה: הבלוקים ששייכים לקבצים שונים לא בהכרח צמודים.
יתרונות: פשוט למימוש ומהיר.
חסרונות: יש לדעת את גודל הקובץ מראש.
גם כאן נצטרך למפות בין הכתובת הלוגית לכתובת הפיזית.
הכתובת הלוגית מורכת מ- <Q,R> כך ש- Q הוא כתובת תחילת הבלוק ו-R זאת ההזחה בתוך הבלוק.
בשיטה זו נצטרך לבצע איחוי מידי פעם על מנת להתגבר על החורים.

- Mapping from logical to physical



Block to be accessed = Q + starting address
Displacement into block = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Linked Allocation-הקצאה משורשרת:

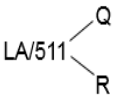
כל קובץ הוא רשימה מקושרת של בלוקים, כאשר הבלוקים אינם יושבים בצורה רציפה בדיסק.
יתרון: לא דורש הזזות.
חסרונות: זמן גישה עצום (הראש הקורא צריך לקפוץ בין מיקומים רחוקים) , יש הגבלה לגודל המצביע ז"א אפשר להצביע מקסימום ל-2 בחזקת 8 כתובות (פחות 1 ל-Null)

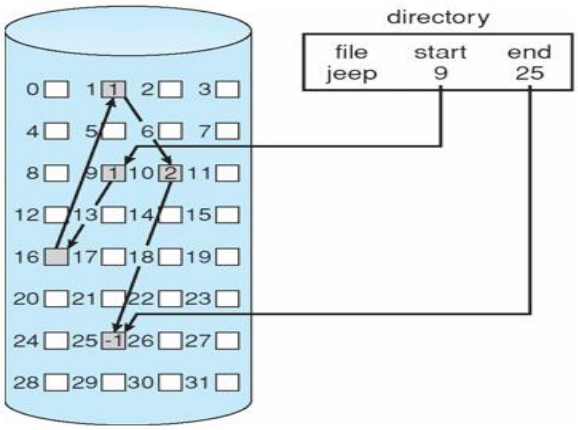
שיפור של השיטה הקודמת יהיה:

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk

block = pointer

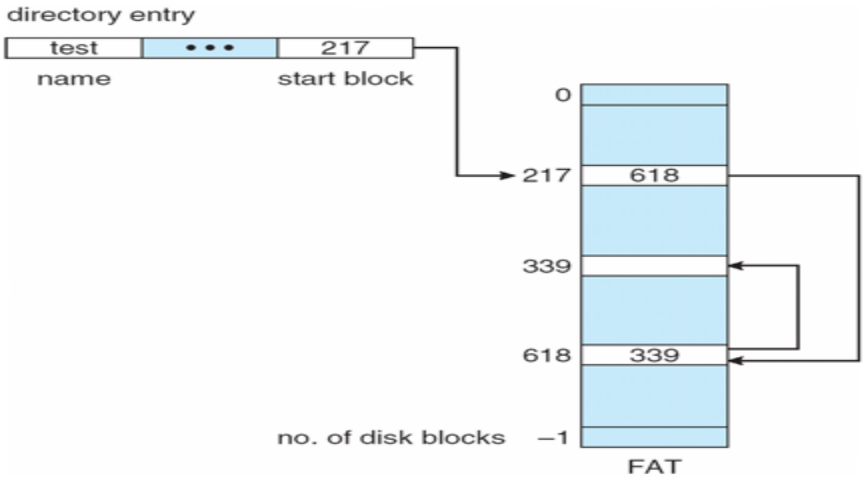
- Mapping





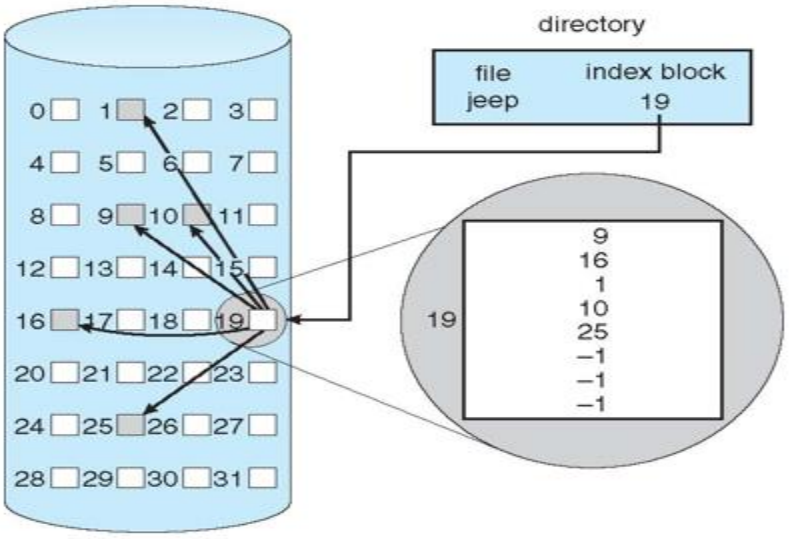
File Allocation Table-FAT:

מחזיקים את שרשרת המצביים בנפרד, ובנוסף מחזיקים טבלה שמכילה "צילום של הדיסק".
לכל בלוק בדיסק תהיה שורה בטלה בלה שתכיל את הכתובת הבלוק הבא בתור:

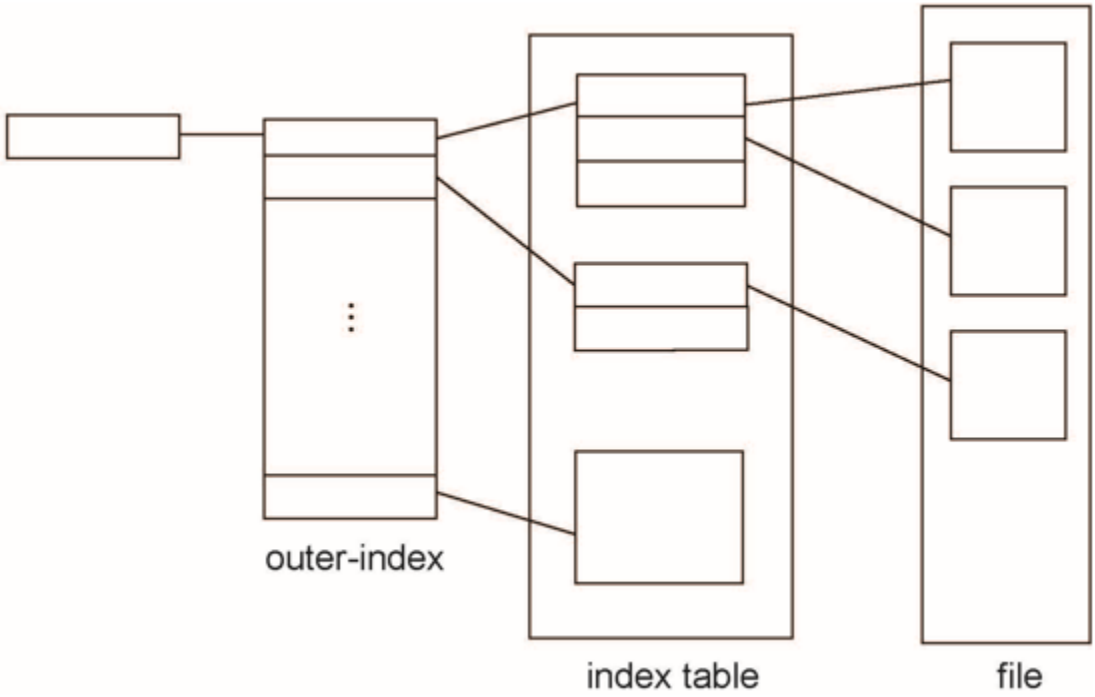


Indexed Allocation-הקצאה לפי אינדקסים:

מחזיקים טבלה של אינדקסים שאומרים לי איפה נמצא כל בלוק.
עבור כל בלוק, יש כניסה בטבלה שמכילה את האינדקסים שמקשרים אותו לשאר הבלוקים:

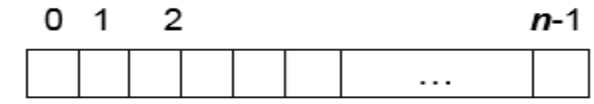


בשיטה זו, לקובץ ששוקל 4 GB כאשר כל בלוק הוא 4 KB נצטרך 1,048,567 אינדקסים! זה עצום ולכן גם פה נממש את שיטת המיפוי ההיררכי :



Free-Space Management:

כדי לנהל את הזיכרון הפנוי (איזה בלוק פנוי) מערכת ההפעלה מחזיק ווקטור שכל תא בו מציין האם הבלוק פנוי או תפוס (1-חופשי, 0-תפוס).



$$bit[i] = \begin{cases} 1 \Rightarrow block[i] \text{ free} \\ 0 \Rightarrow block[i] \text{ occupied} \end{cases}$$

חשוב לדעת את הגדלים השונים למבחן:

2 בחזקת 10 – KB

2 בחזק 20- MB

2 בחזקת 30 -GB

באיור הבא נקבל פרספקטיבה על גודל הזיכרון ומה גודל הווקטור שצריך:

Example:

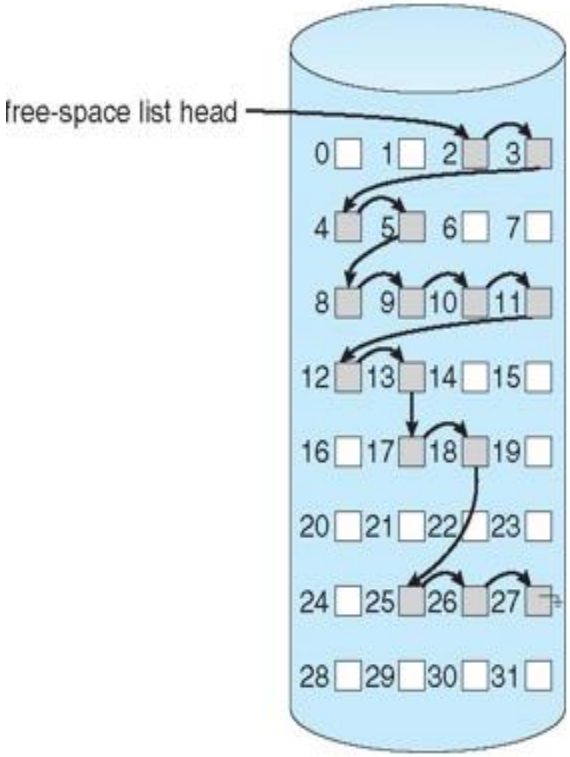
block size = 4KB = 2^{12} bytes

disk size = 2^{40} bytes (1 terabyte)

$n = 2^{40}/2^{12} = 2^{28}$ bits (or 32MB) ——— גודל הווקטור שצריך

if clusters of 4 blocks -> 8MB of memory

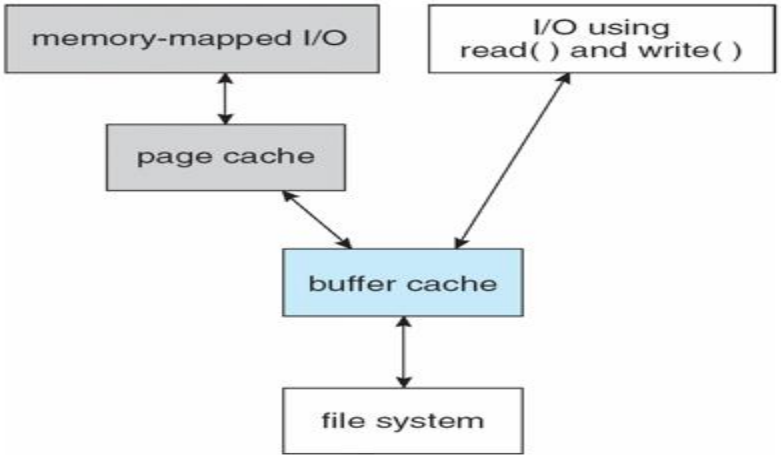
כל טבלה בדוגמא תופסת 8 MG .
וצריך 4 טבלאות לדוגמא הספציפית (ועוד טבלה של 4 bit כדי שמפנה לכולם).



הערה: ברשימה מקושרת של בלוקים אי אפשר לאחסן באחרון כי הוא הציין (מסמן את הסוף)

Page Cache:

רוצים להעביר את המידע שנמצא בבלוקים אל מערכת ההפעלה (שהיא תעביר לתהליכים וכו').

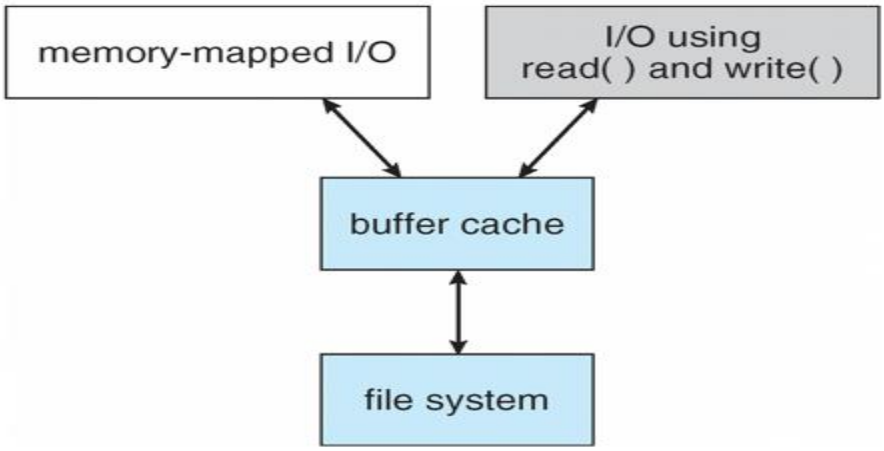


מערכת הקבצים מעבירה את הקובץ ל-Buffer Cache ומשם הקובץ עובר ל-Page Cache ומשם ל-Memory-mapped.

למה צריך את כל ה-Caches האלו?
מכוון שצריך תיאום בין היחידות הללו מבחינת הקצב של העברת המידע (אחד מהיר בהרבה מהשני).

בגישה הנ"ל יש הפרדה בין המערכת הפעלה לבין ההתקן (2 Caches).

יש עוד גישה שמאחדת את מערכת ההפעלה עם ההתקנים:



בגישה הנ"ל מערכת ההפעלה סומכת על ההתקנים (יתרון: מהירות גדולה יותר).

Recovery-שיחזור מידע:
במקרה שיש תקלה ב-Hard Disk ומידע אבד.
רוצים לשחזר ולאמת את המידע ששוחזר.

Consistency Checker- מנגנון שמשווה בין המידע המקורי שאבד למשוחזר (חשוב למשל בבנקים).
Back Up- גיבוי ע"י התקנים חיצוניים, כאשר המידע אבד עושים Restoring ומייבאים אותו.

-בהצלחה-